

KAMP 제조AI데이터셋 활용 권한

No. GB20250818-PU1755498594-243782



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 열처리 공급망 최적화 AI 데이터셋, KAIST(울산과학기술원, (주)에이비에이치), 2022.12.23., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2022, December 23). Heat treatment supply chain optimization AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 열처리 공급망 최적화 AI 데이터셋

업종 : 뿌리(열처리) | **목적** : 공급망 최적화 | **유형** : csv

알고리즘 : 베이지안 신경망 (Bayesian Neural Network)

사용조건 : 콘텐츠 변경 허용

제공기관 : KAIST (수행기관 : 울산과학기술원/(주)에이비에이치)

등록일 : 2022-12-23

이용자 정보

회원 ID : 01soju

다운로드일 : 2025-08-18 17:21:59

활용목적 : 공부

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

「열처리 공급망최적화 AI 데이터셋」

분석실습 가이드북



「열처리 공급망최적화 AI 데이터셋」

분석실습 가이드북



Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	09
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	10
2.1 제조데이터 소개	10
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	12
1) 데이터 및 AI 분석모델 적용	
2) AI 분석모델	
2.3 분석 체험	21
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - Flow Chart	
[단계 ①] 라이브러리, 모듈, 데이터 불러오기	
[단계 ②] 데이터 탐색 및 EDA	
[단계 ③] 데이터 정제(전처리)	
[단계 ④] 주요 변수 탐색	
[단계 ⑤] 모델의 입력 데이터 형태 설정	
[단계 ⑥] 데이터 분리 및 정규화	
[단계 ⑦] 모델 구축/정의 및 학습	
[단계 ⑧] 모델 성능 확인	
[단계 ⑨] 분석결과에 대한 논의 및 해석	
3. 유사 타 현장의 분석 적용	51
부 록	
분석환경 구축을 위한 설치 가이드	53
데이터 품질 전처리 [실습 코드]	66



「열처리 공급망최적화 AI 데이터셋」

분석실습 가이드북

- 필 요 S W : Python, Anaconda – Jupyter Notebook
- 필요 패키지 : Pandas, NumPy, Matplotlib, scikit-learn, TensorFlow
- 분석 환경 : [CPU] Intel i5 3.1 GHz, [RAM] 40GB
- 필요 데이터 : data.csv (생산 및 발주 데이터)

1 분석요약

No	구 분		내 용
1	분석 목적 (현장 이슈, 목적)		다양한 기업과 연계하여 물품을 수주 및 발주하고 공정을 진행하는 제조업의 특성상 네트워크로 이어져 있는 기업의 상황에 따라 수주처와 발주처에 대한 대처와 공정을 위한 원자재 수급에 대한 대처가 유연해야하지만, 현재 기업의 경우 기존의 네트워크 내에 존재하는 업체들에 대한 데이터 분석이 이루어지지 않고 있기 때문에 특이사항이 발생할 경우 유연한 대처가 어려운 상황이다. AI 알고리즘을 활용하여 데이터 분석을 진행하고 특이사항이 발생할 경우 유연하게 수주, 발주 및 원자재 공급을 할 수 있도록 정보를 제공하고 공정 계획을 세울 수 있도록 가이드 할 수 있는 시스템이 필요하다.
2	데이터셋 형태 및 수집방법		- 분석에 사용된 변수명 : 공정 데이터 Product category, 생산량 외 4개 - 수집 방법 : 생산업체의 생산량 및 재고 Data - 데이터셋 파일 확장자 : .csv
3	데이터 개수 데이터셋 총량		- 데이터 개수 : Row 수 (688개), Column 수 (6개) - 데이터셋 총량 : 25 KB
4	분석적용 알고리즘	알고리즘	베이지안 신경망 (Bayesian Neural Network)
		알고리즘 간략소개	베이지안 신경망은 인공지능의 일종으로서, 확률적 모델링을 통해 예측의 불확실성을 고려할 수 있는 알고리즘이다. 베이지안 신경망은 기존의 최대우도 추정법에 기반한 방식으로 최적의 신경망 모수를 탐색하는 것이 아니라, 베이지 정리를 활용한 베이지안 추정법으로 인공지능의 모수가 따르는 확률 분포를 학습함으로써 확률적인 예측을 가능케한다. 이는 결정론적(고정적)인 예측을 수행하는 기존의 인공지능 알고리즘에 비해 예측에 대한 확률적인 해석과 신뢰도를 분석할 수 있게 하는 장점이 있다. 또한, 학습 시 활용하지 않은 데이터 혹은 잘못된 입력 데이터에 대해 보다 강건한 예측을 수행할 수 있는 장점이 있는 알고리즘이다.
5	분석결과 및 시사점		- 과거 시점의 데이터를 기반으로 제품의 미래 출고량을 예측했을 때 본 실습에서 활용한 베이지안 신경망이 높은 성능을 보임을 확인하였다. - 베이지안 신경망을 활용하여 열처리 공정 제품의 수요 예측 실습을 통해, 확률적인 모델링을 수행하고 불확실성을 고려한 모델을 구축할 수 있다.

2 분석 실습

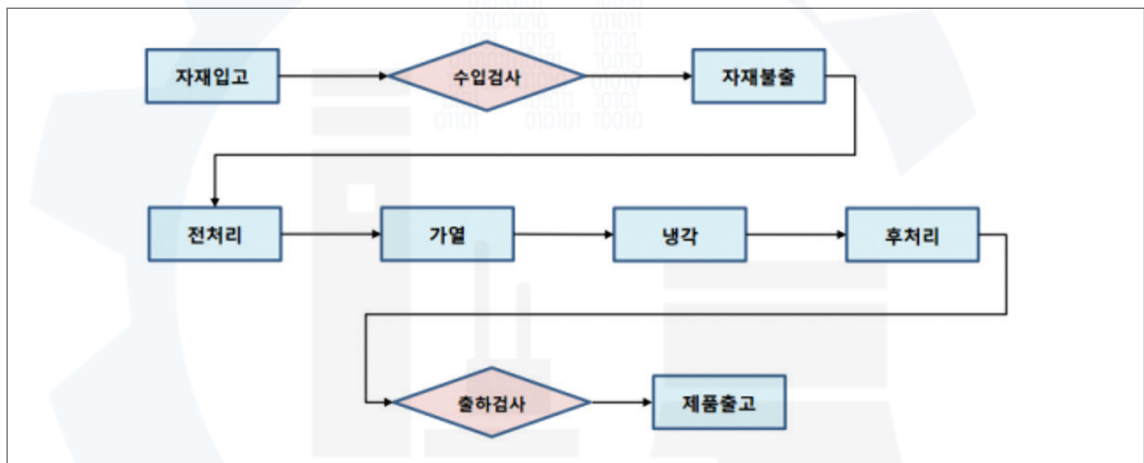
1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

• 열처리(tempering treatment) 공정

- 열처리 공정은 금속을 활용한 제조 공정의 하나로서, 가열 및 냉각 과정을 통하여 금속 재료의 성질을 개선 및 변화시키는 목적을 가진다. 열처리를 통해 성질이 개선된 금속 재료를 활용하여 부품을 생산하는 등 이후의 제조 과정에도 영향을 미칠 뿐만 아니라, 전체적인 공정의 중간 단계로서 원활한 생산 및 공급이 요구되므로 중요한 공정이라고 할 수 있다.



[그림 1] 열처리 공정 흐름도

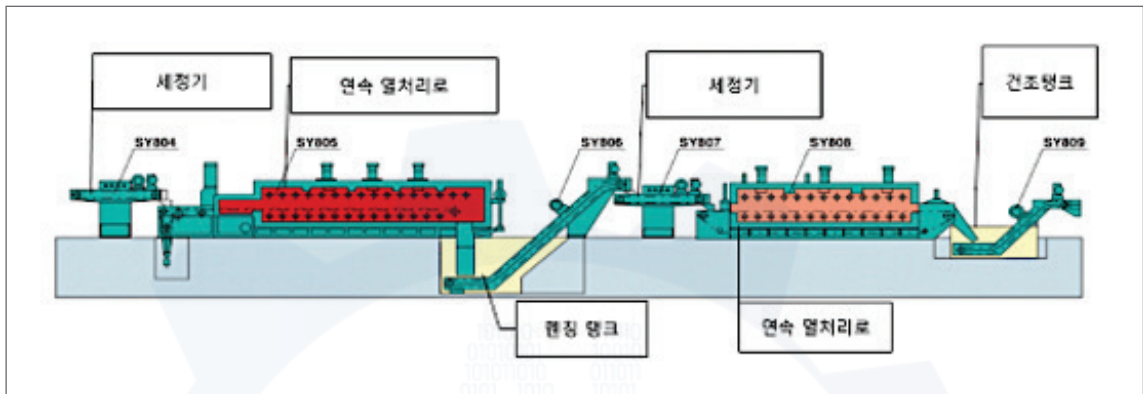
• 열처리 공정의 종류

- 열처리는 가열 및 냉각 방식에 따라 세부 공정, 작업 방식 및 설비가 달라지며 크게 다음과 같이 분류된다.
 - ① 담금질(Quenching)
 - ② 풀림(Annealing)
 - ③ 뜨임(Tempering)
 - ④ 불림(Normalizing)



• 열처리 공정 설비

- [그림 2]는 연속식 열처리로의 예시로 세정과 열처리, 퀀칭, 건조 등을 단일 설비에서 진행할 수 있다.
- 열처리 온도 균일성 및 불순물 제거를 위해 세정 과정을 전처리 과정으로 진행하며, 열처리 후 금속 소재에 원하는 물리/화학적 특성을 부여하기 위해 퀀칭이 진행된다. 잔여물 세정 후 추가적인 열처리가 진행된 후 건조과정을 거쳐 공정이 완료된다.



[그림 2] 열처리 공정 설비

• 열처리 공정의 단계

- 다양한 열처리 공정 중 탄소강 소재 열처리 사례를 예로 공정 설명을 진행한다.
 - ① 담금질 공정 : 금속 부품의 요구되는 경도 및 강도를 높이기 위한 공정으로서, 특정 온도까지 재료를 가열하여 온도를 올린 후 급랭시킴으로써 변화한 금속 특성을 유지시키는 방식으로 진행된다.
 - ② 뜨임 공정 : 담금질을 마친 후 입도를 변경하여 금속 내 조직을 곱게 구성하기 위한 작업으로, 담금질과 마찬가지로 입도를 구성하기 위한 적정 온도까지 가열한 후 입도에 맞게 강온 속도를 조절하며 공기 중에서 식히는 과정을 거친다.

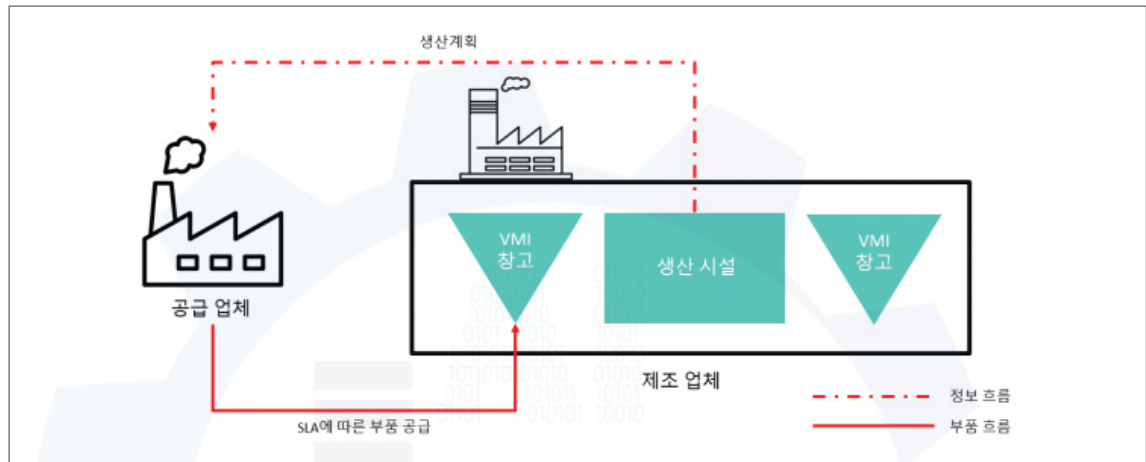
• 제조 공급망

- 다수의 제조 산업 내 공급망은 지속적 재고 보충(Continuous Replenishment Program, CRP) 방식을 활용한다. 이는 제조 업체와 유통 업체가 거래를 하는 상품에 대한 주문정보를 공유하여 생산 및 재고 보충을 관리하는 방법이다. 제조 업체의 판매 정보를 원자재 납품 업체가 이용하여 원자재 납품을 예측할 수 있고, 이를 통해 제조 업체가 원자재 부족 없이 생산을 진행함으로써 상품의 유통 흐름을 효율적으로 할 수 있다.

- CRP는 아래와 같은 산업군 및 제품군에 효율적으로 활용될 수 있다.

- ① 식품 제조 산업

- ② 제조 업체와 원자재 공급업체 관계가 긴밀한 업체
 - ③ 소비자 수요에 기초하여 제품을 생산 및 판매하는 산업
- CRP의 장점 : 판매 정보의 공유를 통해 제조 업체가 원자재를 유연하게 공급받으므로 제조 기업에서는 원자재 재고를 미리 확보하지 않아도 되어 기회비용을 절감할 수 있다.
- CRP의 단점 : 원자재 공급업체와 제조 기업의 관계가 원만하지 않은 경우에는 사용하기 어려운 방식이다. 거래 정보를 공유해야 하므로, 신뢰를 바탕으로 한 전략을 구사해야 한다. 또한, 급격한 수요량의 변화 등 예측이 어려운 상황에 대한 대응이 쉽지 않다.



[그림 3] 지속적 재고보충(CRP) 워크플로우

• 열처리 공정 공급망

- 여러 제조 공정에 활용되는 재료로서의 금속 부품을 만드는 열처리 공정의 특성상 다양한 기업과 연계하여 물품을 수주 및 발주하며 공정을 진행한다.
- 열처리 산업은 전방산업으로 자동차, 중장비, 금형, 공구, 부품 등 다양한 산업분야의 주요 산업군과 연계되며, 후방산업으로는 소재류 및 열처리 공정에 필요한 장비 산업 분야와 긴밀히 연계되어 있다.



[그림 4] 열처리 산업구조



2) 이슈사항(Pain point)

• 공정(공급망)상의 문제 현황

- 물품 공급 및 판매를 위한 수주 및 발주 중심으로 생산이 진행되는 제조업의 특성상, 네트워크로 연결된 전방 및 후방산업의 기업 상황에 따라 수주 및 발주량이 민감하게 영향을 받게 된다. 특히, 열처리 공정의 경우 이러한 공급망의 변동에 따라 재고를 파악하고 생산량을 조절하는 것이 필요하다.
- 특히, 소규모 열처리 업체의 경우 수주에 따라 다양한 작업물 생산이 가능하지만 대부분의 공정은 소량의 주력 품목을 생산하는 데에 집중된다. 따라서, 주력 상품에 대한 생산 진행 시 미래의 수주량을 현장 작업자의 경험을 바탕으로 예측하여 생산을 해두고 있는 실정이다.
- 이 때 미래 수주량 및 적정 생산량 예측에 실패하여 주력 상품의 생산이 특정 품목으로 집중될 경우 다른 품목의 재고가 부족하여 추가 공정을 진행하거나 생산 계획을 급히 변경하는 경우가 발생한다.
- 또한, 열처리 공정 특성상 긴 시간이 소요되므로 한정된 생산 라인으로는 추가되는 주력 상품 외 품목의 생산이 지연되어 납기일이 늦어지거나 추가 수주를 받지 못하는 경우가 발생하기도 한다.

• 문제해결 장애요인

- 열처리 공정 공급망 내 연관된 업체들에 대한 분석 및 수주 특성, 생산량 및 재고량 등에 대한 데이터 분석이 이루어지지 않아 공급망의 잠재적 위험요인의 도출과 특이사항 발생 시 대처 방안의 마련이 이루어지지 않은 상황이다.
- 이상적으로는 열처리 공정을 통해 생산되는 모든 제품에 대한 수주, 발주 및 재고량에 대한 데이터가 존재한다면 이를 활용한 전체적인 공급망 분석을 수행하여 미래 수주량 및 생산량을 예측하고 최적화할 수 있다. 하지만, 실제로는 소량의 주력 상품에 대한 데이터만 존재하는 경우가 대다수이며, 생산 라인 가동률 및 타 품목의 생산량에 대한 예측이 불가능하므로 전체적인 공급망 분석은 현실적으로 어려울 것으로 판단된다.

• 극복 방안

- 일반적으로 생산량, 발주량 등 재고관리를 위한 미래 예측에는 회귀분석을 위한 방법론이 활용된다. 본 분석에서는 회귀 분석을 위한 인공지능망 기반의 AI 알고리즘을 구축하고 이를 활용하여 미래 출고량에 대한 예측을 수행하고자 한다.
- 또한, 불확실성을 고려한 베이지안 신경망 알고리즘을 활용하여 데이터 분석을 진행함으로써, 출고량 예측 시 특이사항이 발생할 경우에도 유연하게 수주, 발주 및 원자재 공급 계획을 수립하는 데에 큰 도움이 될 것으로 기대된다.



1.2 분석 목표

1) 분석 목표

- 본 분석에서는 한 열처리 공정 기업의 공급망 관련 데이터를 활용하여 미래 시점의 출고량을 예측할 수 있는 AI 기반 분석 모델을 구축하고자 한다.
- 일별로 여러 시점에 기록된 수주량, 생산 필요량, 재고, 실제 생산량 등을 고려하여 미래의 실제 출고량을 예측하는 모델을 개발함으로써 정량적으로 향후 생산 계획을 수립하여 안정적인 공급망 관리에 도움이 되고자 한다.
- 불확실성을 고려한 인공신경망 기반 예측 알고리즘을 활용하여 비정상적인 입력에도 강건한 예측을 수행하는 확률적 모델링 방식을 사용함으로써, 향후 탄력적이며 합리적인 AI 분석모델 구축이 가능하도록 한다.

2) 데이터 정의 및 소개

- 본 분석에서 사용하는 제조 공정 데이터는 열처리 공정을 통해 생산하는 두 가지 품목에 대한 수주량, 생산량, 발주 수량 등의 정보를 일자 및 시간별로 수집한 데이터이다. 일자별로 업무 시간(8시부터 20시) 내 3시간에서 4시간 단위로 기록된 여러 수량 정보가 존재하며, 품목별로 순차적으로 기록되어 있다.

3) 데이터 분석 기대효과

- 본 분석은 공급망 관련 제조 데이터를 기반으로 AI 분석 모델을 구축함으로써 품목별 미래 출고량 예측을 수행한다. 또한, AI 분석 모델 구축을 위한 방법론의 이론적 설명, 데이터 변환 및 조작, 실제 데이터를 활용한 실습 등을 통해 다양한 산업 공급망에서 쉽게 응용이 가능한 분석 방법을 제공한다.
- 복잡한 공급망 구조에 대해 맞춤형으로 유연하게 수주, 발주 및 원자재 공급을 할 수 있도록 예측에 기반한 정보를 제공하고, 이를 통해 합리적인 공정 계획을 수립할 수 있을 것으로 판단된다.

4) 시사점(implication) 요약기술

- 공급망 내 수주, 발주, 생산 등의 데이터를 활용한 AI 기반 분석을 통해 공급망 구조에 위험 요소가 될 수 있는 특이사항을 검출하고 조기 대응함으로써 공급망의 안정성 및 구조 효율성을 확보할 수 있다.
- 본 분석에서 수행하는 AI 기반 예측 알고리즘 구축을 통해 복잡한 공급망 구조에서 수집되는 데이터를 바탕으로 한 유용한 정보를 추출하고, 이를 기반으로 유연한 공정 계획을 수립할 수 있을 것으로 기대한다.



2. 분석실습

2.1 제조데이터 소개

1) 데이터 수집 방법

- 제조 분야 : 공급망 및 발주 관리
- 제조 공정명 : 열처리 공정
- 수집 장비 : Enterprise Resource Planning (ERP) 시스템
- 수집 기간 : 2022년 3월 1일 ~ 2022년 6월 30일
- 수집 주기 : 평균 3.5시간

2) 데이터 유형/구조

- 데이터셋 구조 : 테이블 형식(tabular)
- 데이터 개수 : column 수 6개, row 수 688개, 데이터수 4,128개
- AI 데이터셋 주요 변수(속성) 정의

데이터셋 내에는 기록일자, 시간 및 생산 품목 식별자(비식별화됨)와 이에 해당되는 공급망 관련 여러 정보가 존재하며 주요 변수는 아래 표와 같다. 공급망 관련 변수를 활용하여 종속 변수인 실제 출고 수량을 만들고, 나머지를 독립 변수로 활용할 것이다.

속성(column)	설명	비고
시간	특정 품목의 공급망 관련 수량이 기록된 시점의 시간 정보	연도, 월, 일, 시각(24시 기준) 데이터형 : datetime
품목	품목의 식별자(ID)	비식별화됨 데이터형 : string
생산필요량	해당 시점에 기록된 특정 품목의 생산필요량	단위 : kg 데이터형 : int
재고	해당 시점에 기록된 특정 품목의 재고량	단위 : kg 데이터형 : int
생산량	해당 시점에 기록된 특정 품목의 실제 생산량	단위 : kg 데이터형 : int
수주량	해당 시점에 기록된 특정 품목의 실제 수주량	단위 : kg 데이터형 : int



- 주요 변수 기술 통계

속성(column)	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
생산필요량	int	688	59.375	115.828	0.000	0.000	400.000
재고	int	688	177.252	149.630	0.000	200.000	500.000
생산량	int	688	131.250	78.866	0.000	150.000	200.000
수주량	int	688	237.500	106.297	0.000	200.000	400.000

3) 데이터 (품질) 전처리

• 데이터 품질 전처리 목적

- 실제 공정 및 공급망 상에서 수집되는 데이터는 무의미한 값을 지니거나 누락값 등의 발생으로 품질이 저하되는 경우가 존재한다.
- 적합한 데이터 전처리를 거치지 않고 품질이 낮은 데이터를 그대로 사용 시, 좋은 분석 방법론을 이용하더라도 잘못된 분석 결과가 도출될 가능성이 존재한다.
- 이를 방지하기 위해 적합한 데이터 전처리를 수행하여 전체적인 분석 수준을 높이는 것이 필수적이다.
- 본 분석에서는 데이터의 6가지 품질 지수를 파악하고, 데이터 전처리를 통해 품질 지수를 향상시키고자 한다.

• 데이터 품질 지수

- 제조 공급망에서 수집된 데이터를 기반으로 AI 분석 모델을 적용하기 위해서는, 먼저 데이터 가공 전 품질 확인이 선행되어야 한다. 분석을 위한 데이터의 품질이 전제되었을 때, AI 분석 모델의 우수한 성능 및 유의미한 시사점 도출을 기대할 수 있다.
- 공급망 데이터의 경우 수주, 발주, 생산 등 실제 기록된 데이터 항목이 많으므로, 결측치 및 오 기록된 값을 확인하고 제거하는 등의 데이터 전처리를 통해 데이터의 품질을 제고할 수 있다. 본 분석에서는 데이터 품질 확인을 위해 Gartner에서 제시한 아래 6종의 데이터 품질 평가지표를 고려하였다.

- ① **완전성(Completeness)** : 필수항목에 누락(결측치)이 없어야 한다.
- ② **유일성(Uniqueness)** : 데이터 항목은 유일해야 하며 중복되어서는 안 된다.
- ③ **유효성(Validity)** : 데이터 항목은 정해진 데이터 유효 범위 및 도메인을 충족해야 한다.
- ④ **일관성(Consistency)** : 데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 동일한 데이터 간에 불일치가 발생하지 않아야 한다.
- ⑤ **정확성(Accuracy)** : 실제 존재하는 객체의 표현 값이 정확히 반영되어야 하며, 정보의 수집 주기를 유지해야 한다.



- ⑥ **무결성(Integrity)** : 데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 유일성, 유효성, 일관성이 보호되어야 한다.

• 데이터 품질 지수 세부 설명

- 완전성 품질 지수 = $(1 - (\text{결측치 수} / \text{전체 데이터 수})) \times 100$

- ① 데이터에 존재하는 결측치 수를 확인하기 위해 pandas 패키지의 'isna()' 와 'sum()' 메서드를 이용하여 총 결측치 개수를 구한다.
- ② 메서드를 통해 구한 결측치의 개수를 이용하여 완전성 품질 지수를 계산한다.

- 유일성 품질 지수 = $(1 - (\text{중복 데이터 수} / \text{전체 데이터 수})) \times 100$ 본 실습의 데이터는 '품목' 및 '시간' 열을 기준으로 유일한 값을 가지는지 판단한다.

- 유효성 품질 지수

- ① 데이터가 유효 범위 내에 있는가?
- ② 데이터가 올바른 형식으로 구성되어 있는가?
- ③ 수집된 날짜 안에 들어가는가?

- 일관성 품질 지수 = $(\text{일관성 만족 데이터 수} / \text{전체 데이터 수}) \times 100$

- 정확성 품질 지수 = $(1 - (\text{정확성 위배 데이터 수} / \text{전체 데이터 수})) \times 100$

- 무결성 품질 지수 = $(1 - (\text{유일성, 유효성, 일관성 지수 중 100%가 아닌 지수 개수} / 3)) \times 100$

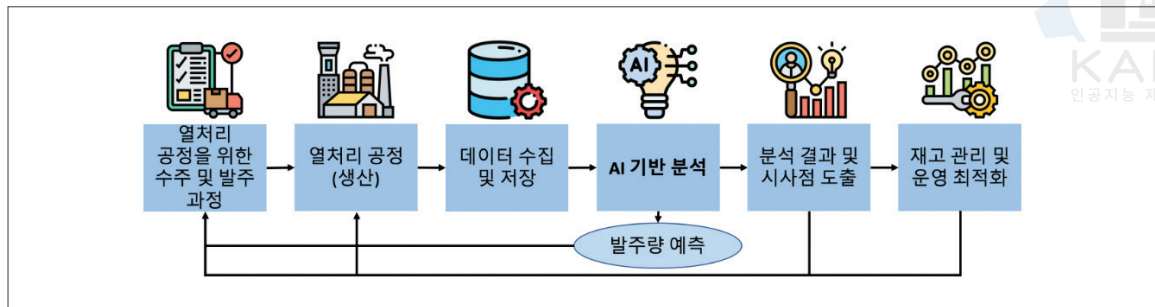
• 데이터 품질 지수 계산 실습

- 본 분석에서 사용되는 열처리 공급망 데이터의 6가지 데이터 품질 지수 계산 실습은 부록_데이터 품질 전처리 [실습코드]에 자세히 설명되어 있다.

2.2 분석모델 소개

1) 데이터 및 AI 분석모델 적용

- 앞서 설명된 바와 같이, 열처리 공정 및 공급망 내에서 AI 분석 모델을 활용하는 방안은 다음과 같다. 먼저, 열처리 공정에서 수집되는 데이터를 저장하여 활용한다. 이를 기반으로 AI 분석 모델을 구축하여 목적에 부합하는 예측을 수행한다. 예측 결과를 기반으로 시사점을 도출하고, 더불어 재고 관리 및 공급망 및 생산 운영의 최적화를 도모한다.



[그림 5] 열처리 공급망 데이터를 활용한 AI 분석 모델 적용 흐름도

2) AI 분석모델

• 열처리 공급망 데이터 분석 방법

- 데이터 기반 분석

일반적으로 수집된 데이터를 활용하는 방법은 데이터에 예측하고자 하는 값의 종류에 따라서 지도학습(supervised learning), 비지도학습(unsupervised learning), 준지도학습(semi-supervised learning)으로 분류된다. 이는 데이터 내 레이블(label)의 존재 유무로 분류한다고도 한다. 제조 데이터를 활용한 AI 분석의 경우, 특정 수량을 예측하거나 제품의 품질, 양불 여부 혹은 설비의 이상 여부 등을 예측하는 등의 지도 학습 접근방식이 빈번히 활용된다.

- 공급망 데이터 활용 방안

열처리 공정 공급망에서 수집된 데이터에는 수주량, 발주량, 생산량 등의 변수가 존재하며, 수집시간에 대한 정보가 같이 기록되어있다. 본 분석에서는 이러한 특성을 고려하여, 과거의 수량 정보를 독립변수로 사용하고 실제 출고 수량을 종속변수로 활용하여 예측을 수행하고자 한다.

- 공급망 내 불확실성 고려

앞서 언급된 바와 같이, 여러 제조 업체가 협력하여 운영되는 공급망의 특성상 예상치 못한 외재적 변동성이 존재하거나 업체의 실수로 인해 데이터에 영향이 생겼을 때 탄력적으로 대응하기 어려운 특성이 있다. 본 분석에서는 이러한 공급망 내 불확실성을 고려하여 AI 예측 모델을 구축하고, 기존의 지도학습 기반 예측 모델과 융합될 수 있는 분석 방법론을 제시한다.

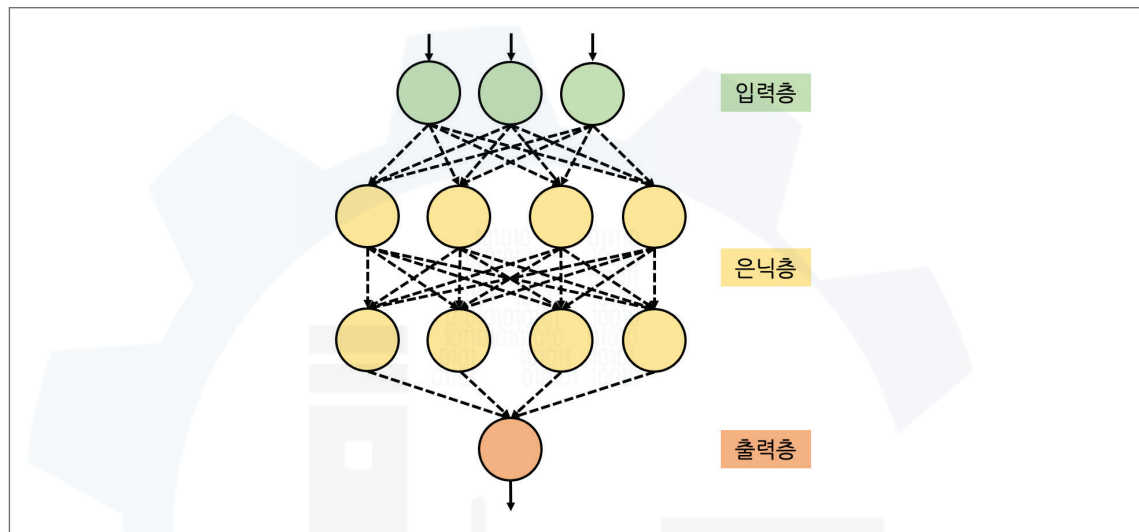
- AI 모델을 활용한 분석 목적

전통적인 통계 기반 방법론은 인위적인(handcrafted) 특성 추출 및 변환 단계를 요구한다. 반면 인공지능망(Artificial Neural Network, ANN) 기반 방법론은 이러한 특성 추출을 데이터를 통해 자동적으로 수행할 수 있으며, 특히 본 분석에서 사용될 베이지안 신경망의 경우 자동적인 특성 추출이 가능할 뿐 아니라 예측의 불확실성 또한 고려할 수 있다.



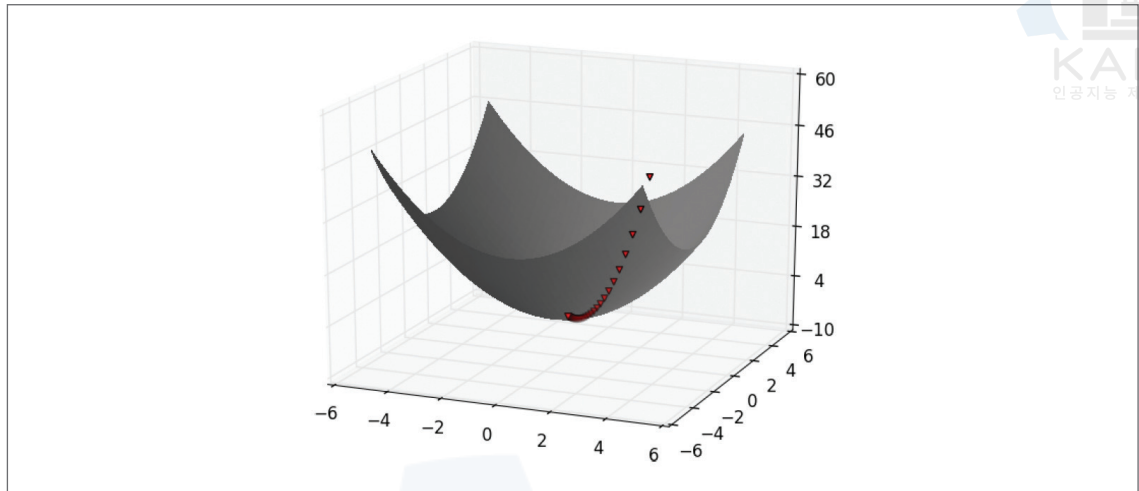
• 인공지능망 기반 알고리즘

- 인공지능망은 인공 뉴런(artificial neuron)이라고 불리는 노드(node)들이 가중치를 가지는 엣지(edge)를 통해 연결된 구조를 갖는 모델이다. 가중치는 한 노드에서 다른 노드로의 정보 흐름의 세기를 조정하며 데이터를 통해 학습되는 모수(parameter)이다. 각 노드는 [그림 6]과 같이 입력층(input), 은닉층(hidden), 출력층(output)을 이룬다. 신경망을 이루는 각 층 사이에는 활성화 함수(activation function)를 통한 비선형 변환이 가능하여 입력과 출력 간의 관계와 같은 데이터에 내재된 특성이 복잡할 때도 해당 특성을 나타내는 함수를 잘 추정할 수 있다.



[그림 6] 인공지능망의 예시

- 지도학습에서 인공지능망은 입력과 출력 사이의 관계를 모방하는 함수를 학습한다. 인공지능망은 손실 함수(loss function)의 값을 감소시키는 방향으로 학습되는데, 손실함수는 예측하고자 하는 실제 값과 신경망을 통해 예측된 값 사이의 차이를 나타내는 지표이다. 그러한 손실함수의 값을 최소화하는 모수의 최적값을 찾기 위해서는 일반적으로 경사 하강법(gradient descent)이 사용된다. 경사하강법은 최적화 알고리즘 중의 하나로, 손실함수의 모수에 대한 편미분 값을 이용해 모수의 값을 함숫값이 감소하는 방향으로 갱신해나가는 방법이다. 편미분 값은 역전파(backpropagation)를 통해 구할 수 있는데, 학습하고자 하는 모든 모수에 대한 손실함수의 편미분 값을 계산하기 위해 연쇄 법칙(chain rule)을 통해 신경망 내부의 층별 편미분 값을 이용한다.



[그림 7] 경사 하강법(gradient descent)

〈출처 : <https://pub.towardsai.net/gradient-descent-algorithm-for-machine-learning-python-tutorial-ml-9ded189ec556>〉

• 확률론적 모델링

- 불확실성

AI 분석 모델을 활용하여 데이터를 기반으로 예측 등의 업무를 수행할 때 고려할 사항 중 하나는 불확실성(uncertainty)이다. 불확실성은 분석 과정에서 완전히 알 수 없거나 온전하게 정확한 정보를 가지고 예측을 수행할 때 발생한다. 불확실성은 크게 데이터 자체의 불확실성인 aleatoric uncertainty와 분석 모델의 불확실성인 epistemic uncertainty로 구분된다. 데이터 자체의 불확실성은 보통 데이터 계측 및 수집 단계에서 발생하는 측정 오차 등에 의해 유발되며, 분석 모델의 불확실성은 관측된 데이터를 활용하여 분석 모델을 구축하는 단계에서 발생한다. 이 두가지 불확실성을 통합하여 예측 불확실성(predictive uncertainty)라고 부르며, 이러한 불확실성을 고려하는 분석 모델을 활용하면 기존의 방법론들과 다른 장점을 가질 수 있다.

- 최대 우도 추정법의 단점

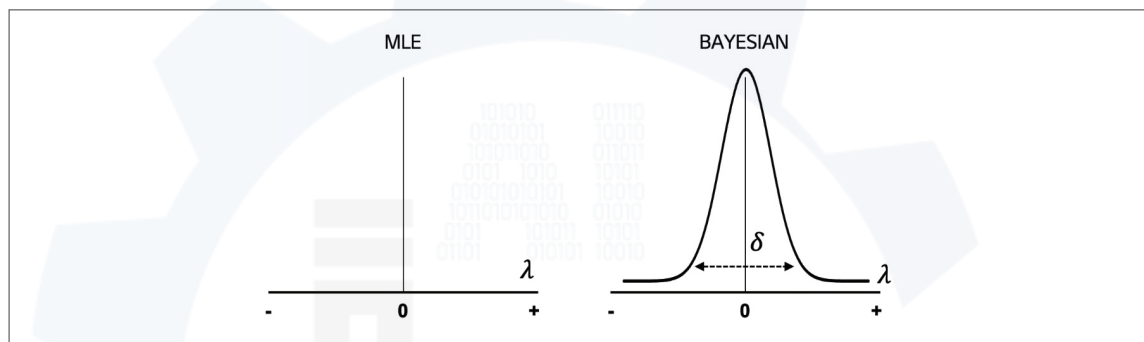
앞서 설명한 대로, 일반적인 인공지능 기반 모델은 설정한 손실함수를 사용하여 계산한 손실값과 최적화하고자 하는 모델의 모수에 대한 손실함수의 기울기 정보를 활용하여 학습을 진행한다. 회귀 문제의 경우, 제곱 오차(squared error)를 활용하여 손실 함수를 설정하는데 이는 모수 추정(parameter estimation)의 관점에서 최대 우도 추정법(maximum likelihood estimation, MLE)으로 해석할 수 있다. 즉, 이는 가능도 혹은 우도를 최대화하는 방향으로 모수를 추정하는 방법이다. 이러한 MLE 관점은 많은 데이터가 존재할 때 합리적인 추정을 수행할 수 있기 때문에 많은 심층신경망 기반 분석 모델의 학습 과정에서 사용되는 원리이다. 하지만 MLE 기반 추정은 예측 불확실성을 고려하기 어렵다는 단점이 있다. 또한, 추정치에 기반한 분석 모델의 예측값이 항상 고정적(deterministic)이라는 단점이 존재한다. 반면, [그림 8]과 같이 베이지 정리를 사용하는 베이지안 추정법은 이러한 단점을 극복한다.

$$p(w|D) = \frac{p(D|w)p(w)}{p(D)} = \frac{p(D|w)p(w)}{\int p(D|\hat{w})p(\hat{w})d\hat{w}}.$$

[그림 8] 베이즈 정리(Bayes Theorem)

- 베이지안 추정(Bayesian estimate)

베이지안 추정법은 MLE와 같이 우도($P(D|w)$)를 최대화하는 모수를 찾는 것이 아니라, 반대로 모수에 대한 사후 확률($P(w|D)$)을 최대화하는 데에 목적이 있다. 이 과정에서 베이즈 정리를 사용하여 사후 확률분포를 우도($P(D|w)$), 사전 확률분포($p(w)$), 주변 확률분포($p(D)$)로 분리하여 계산을 유도한다. 이렇게 추정을 진행하면, MLE에서의 고정적인 모수값이 아닌, 모수가 따르는 확률분포 자체를 학습할 수 있게 된다.



[그림 9] 최대 우도 추정법과 베이지안 추정법의 비교

- 베이지안 딥러닝

확률론적 모델링의 일종인 베이지안 딥러닝은 앞서 설명한 베이지안 추정법을 심층 신경망에 적용한 방법론이다. 기본적으로 고정적인 모수를 찾고 고정된 예측값을 출력하는 기존 MLE 기반의 분석 모델과 달리, 매 예측마다 동일한 입력에 대해서도 확률적인 출력을 수행하게 된다. 이는 계산된 사후 확률분포를 사용하여 베이지안 추론(Bayesian inference)을 수행하여 예측분포(predictive distribution)을 얻는 과정이라고 볼 수 있다. [그림 10]은 입력 x 를 사용하여 y 를 예측하는 과정에서의 베이지안 추론을 나타내는 수식이다.

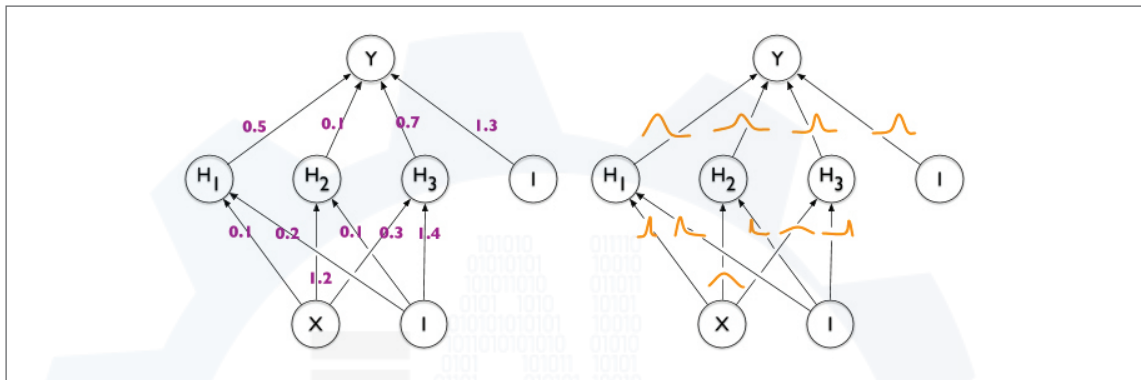
$$p(y|x, D) = \int p(y|x, w)p(w|D)dw.$$

[그림 10] 베이지안 추론(Bayesian inference)

- 베이지안 신경망

베이지안 딥러닝에는 전통적인 베이지안 신경망(Bayesian neural network, BNN), 여러 모델을 활용한 딥앙상블(deep ensemble), 확률적 경사 하강법을 활용한 추론

법, 드롭아웃을 사용한 몬테카를로 드롭아웃(MC dropout) 등 다양한 종류의 방법론이 존재한다. 그중에서도 가장 전통적이며 이론적 배경이 충실한 베이지안 신경망을 소개한다. 베이지안 신경망은 예측 불확실성을 고려하기 위해 모델 학습 시 최적화하는 신경망의 모수 자체가 아닌 모수가 따르는 확률 분포를 학습한다. [그림 11]는 베이지안 신경망이 모수의 확률분포를 학습한다는 개념을 시각적으로 보여준다. 일반적으로 계산의 편의를 위해, 각 모수는 정규분포(Gaussian distribution)를 따른다고 가정하고 모든 모수는 서로 독립(independent)이라는 가정하에, 각 모수가 따르는 정규분포를 정의하는 평균과 표준편차를 학습하게 된다.



[그림 11] 베이지안 신경망(Bayesian neural network)

〈출처: <https://neptune.ai/blog/bayesian-neural-networks-with-jax>〉

• 해당 AI 방법론(알고리즘) 선정 이유 기술

- 제조 데이터 분석 알고리즘 : 베이지안 신경망(BNN)

본 분석에서는 베이지안 신경망을 기반으로 발주량, 수주량, 생산량 등의 수치 데이터를 활용하여 미래 실제 출고 수량에 대한 예측을 수행한다. 앞서 설명한 대로, 베이지안 신경망 학습 시, 모델을 구성하는 모수가 따르는 확률 분포 자체를 학습함으로써 예측 불확실성을 고려한 추론을 수행하게 된다.

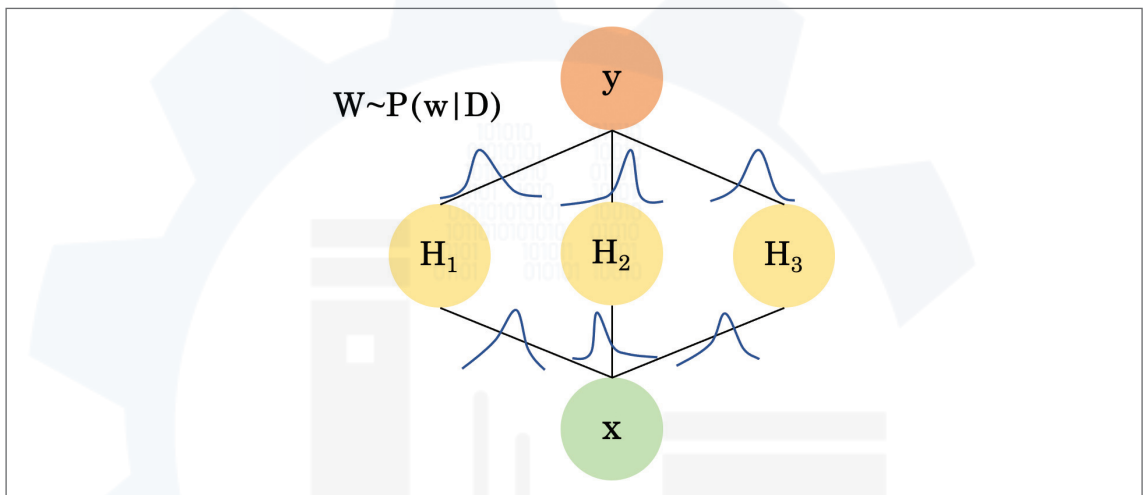
- 해당 알고리즘 선정 이유

확률적 모델링 방법론으로서의 베이지안 신경망을 사용함으로써, 예측 불확실성을 고려할 수 있다는 것이 가장 중요한 선정 이유이다. 특히, 출고 수량 예측 등 제조 공정 내 생산성, 효율성 등 기업의 이윤과 직결되는 분석 및 예측 과정에서 불확실성을 고려함으로써 분석 결과를 활용한 안전한 판단을 가능케 할 수 있다. 추가로 앞서 설명된 바와 같이, 베이지안 신경망을 사용함으로써 점 추정이 아닌 예측 분포를 출력하는 확률론적 모델링 방법론을 활용할 경우 기존 접근방식보다 정상 범주에서 벗어난 입력값 등 예측불가능한 이상치에 대한 탐지가 용이할 것이라고 기대할 수 있다.

• 적용하고자 하는 AI 분석 방법론(알고리즘)의 구체적 소개

- 사후 확률 분포 계산

베이지안 신경망은 기본적으로 베이지 정리를 활용하여 모수에 대한 사후 확률분포를 구하고자 한다. 따라서, 앞서 설명한 [그림 12]와 같이 사후 확률 분포 계산이 필요한데 이를 계산하는 과정에서 문제가 발생한다. 그것은 바로 분포에 위치한 적분식이 분석적으로 계산하기 어렵다(intractable)는 것이다. 특히, 추정해야하는 모델의 모수 개수가 아주 많은 심층신경망의 경우 정확한 적분을 수행하기는 계산적으로 불가능에 가깝다. 이러한 문제를 회피하기 위해 샘플링(sampling) 혹은 근사 및 추정 방법이 활용된다. 본 분석에서는 근사법의 일종인 변분 추론(variational inference)을 기반으로 한 방법론을 활용한다.



[그림 12] 베이지안 신경망 예시

- 변분 추론

변분 추론은 처리하기 어려운 사후 확률 분포 계산을 위해 실제 사후 확률 분포를 대체할 수 있으며 비교적 계산이 용이한 변분 분포(variational distribution)를 정의함으로써 시작한다. 정의한 변분 분포와 실제 사후 분포의 유사도를 정의하기 위해 쿨백-라이블러 발산(Kullback-Leibler divergence)이라는 개념을 사용한다. 이 발산값은 0 이상의 값을 가지는 비대칭적 척도이며, 비교하는 두 확률분포가 비슷할수록 낮은 값을 가진다. 따라서, 변분 분포와 실제 사후 분포 사이의 쿨백-라이블러 발산값을 정의 후 유도하면, [그림 14]와 같은 식이 유도되며 최종적으로는 ELBO (evidence lower bound) 라는 하한값을 찾을 수 있게 된다. 쿨백-라이블러 발산값은 항상 0 이상의 값을 가지므로 ELBO는 $p(D)$ 에 대한 하한이 된다.

$$D_{KL}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right).$$

[그림 13] 쿨백-라이블러 발산(Kullback-Leibler divergence)

$$\begin{aligned}
 KLD &= D_{KL}(q(w|D)||p(w|D)) \\
 &= \int q(w|D) \log \frac{q(w|D)}{p(w|D)} dw \\
 &= \int q(w|D) [\log q(w|D) - \log p(w|D)] dw \\
 &= - \int q(w|D) [\log \frac{p(D, w)}{p(D)} - \log q(w|D)] dw \\
 &= \log p(D) - \int q(w|D) [\log \frac{p(D, w)}{q(w|D)}] dw \\
 &= \log p(D) - \int q(w|D) [\log \frac{p(w)p(D|w)}{q(w|D)}] dw \\
 &= \log p(D) - ELBO.
 \end{aligned}$$

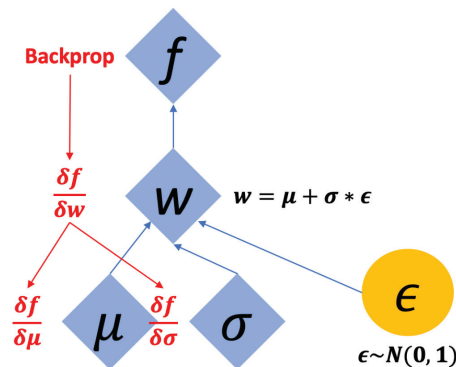
[그림 14] ELBO 유도식

- 앞서 설명한 대로 베이지안 신경망의 학습 과정에서는 실제 사후 분포를 근사하는 변분 분포를 찾기 위해 쿨백-라이블러 발산을 최소화하게 되는데, 이는 동시에 ELBO를 최대화하는 것과 동일하게 된다. 일반적인 학습 과정에서는 손실함수를 최소화하므로, ELBO에 -1을 곱한 값인 [그림 15]에 표현된 수식을 최소화하게 된다.

$$\mathcal{L}_{ELBO} = D_{KL}(q(w|D)||p(w)) - E_q[\log p(D|w)].$$

[그림 15] ELBO 식

- 추가로, 확률적 경사 하강법을 활용하여 위와 같이 베이지안 신경망 학습을 진행할 때 활용되는テクニック이 존재한다. 그 이유는 신경망 학습 시 오차 역전파를 하는 과정에서, 모수가 따른다고 가정한 변분 분포에서 샘플링을 수행할 경우 역전파가 수행될 수 없기 때문이다. 따라서 변분 분포를 정의하는 평균과 표준편차를 모수로 설정하여 재모수화(reparameterization trick)를 수행하게 된다. 이는 [그림 16]에 잘 표현되어 있다.



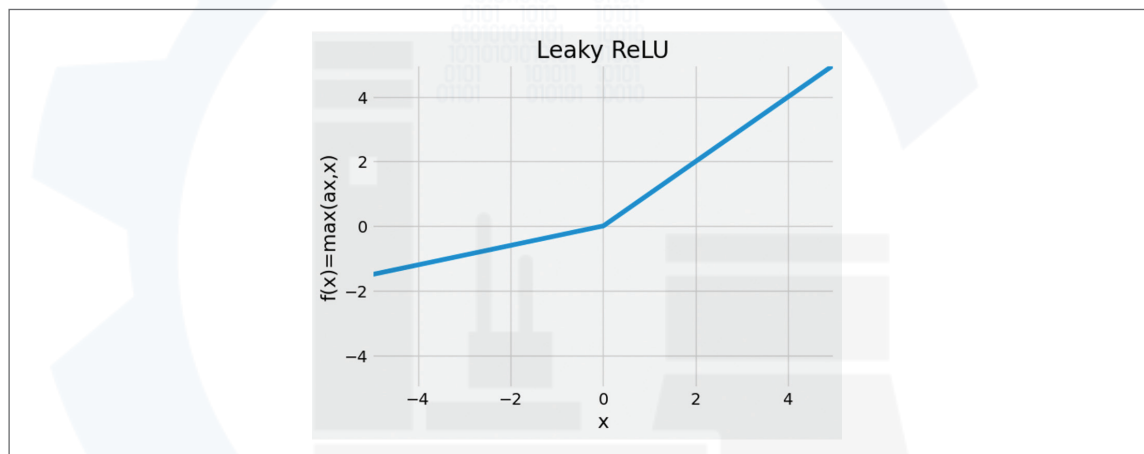
[그림 16] 재모수화(reparameterization)

- 위에 설명된 내용을 기반으로 사용자가 원하는 대로 베이지안 신경망의 구조를 설계 후 모델을 구축할 수 있다. 입력층, 출력층 및 은닉층의 구조를 적절히 설정한 뒤, 확률적 경사 하강법 기반의 최적화 방법을 사용하여 최적의 모수 분포를 탐색한다. 학습 종료 후 새로운 데이터에 대한 예측을 수행할 때는, 베이지안 추론을 수행한다. 학습 시 도출한 변분 분포에서 모델 모수의 조합을 N회 샘플링하여 몬테카를로 방식으로 추론을 수행한다. 이는 곧 서로 다른 독립적인 모델을 학습된 변분 분포로부터 추출하여 예측을 수행한 뒤 이를 평균내어 예측 분포로서 사용하는 것이라고 볼 수 있다.

$$p(y|x, D) = \int p(y|x, w)p(w|D) dw \approx \frac{1}{T} \sum_{t=1}^T p(y|x, w_{(t)})$$

[그림 17] 베이지안 신경망 예측

- 추가적으로, 베이지안 신경망 내 활성화 함수로는 기존 ReLU 함수의 변형인 LeakyReLU를 사용한다. LeakyReLU는 $\max(\alpha x, x)$ ($\alpha \geq 0$)으로 계산된다.



[그림 18] Leaky ReLU 활성화 함수

• AI 분석 방법론(알고리즘) 구축 절차 설명

- 수량 데이터를 활용하여 AI 분석모델을 적용하기 위해서는 적합한 형태로 데이터셋을 변환하는 것이 필요하다. 전처리 과정을 통하여 수집된 데이터에 대해 정제 및 필터링을 진행한다. 이후 데이터셋을 학습-검증-평가 데이터로 분리하여 일련의 학습 과정 내에서 목적에 부합하게 사용한다. 본격적인 AI 분석 방법론 구축으로는, 선택한 알고리즘(베이지안 신경망)을 정의하는 과정이 요구된다. 인공신경망 기반 모델의 경우, 조정할 수 있는 초매개변수(hyper-parameter)가 존재한다. 예를 들어, 은닉층의 노드 개수, 활성화 함수, 규제항 및 손실 함수의 종류 등을 사용자의 선택에 따라 다양하게 조정할 수 있다. 따라서 먼저 적합한 초매개변수를 설정함으로써 모델을 정의한다. 이후 준비된 데이터셋을 활용하여 학습, 검증 및 평가를 위한 데이터셋 분리 진행 후,

모델 학습 및 성능 검증을 수행한다. 필요시에는 모델 성능 제고를 위해 모델을 재정의하거나 변경할 수 있다. 최종적으로 학습이 종료된 모델을 저장하고 평가용 데이터셋을 사용하여 성능을 평가한다.

2.3 분석 체험

1) 필요 SW, 패키지 설치 방법 및 절차 가이드

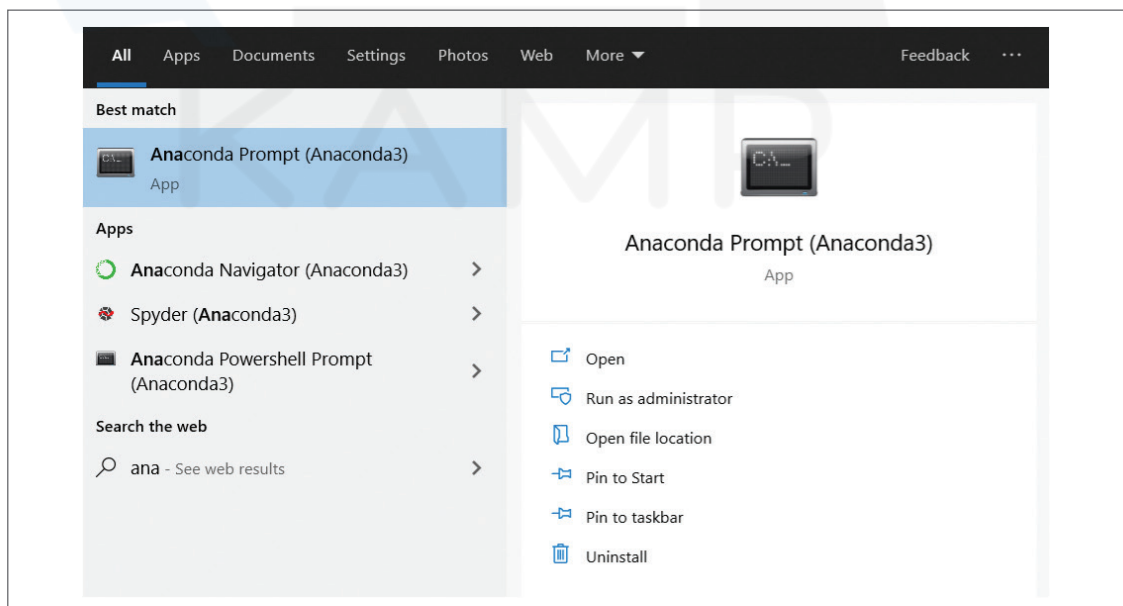
▶ 필요 SW 설치

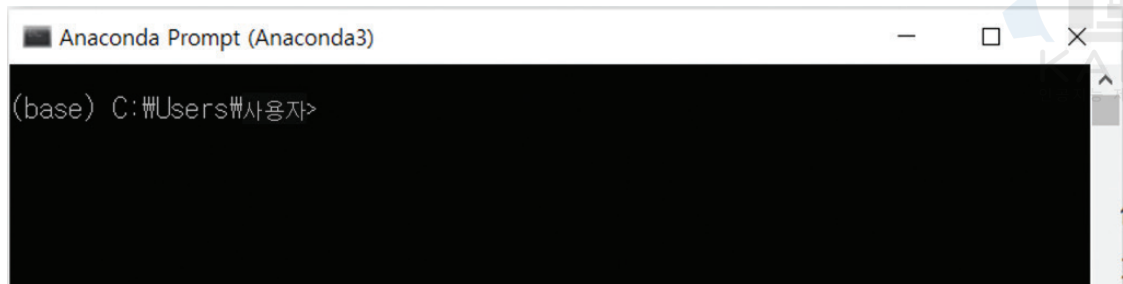
- 프로그래밍 언어인 파이썬(Python)을 사용하며 개발 환경으로는 주피터 노트북(Jupyter Notebook)을 활용한다. 관련 SW 설치는 3. 부록 [분석 환경 구축을 위한 설치 가이드]를 참고하면 된다.

▶ 가상환경 구축 가이드

- 3. 부록 [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

- 아나콘다 프롬프트 실행

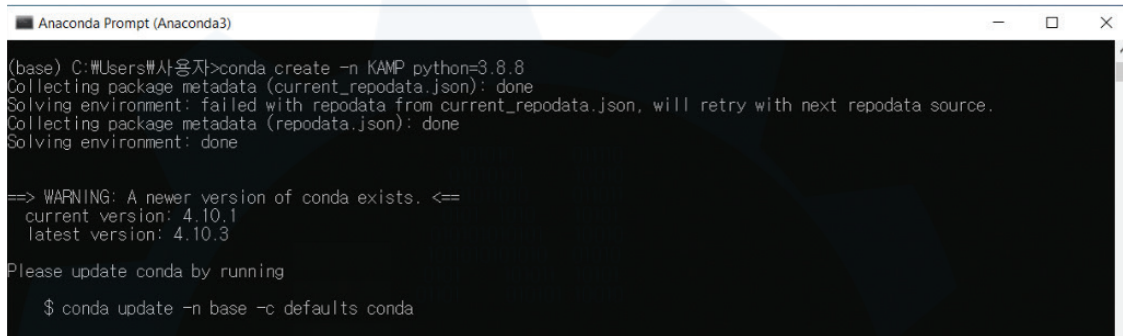




- 가상 환경 생성

```
# conda create -n 가상환경이름 python=버전
```

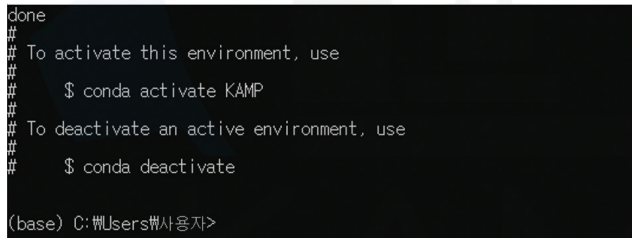
```
conda create -n KAMP python=3.8.8
```



...

Proceed ([y]/n)? y # 해당 문구가 뜨면 'y' 를 입력한다.

...

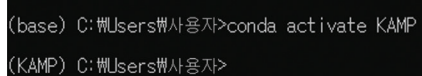


- 가상 환경 실행

가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시된다.

```
# conda activate 가상환경이름
```

```
conda activate KAMP
```



- 분석이 끝난 후에는 가상 환경을 종료한다.

```
conda deactivate
```

```
(KAMP) C:\Users\회사용자>conda deactivate
(base) C:\Users\회사용자>
```

- 주피터 노트북에서 가상 환경 실행

주피터 노트북에서 가상 환경을 실행하기 위해 가상 환경 비활성화(종료) 상태에서 아래의 코드를 작성한다. 주피터 노트북 실행창에서 빨간 박스처럼 생성한 가상 환경이 있는 것을 확인할 수 있다. 해당 가상 환경을 선택 후 실습을 진행한다.

```
pip install ipykernel
```

```
# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
python -m ipykernel install --user --name KAMP --display-name "KAMP"
```

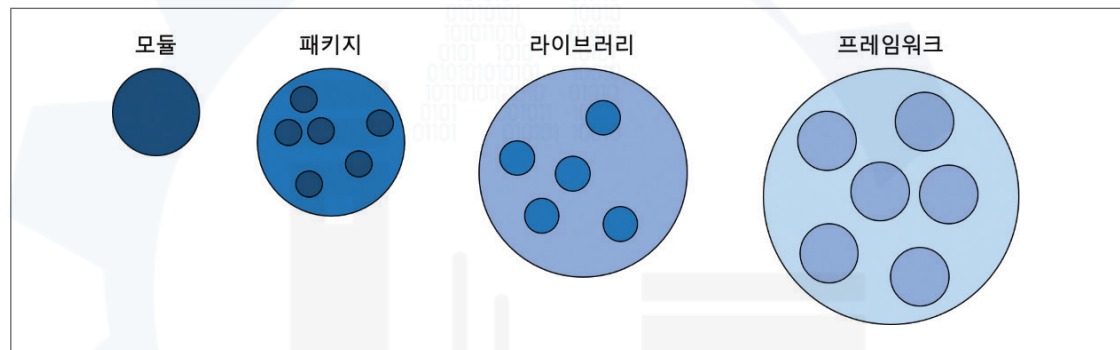
```
# jupyter notebook 실행
jupyter notebook
```



▶ 패키지 설치 가이드

- 패키지 설치 전 파이썬 관련 용어 정리

모듈 (module)	- 함수나 변수 또는 클래스를 모아 놓은 파일 - 파이썬 확장자(.py) 파일에 특정 함수, 변수, 클래스 등이 저장된 코드이다.
패키지 (package)	- 여러 모듈의 집합 - 하나의 디렉토리(경로)에 놓인 모듈들의 집합을 의미하며, 일반적으로 <code>__init__.py</code>라는 패키지 초기화 파일이 존재한다. - 예) NumPy : 수치해석 패키지, pandas : 데이터 분석 패키지
라이브러리 (library)	- 여러 패키지의 집합 - 파이썬을 설치할 때 기본적으로 설치되는 라이브러리를 표준라이브러리라고 한다. 파이썬 공식이 아닌 외부에서 개발한 모듈과 패키지를 묶어 외부 라이브러리라고 한다. - 예) matplotlib : 데이터 시각화를 생성하기 위한 표준 라이브러리
프레임워크 (framework)	- 라이브러리의 집합 - 목적에 맞게 코드를 작성할 수 있는 기본 아키텍처이며, 필수 구성 요소를 제공한다.



[그림 19] 파이썬 관련 개념도

▶ 필요 패키지 설치 방법

- 분석 환경인 주피터 노트북 내에서 데이터 분석 및 AI 분석모델 적용을 위한 패키지를 설치한다. 패키지 설치에는 pip 또는 conda 명령어를 사용하여 수행할 수 있는데, 본 분석에서는 둘 다 사용한다. 먼저, pip install <패키지 이름> 을 입력하여 원하는 라이브러리(혹은 패키지)를 설치한다.

```
!pip install pandas
!pip install numpy matplotlib scikit-learn
```


- 이미 설치되어 있는 패키지 외에 신규 설치가 필요한 패키지들이 자동으로 설치된다.

```
Requirement already satisfied: pandas in /home/ubuntu/anaconda3/lib/python3.6/site-packages (0.23.0)
Requirement already satisfied: python-dateutil>=2.5.0 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (2.7.3)
Requirement already satisfied: pytz>=2011k in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (2018.4)
Requirement already satisfied: numpy>=1.9.0 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from pandas) (1.19.5)
Requirement already satisfied: six>=1.5 in /home/ubuntu/anaconda3/lib/python3.6/site-packages (from python-dateutil>=2.5.0->pandas) (1.15.0)
```

- 원하는 패키지의 특정 버전을 설치하고 싶은 경우, pip install <패키지 이름>==<버전 번호>를 입력하여 설치할 수 있다.

```
!pip install tensorflow==2.9.0
```

- 본 분석에서는 tensorflow probability 패키지를 추가로 사용하는데, 이는 특정 버전의 tensorflow가 요구되므로 아래 코드와 같이 패키지 간의 호환성을 고려하여 설치한다.

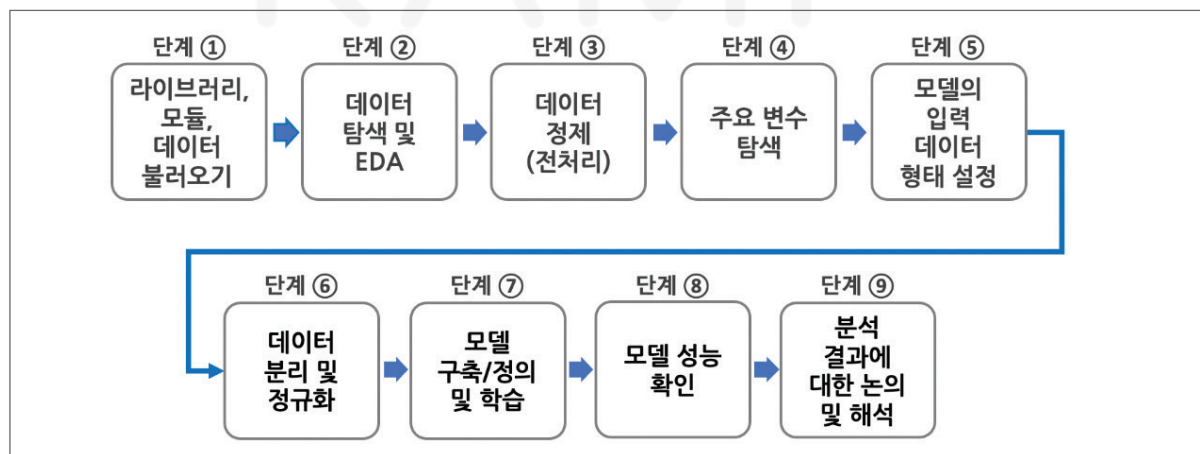
```
!pip install tensorflow-probability==0.17.0
```

- conda list 혹은 pip list를 사용하여 설치된 패키지들의 전체 목록을 확인할 수 있다.

```
!conda list
```

#	Name	Version	Build	Channel
	_libgcc_mutex	0.1	main	
	_openmp_mutex	5.1	1_gnu	
	_tfflow_select	2.3.0	mk1	
	abseil-cpp	20211102.0	hd4dd3e8_0	
	abs1-py	1.2.0	pypi_0	pypi
	aiohttp	3.8.1	py37h7f8727e_1	
	aiosignal	1.2.0	pyhd3eb1b0_0	
	anyio	3.5.0	py37h06a4308_0	

2) 분석 단계별 프로세스 - Flow Chart



[그림 20] AI 분석모델 활용 흐름도



[단계 ①] 라이브러리, 모듈, 데이터 불러오기

①-1. 패키지 불러오기

- import <패키지 이름>, 혹은 from <패키지 이름> import <하위 모듈 이름> 을 사용하여 전체 패키지를 불러오거나 특정 패키지 안의 모듈만 불러오기가 가능하다. 또한 import <패키지 이름> as <약자> 를 사용하여 추후 패키지 사용 시 약자(alias)로 이름을 대신할 수 있다.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

[코드 1] 필요 패키지 불러오기

- 분석 시 여러 라이브러리 혹은 패키지를 사용하다 보면 향후 업데이트 버전 변경이나 업데이트 등 변경 사항을 알려주는 경고 메시지가 출력될 수 있다. 분석 실습 시 시인성을 높이기 위해 warnings 라이브러리를 활용해 이러한 경고 메시지를 무시하고 숨길 수 있다.

```
import warnings
warnings.filterwarnings('ignore')
```

[코드 2] 경고 메시지 무시 코드

①-2. 데이터 불러오기

- 분석에 활용할 데이터인 'data.csv' 를 분석 실습 코드와 같은 폴더 내에 저장 후 사용한다. pandas 패키지 내의 'read_csv()' 함수를 활용하여 CSV(.csv) 파일 형식의 데이터를 불러올 수 있다. 데이터를 df라는 이름으로 불러온다.

```
df = pd.read_csv('./data.csv')
```

[코드 3] csv 형태의 데이터 불러오기

- 불러온 데이터의 상단 5개 내용을 확인할 수 있다.

```
df.head()
```

	시간	품목	생산필요량	재고	생산량	수주량
0	2022.3.1 8:00	GEAR_21	100	100	0	100
1	2022.3.1 8:00	GEAR_42	400	250	0	400
2	2022.3.1 11:00	GEAR_21	0	150	150	100
3	2022.3.1 12:00	GEAR_42	0	50	200	400
4	2022.3.1 14:00	GEAR_21	0	300	150	100

[코드 4] 불러온 데이터의 상단 5개 형태 확인

- 불러온 데이터 내 변수명, 데이터 타입 등의 속성을 info()를 사용하여 확인가능하다.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 688 entries, 0 to 687
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   시간        688 non-null    object
1   품목        688 non-null    object
2   생산필요량  688 non-null    int64
3   재고        688 non-null    int64
4   생산량      688 non-null    int64
5   수주량      688 non-null    int64
dtypes: int64(4), object(2)
memory usage: 32.4+ KB
```

[코드 5] 데이터의 속성 확인

①-3. 분석하고자 하는 품목 데이터 선택하기

- 본 분석에서 사용되는 데이터는 주력 생산 상품인 2가지 품목에 대한 데이터이다. 분석하고자 하는 품목을 선택하기 위해 데이터의 '품목' 변수를 기준으로 선별한다. 비식별화된 품목 ID는 'GEAR_21'과 'GEAR_42'가 존재한다. 이를 먼저 value_counts() 메서드를 활용하여 데이터 수를 확인한다.

```
df['품목'].value_counts()
```

```
GEAR_21    344
GEAR_42    344
Name: 품목, dtype: int64
```

[코드 6] 데이터의 품목 코드 확인

- 본 분석에서는 두 품목 중 'GEAR_42' 품목의 실제 출고량을 예측하는 모델을 구축하고자 한다. 실습에서는 해당 품목의 데이터만 활용하지만, 독자가 실습내용을 기반으로 연습을 할 때는 다른 품목의 데이터를 사용해보면 된다. 'GEAR_42' 품목에 대한 데이터를 df_42라는 이름으로 저장한다. 'GEAR_21' 품목의 실제 출고량을 예측하는 모델을 구축할 때도 코드에서 품목 ID를 변경하여 비슷한 방식으로 분석을 진행할 수 있다.

```
df_42 = df[df['품목']=='GEAR_42']
```

[코드 7] GEAR_42 품목에 대한 데이터 선택 및 저장

- df_42 데이터는 원본 데이터에서 특정 변수(품목)를 기준으로 선별한 데이터이기 때문에 데이터의 순서, 즉 인덱스가 순차적이지 않게 된다. 이를 다시 0부터 초기화하기 위해 reset_index() 메서드를 사용한다.

```
df_42.reset_index(drop=True, inplace=True)
```

[코드 8] GEAR_42 품목에 대한 데이터 인덱스 초기화

[단계 ②] 데이터 탐색 및 EDA

②-1. 기초 통계 확인

- 분석에 사용할 품목인 'GEAR_42'에 대한 데이터인 df_42의 기초 통계를 확인한다.
pandas 패키지의 'describe()' 메서드를 사용하면 데이터의 행 개수, 평균, 표준편차 등의 통계치(statistics)를 확인할 수 있다.

```
df_42.describe()
```

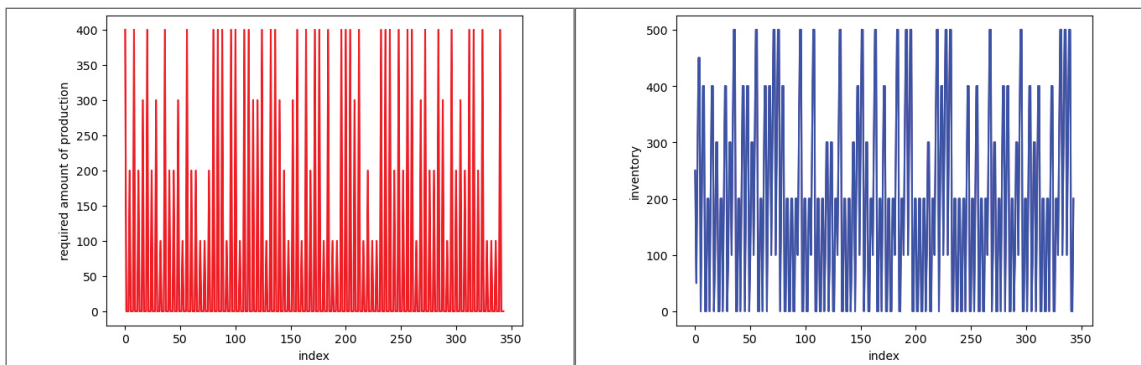
	생산필요량	재고	생산량	수주량
count	344.000000	344.000000	344.000000	344.000000
mean	68.895349	201.598837	150.000000	275.581395
std	134.415702	170.418971	86.728691	123.066785
min	0.000000	0.000000	0.000000	100.000000
25%	0.000000	0.000000	150.000000	200.000000
50%	0.000000	200.000000	200.000000	300.000000
75%	25.000000	300.000000	200.000000	400.000000
max	400.000000	500.000000	200.000000	400.000000

[코드 9] 데이터의 통계치 확인

②-2. 데이터 시각화

- matplotlib 패키지를 사용하여 df_42 데이터내 각 변수에 대한 수량을 시각화한다.
시각화 시에는 'plot()' 함수를 활용하면 된다. 선 그래프뿐만 아니라 산점도, 바 그래프 등 다양한 응용이 가능하다. 본 분석에서는 예시로 생산 필요량 및 재고량에 대한 시각화를 수행한다.

```
plt.plot(df_42['생산필요량'], color='red')
plt.xlabel('index')
plt.ylabel('required amount of production')
plt.show()
plt.plot(df_42['재고'], color='blue')
plt.xlabel('index')
plt.ylabel('inventory')
plt.show()
```



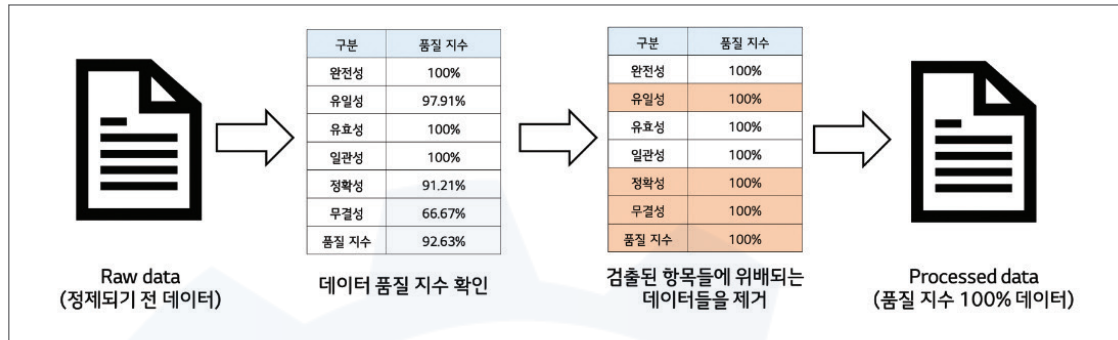
[코드 10] 데이터 시각화

[단계 ③] 데이터 정제(전처리)

③-1. 데이터 전처리

• 데이터 전처리 방법

- 개발 언어 : Python (파이썬)
- 개발 환경 : Jupyter Notebook (주피터 노트북)



[그림 21] 데이터 전처리 과정 도식화

③-2. 데이터 정제

- 데이터 내부에 'NaN(Not-a-Number)'으로 표기된 누락 데이터가 있는지 확인한다.
pandas 패키지의 'isna()' 메서드를 통해 데이터 내에 누락값 여부를 True 혹은 False로 확인할 수 있다.

```
df_42.isna()
```

	시간	품목	생산필요량	재고	생산량	수주량
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
...	...	...	...	...	...	...
339	False	False	False	False	False	False
340	False	False	False	False	False	False
341	False	False	False	False	False	False
342	False	False	False	False	False	False
343	False	False	False	False	False	False

344 rows × 6 columns

[코드 11] 데이터 누락값 여부 확인



- 'isna().sum()' 메서드를 사용하여 변수별 누락 데이터의 개수를 확인할 수 있다.

```
df_42.isna().sum()
```

```
시간      0
품목      0
생산필요량  0
재고      0
생산량     0
수주량     0
dtype: int64
```

[코드 12] 누락 데이터 개수 확인

- pandas의 duplicated() 메서드와 numpy의 sum() 메서드를 사용하여 데이터 내 중복된 행의 개수를 확인할 수 있다.

```
np.sum(df_42.duplicated())
```

```
0
```

[코드 13] 중복 데이터 개수 확인

③-3. 데이터 타입 변환

- df_42 데이터는 로그가 기록된 시각 정보가 '시간' 변수에 저장되어있다. 이를 분석의 용이함을 위해 변수 형태를 날짜 시간형(datetime)으로 변환한다. pandas 패키지의 'to_datetime()' 메서드를 사용하면 자동으로 년, 월, 일, 등 시간정보를 변환할 수 있다.

```
df_42['시간'] = pd.to_datetime(df_42['시간'])
```

[코드 14] 데이터 타입 변환

- 수집된 시간 순으로 데이터를 다시 정렬하기 위해 sort_values() 메서드를 사용한다. 메서드 안에 '시간' 변수를 기준으로 정렬하라는 조건을 주입한다.

```
df_42.sort_values(by='시간', inplace=True)
```

[코드 15] 중복 데이터 개수 확인

- 변환한 데이터 내 '시간' 변수를 기반으로 각 데이터의 수집 시점의 년도, 월, 일, 시, 분, 요일 정보를 새로운 변수로서 데이터에 추가한다. 이는 추후 시각화 및 분석을 용이하게 한다.

```
df_42['년'] = df_42['시간'].dt.year
df_42['월'] = df_42['시간'].dt.month
df_42['일'] = df_42['시간'].dt.day
df_42['시'] = df_42['시간'].dt.hour
df_42['분'] = df_42['시간'].dt.minute
df_42['요일'] = df_42['시간'].dt.dayofweek # 월요일=0
```

[코드 16] 날짜 시간형 변수의 형태 지정



③-4. 종속 변수 생성

- 본 분석에 활용되는 열처리 공급망 데이터 특성상 데이터에 기록된 생산필요량, 재고, 생산량, 수주량 외에 비정기적으로 잔여 재고를 모두 출고하는 경우가 빈번하다. 따라서, 실제 생산 공정을 통해 출고된 실제 출고량은 생산량과 재고 변화량을 고려하여 계산하여야 한다. 이렇게 계산되는 실제 출고량을 pandas의 diff() 메서드를 사용하여 새로운 변수인 '출고'로 설정한다.

```
df_42['출고'] = df_42.생산량 - df_42.재고.diff(periods=1)
```

[코드 17] '출고' 변수 생성

- 두 변수를 사용하여 서로 다른 시간의 값을 빼줌으로써 계산을 하였으므로, periods의 값만큼 '출고' 열에 빈 값(NaN)이 생기게 된다. 이를 0으로 바꾸기 위해 pandas의 replace() 메서드를 사용한다.

```
df_42.replace(to_replace=np.nan, value=0, inplace=True)
```

[코드 18] 빈 값 0으로 변경

- 추후 예측에 사용할 종속 변수인 '출고'를 생성한 뒤 df_42 데이터의 형태 확인을 위해 pandas의 tail() 메서드를 사용한다. 데이터의 마지막 5개 행을 볼 수 있다.

```
df_42.tail()
```

	시간	품목	생산필요량	재고	생산량	수주량	년	월	일	시	분	요일	출고
339	2022-06-27 20:00:00	GEAR_42	0	500	200	100	2022	6	27	20	0	0	0.0
340	2022-06-28 08:00:00	GEAR_42	400	500	0	400	2022	6	28	8	0	1	0.0
341	2022-06-28 12:00:00	GEAR_42	0	0	200	400	2022	6	28	12	0	1	700.0
342	2022-06-28 16:00:00	GEAR_42	0	0	200	400	2022	6	28	16	0	1	200.0
343	2022-06-28 20:00:00	GEAR_42	0	200	200	400	2022	6	28	20	0	1	0.0

[코드 19] 데이터의 하단 5개 형태 확인

[단계 ④] 주요 변수 탐색

④-1. 변수별 상관관계 파악

- 전체 수치형 변수 중 유의미한 상관관계를 가진 변수 조합을 파악하는 것은 분석 및 예측에 도움이 된다. 변수 간의 표준 상관관계 분석을 위해 pandas의 corr() 메서드를 사용하고 결과를 확인하였다. 분석 시에는 수치형 변수인 '생산필요량', '재고', '생산량', '수주량', '출고'를 선택하여 상관관계를 확인하였다. 상관관계수는 -1에서 +1 사이의 값을 가지며, 절댓값의 크기에 따라 상관관계의 강한 정도를 나타낸다.



```
df_42[['생산필요량', '재고', '생산량', '수주량', '출고']].corr()
```

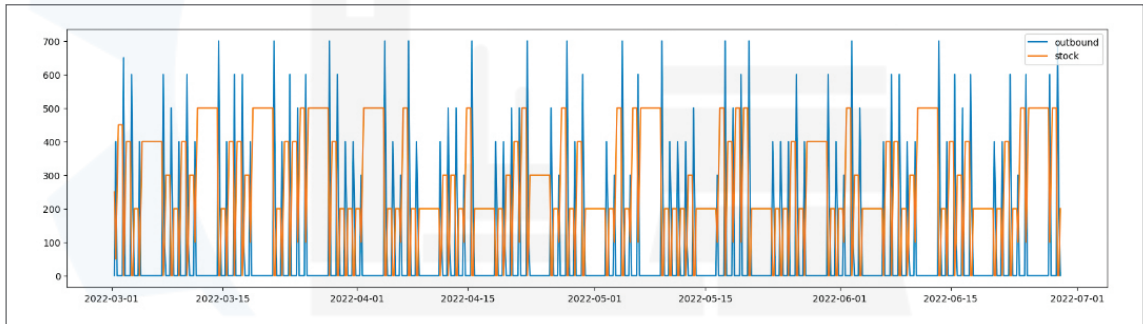
	생산필요량	재고	생산량	수주량	출고
생산필요량	1.000000	0.402452	-8.890633e-01	2.288921e-01	-0.345395
재고	0.402452	1.000000	-4.285337e-01	-3.769362e-01	-0.722901
생산량	-0.889063	-0.428534	1.000000e+00	2.037118e-17	0.388493
수주량	0.228892	-0.376936	2.037118e-17	1.000000e+00	0.152260
출고	-0.345395	-0.722901	3.884933e-01	1.522602e-01	1.000000

[코드 20] 수치형 변수간 상관관계 확인

④-2. 시각화를 통한 변수별 추이 파악 및 이를 기반으로 한 유의미한 변수 선택

- 앞서 확인한 상관분석 결과를 기반으로, 강한 상관관계를 보인 변수 조합인 실제 출고량 '출고' 변수와 재고량을 함께 시각화하여 전반적인 두 변수간의 관계와 추이를 확인한다.

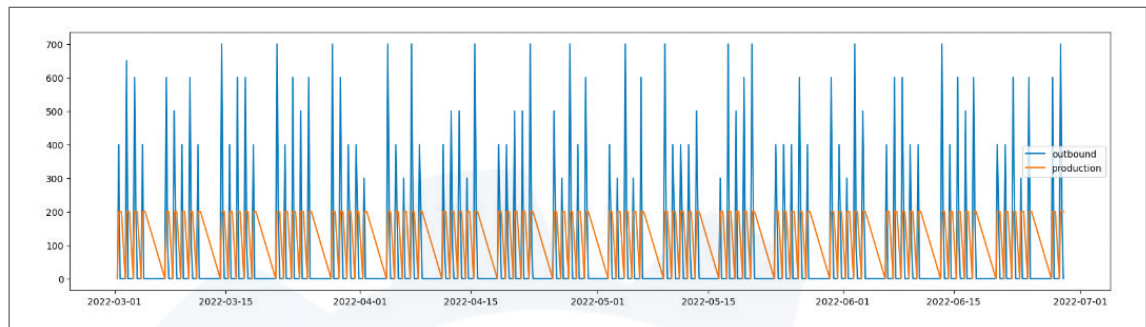
```
plt.figure(figsize = (20,5))
plt.plot(df_42['시간'], df_42['출고'], label = 'outbound')
plt.plot(df_42['시간'], df_42['재고'], label = 'stock')
plt.legend()
plt.show()
```



[코드 21] 시간에 따른 출고, 재고 변수값 시각화

- 추가로, 실제 출고량 '출고' 변수와 생산량을 함께 시각화하여 전반적인 두 변수간의 관계와 추이를 확인한다.

```
plt.figure(figsize = (20,5))
plt.plot(df_42['시간'], df_42['출고'], label = 'outbound')
plt.plot(df_42['시간'], df_42['생산량'], label = 'production')
plt.legend()
plt.show()
```



[코드 22] 시간에 따른 출고, 생산량 변수값 시각화

④-3. 일별 수량 데이터 분석 및 시각화

- 현재 df_42 및 원본 데이터는 하루마다 약 4회씩 수집된 수량 정보가 기록되어 있다. 이를 일별로 통합하여 분석을 수행함으로써 시간별로 분석한 결과에 추가로 새로운 인사이트를 얻고자 한다. 이를 위해 먼저 데이터를 월, 일별로 통합하고, 분석할 변수인 'out', '생산량'은 일별로 합산한다. '재고' 변수는 누적되어 기록되므로 합산이 불가능해 분석에서 제외한다. 일별로 통합한 데이터를 tmp_42로 저장한다.

```
tmp_42 = df_42.groupby(by=['월','일'])['생산량','출고'].sum()
tmp_42 = pd.DataFrame(tmp_42)
```

[코드 23] 생산량 및 출고 일별 합산

- '월', '일' 변수로 묶어 합산한 tmp_42 데이터의 인덱스를 초기화하여 월, 일 변수를 다시 풀어낸다.

```
tmp_42.reset_index(inplace=True)
tmp_42.head()
```

	월	일	생산량	출고
0	3	1	600	400.0
1	3	2	600	650.0
2	3	3	600	800.0
3	3	4	600	400.0
4	3	7	600	700.0

[코드 24] 일별 생산량 및 출고 데이터 tmp_42 생성

- '생산 필요량', '수주량' 변수는 하루에 한 번만 기록되므로 '월', '일' 변수를 기준으로 중복행을 삭제하여 하루에 하나의 데이터만 남도록 한다. '생산필요량' 변수는 매일 첫 시간에 기록되므로 keep = 'first' 조건을 통해 매일 첫 시간에 기록된 값만 남긴 데이터를 df_42_day로 저장한다.

```
df_42_day = df_42.drop_duplicates(subset=['월', '일'], keep='first')
```

[코드 25] 일별 데이터 df_42_day 생성

- 시각화 및 tmp_42, df_42_day 두 데이터를 합치는데 필요한 열만 남긴다.

```
df_42_day = df_42_day[['생산필요량', '수주량', '시간', '월', '일', '요일']]
```

[코드 26] 필요한 열 선택저장

- tmp_42, df_42_day 두 데이터를 병합하여 일별 '생산필요량', '수주량', '생산량' 및 '출고' 데이터를 확인한다.

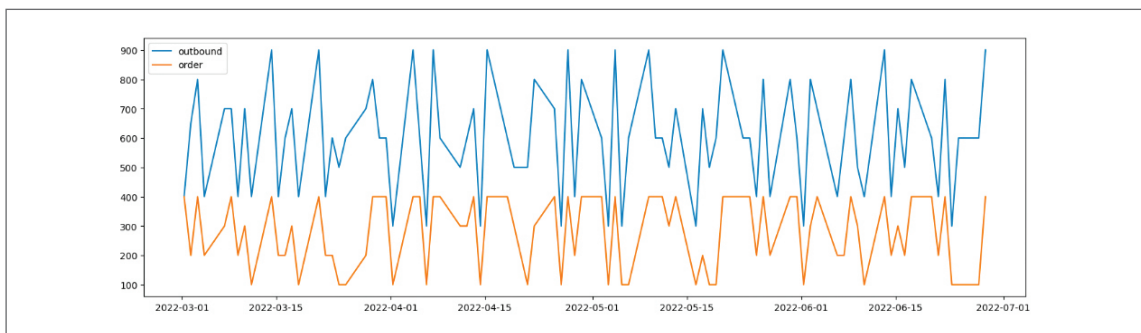
```
df_42_day = pd.merge(df_42_day, tmp_42, how='inner', on=['월', '일'])
df_42_day.head()
```

	생산필요량	수주량	시간	월	일	요일	생산량	출고
0	400	400	2022-03-01 08:00:00	3	1	1	600	400.0
1	200	200	2022-03-02 08:00:00	3	2	2	600	650.0
2	400	400	2022-03-03 08:00:00	3	3	3	600	800.0
3	200	200	2022-03-04 08:00:00	3	4	4	600	400.0
4	300	300	2022-03-07 08:00:00	3	7	0	600	700.0

[코드 27] 병합된 일별 데이터 확인

- 일별 수주량과 출고량의 관계를 확인하기 위해 '수주량', '출고' 변수를 '시간' 변수에 따라 함께 시각화한다.

```
plt.figure(figsize=(17,5))
plt.plot(df_42_day['시간'], df_42_day['출고'], label='outbound')
plt.plot(df_42_day['시간'], df_42_day['수주량'], label='order')
plt.legend()
plt.show()
```



[코드 28] 일별 출고, 수주량 변수값 시각화

[단계 ⑤] 모델의 입력 데이터 형태 설정

⑤-1. 시계열 형태로 데이터 변환

- 본 분석에서 사용하는 열처리 공급망에서 수집된 데이터는 순차적으로 수집된 시계열적인 특성을 어느 정도 가지고 있다. 하지만, 수집 시간의 간격이 일정하지 않고, 수집 시점 사이의 충분한 정보가 포함되어 있지 않은 경향이 있다. 또한, 기본적인 베이지안 신경망 실습을 위해 시계열 데이터로의 변환보다는 일반적인 2차원 데이터로 변환하여 분석을 수행하는 것이 적합할 것으로 사료된다. 본 분석에서는 과거 T개 시점의 수량 데이터를 활용하여 다음 시점(T+1)의 실제 출고량을 예측한다. 이는 미래 시점의 실제 출고량을 예측함으로써 미리 생산량 및 재고를 조절하여 보다 원활한 공급망 관리를 가능하게 도움을 줄 수 있다.
- 아래 [그림 22]처럼, 예측 대상(target, y)과 예측에 사용될 데이터(input, X)가 모델 학습을 위해 짝지어지고, 해당 과정이 전체 데이터셋에 대해 순차적으로 진행되어 적합한 형태의 데이터로 변환된다. 모델 관점에서는 학습에 활용되는 입력과 출력 데이터를 만드는 과정이라고 볼 수 있다.

시간	품목	생산 필요 량	재 고	생 산 량	수 주 량	year	month	day	hour	minute	day_of_week	out	
0	2022-03-01 08:00:00	GEAR_21	100	100	0	100	2022	3	1	8	0	1	100.0
1	2022-03-01 11:00:00	GEAR_21	0	150	150	100	2022	3	1	11	0	1	0.0
2	2022-03-01 14:00:00	GEAR_21	0	300	150	100	2022	3	1	14	0	1	0.0
3	2022-03-01 17:00:00	GEAR_21	0	450	150	100	2022	3	1	17	0	1	0.0
4	2022-03-02 08:00:00	GEAR_21	150	450	0	150	2022	3	2	8	0	2	600.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
339	2022-06-27 17:00:00	GEAR_21	0	300	150	150	2022	6	27	17	0	0	0.0
340	2022-06-28 08:00:00	GEAR_21	200	300	0	200	2022	6	28	8	0	1	450.0
341	2022-06-28 11:00:00	GEAR_21	0	0	150	200	2022	6	28	11	0	1	50.0
342	2022-06-28 14:00:00	GEAR_21	0	100	150	200	2022	6	28	14	0	1	0.0
343	2022-06-28 17:00:00	GEAR_21	0	250	150	200	2022	6	28	17	0	1	0.0

입력 데이터

출력(예측) 데이터

[그림 22] 분석 모델 활용을 위한 입력 및 출력 데이터 변환

- 앞서 설명한 분석 모델 활용을 위한 입력 및 출력 데이터 변환을 위해 본 분석에서는 사용자 정의 함수인 'transform_data'를 만들어 사용한다. 변환 전 데이터의 인덱스를 기준으로 미리 정해진 T 길이의 시점 데이터를 선택하여 입력 데이터 샘플로 생성



하고, 다음 시점인 T+1에 해당하는 실제 출고량 값을 출력 데이터 샘플로 생성한다. 이를 전체 데이터에 반복하여 X와 y 데이터를 반환하는 함수를 구성한다.

```
# 사용자 정의 함수 만들기
def transform_data(data, T):
    X, y = list(), list()
    for i in range(0, len(data)):
        try:
            y.append(data.iloc[i+T,-1])
            # 예측할 변수인 '출고'의 index인 -1 입력

            X_ = data.iloc[i:i+T,[2,3,4,5,11]].to_numpy().flatten()
            # 사용할 변수의 index인 2,3,4,5,11 입력
            X.append(X_)

        except:
            pass

    return np.vstack(X), np.array(y)
```

[코드 29] 데이터 변환용 사용자 정의 함수 만들기

- 본 분석에서는 과거 T=3개의 시점의 수량 데이터를 사용하여 다음 시점의 출고량을 예측하도록 한다. df_42를 사용하여 변환된 데이터 X, y의 형태(샘플 개수, 변수 개수)를 확인한다.

```
X_42, y_42 = transform_data(df_42, T=3)

print("입력 X 데이터 형태: {}".format(X_42.shape))
print("출력 y 데이터 형태: {}".format(y_42.shape))
```

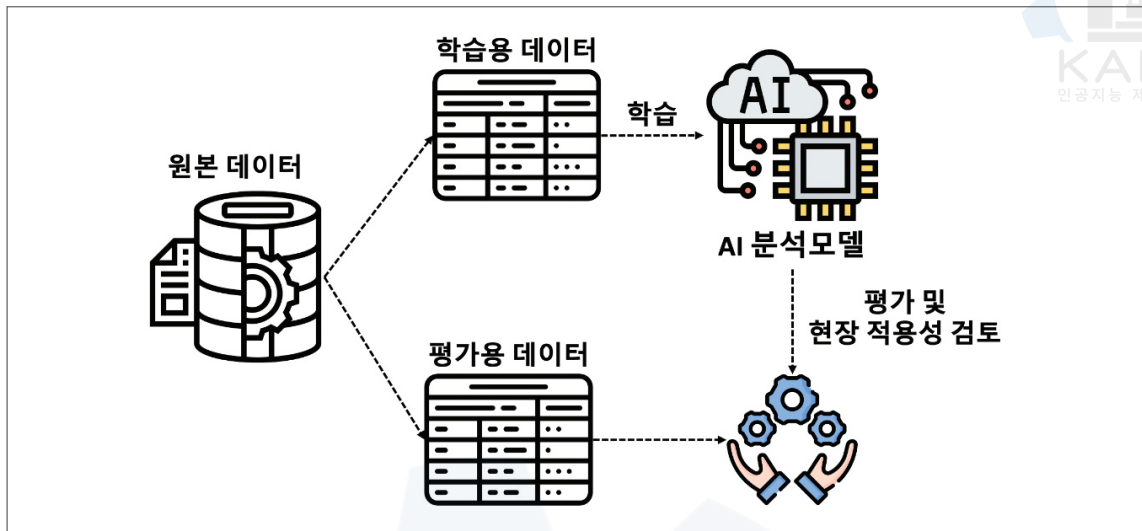
입력 x 데이터 형태: (341, 15) 출력 y 데이터 형태: (341,)

[코드 30] 분석에 맞게 사용자 정의 함수 만들기 및 적용

[단계 ⑥] 데이터 분리 및 정규화

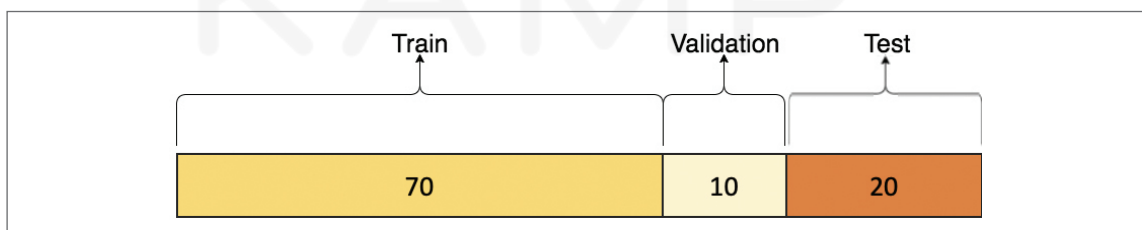
⑥-1. 학습, 검증, 평가용 데이터셋 분리

- 데이터 사이언스 및 기계학습 분야에서의 좋은(desirable) 모델은 학습 시에 활용된 적 없는 완전히 새로운 데이터에 대해 일반화 가능한 성능을 보이는 모델이라고 할 수 있다. 이러한 개념은 실제 제조 현장에서 AI 분석모델을 적용할 때도 해당된다. 따라서 평가 데이터를 사용하여 AI 분석모델의 일반화 성능을 확인하는 것이 보편적이다. 데이터셋을 용도에 맞게 총 3가지의 학습(train), 검증(validation), 평가(test) 데이터로 나눌 필요가 있다. 학습 데이터셋은 AI 분석모델 학습 시에 사용하고, 검증 데이터셋은 학습 데이터셋으로 훈련된 모델의 성능 측정 시에 사용된다. 모델의 성능이 검증 데이터셋을 통해 일차적으로 검증이 되면, 마지막으로 평가 데이터셋을 사용하여 모델의 성능을 최종적으로 판단한다.



[그림 23] AI 분석모델을 위한 원본 데이터 활용 방법

- 모델의 성능 판단을 위해 데이터셋을 검증용과 평가용으로 분리하는 이유는 모델이 학습 데이터에 과적합(overfitting)되지 않았는지 확인하기 위한 목적이다. 과적합된 모델은 학습 데이터에 대해서는 잘 작동하지만, 실제 산업 현장에서 사용될 본 적 없는 데이터(unseen data)에 대해서는 낮은 성능을 보인다. 학습 중간에 검증 데이터를 사용하여 모델의 성능을 검증하고, 이를 바탕으로 모델의 초매개변수를 조정함으로써 학습된 모델이 평가 데이터셋에도 잘 작동하는 우수한 일반화 성능(generalization power)을 확보할 수 있다.
- 본 분석에서는 전체 데이터셋의 70%, 10%, 20%를 각각 학습, 검증, 평가 데이터로 활용한다. 특히 전처리 이후의 시계열 데이터가 시간 순으로 존재하므로, 학습-검증-평가 데이터를 시간순으로 분할한다. numpy 패키지에서 제공하는 'split()' 함수로 데이터가 분리되는 구간을 지정하여 데이터를 분할한다. (70%와 80% 위치에서 데이터 분할 실시)



[그림 24] 학습, 검증, 평가용 데이터셋 분리

```

X_train_42, X_val_42, X_test_42 = np.split(X_42, [int(0.7*len(X_42)), int(0.8*len(X_42))])
y_train_42, y_val_42, y_test_42 = np.split(y_42, [int(0.7*len(y_42)), int(0.8*len(y_42))])
  
```

[코드 31] 데이터셋 분리



- 학습, 검증, 평가 데이터로 분리 후 X, y 각각의 형태를 확인한다.

```
print("X 학습: {}, X 검증: {}, X 평가: {}".format(X_train_42.shape, X_val_42.shape, X_test_42.shape))
print("y 학습: {}, y 검증: {}, y 평가: {}".format(y_train_42.shape, y_val_42.shape, y_test_42.shape))
```

```
x 학습: (238, 15), x 검증: (34, 15), x 평가: (69, 15)
y 학습: (238,), y 검증: (34,), y 평가: (69,)
```

[코드 32] 데이터셋 분리 및 형태 확인

⑥-2. 데이터 스케일링 (Data Scaling)

- AI 분석모델의 입력 데이터로 활용하기 위해서는 데이터 변수의 크기를 적절히 조정할 필요가 있다. 값의 범위가 다른 변수들이 있는 경우, 모델 학습 시 변수별로 공평하게 기여하지 못하거나 편향이 발생할 수 있다. 이를 방지하기 위해 전체 데이터를 정규화(normalization)함으로써 데이터가 0과 +1구간 안에 존재하도록 변환한다. 각 데이터에서 최솟값을 빼고, 최댓값과 최솟값의 차이인 범위값(range)로 나눔으로써 아래 [그림 25]의 식과 같이 값을 변환할 수 있다. 데이터 정규화는 각 변수별로 실시한다.

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

[그림 25] 데이터 정규화 공식

- scikit-learn (sklearn) 에서 제공하는 MinMaxScaler를 사용하여 학습 데이터를 기준으로 변수별 최솟값, 최댓값, 범위값을 계산한 후 학습, 검증, 평가 데이터셋에 정규화를 실시한다. 학습 데이터의 경우 'fit_transform()' 메서드를 활용하여 통계치 계산하기(fit)와 정규화 적용(transform)을 동시에 실행하고, 검증 및 평가 데이터에는 'transform()' 메서드를 통해 정규화만 실행한다.

```
from sklearn.preprocessing import MinMaxScaler
```

```
X_scaler_42 = MinMaxScaler()
```

```
X_train_42 = X_scaler_42.fit_transform(X_train_42)
```

```
X_val_42 = X_scaler_42.transform(X_val_42)
```

```
X_test_42 = X_scaler_42.transform(X_test_42)
```

[코드 33] 데이터 정규화



[단계 ⑦] 모델 구축/정의 및 학습

⑦-1. 분석모델 구축

- 본 분석에서 예측을 수행할 베이지안 신경망 모델을 구축하기 위하여 TensorFlow 라이브러리와 TensorFlow에서 제공하는 확률적 모델링을 위한 라이브러리인 TensorFlow probability를 활용한다. 각각의 라이브러리를 약어(alias)인 tf와 tfp로 불러온다.

```
# 필요한 패키지 불러오기
import tensorflow as tf
tf.random.set_seed(42)
import tensorflow_probability as tfp
```

[코드 34] 분석에 필요한 필요 패키지 불러오기

- 본 분석에서는 TensorFlow probability에서 제공하는 재모수화 기반의 신경망 층(레이어)을 활용한다. 먼저, 앞서 설명한 ELBO 최적화를 위해 필요한 쿨백-라이블러 발산을 신경망 학습 시 계산하는 함수를 만든다. 여기에서는 파이썬 내 lambda 표현식을 사용하는데, lambda는 수식 형태로 함수와 같은 역할을 할 수 있고 간편하게 작성할 수 있기 때문에, 다른 함수의 인수로 자주 사용된다.

```
kl_divergence_function = (lambda q, p, _ : tfp.distributions.kl_divergence(q, p) //
                          tf.cast(X_train_42.shape[0], dtype=tf.float32))
```

[코드 35] 쿨백-라이블러 발산 계산 함수 생성

- 분석모델의 활성화 함수로는 앞서 설명한 Leaky ReLU 함수를 사용한다. 실습에서는 TensorFlow 라이브러리 내 Keras 패키지에서 제공하는 layers 모듈에서 불러와 사용한다. 해당 모듈은 활성화 함수뿐만 아니라 여타 다른 신경망 층의 구현체를 활용하기에 용이하다.

```
leaky_relu = tf.keras.layers.LeakyReLU()
```

[코드 36] Leaky ReLU 함수 불러오기

- 먼저 TensorFlow 내 keras 패키지에서 제공하는 Sequential 모델을 사용하여 전체적인 베이지안 신경망의 틀을 구성한다. 이후 연쇄적으로 신경망 층을 정의하고 이를 Sequential 모델 틀에 추가함으로써 사용자 정의에 따른 베이지안 신경망을 구성할 수 있다. 모델 정의 과정에서 직접 신경망 층의 개수, 각 신경망 층의 유닛 개수 및 활성화 함수 등 다양한 초매개변수를 설정할 수 있다. 본 실습에서는 3개의 은닉층을 가지는 베이지안 신경망을 구성한다. 또한 출력층을 마지막 신경망 층으로 구성한다. 추가로, 첫 번째 신경망 층에는 입력 데이터의 형태를 인자로 제공할 수 있다.



```

model = tf.keras.Sequential([
])

layer_1 = tf.layers.DenseReparameterization(units=32, input_shape=(X_train_42.shape[1],), kernel_divergence_
fn=kl_divergence_function, activation=leaky_relu)
layer_2 = tf.layers.DenseReparameterization(units=32, kernel_divergence_fn=kl_divergence_function,
activation=leaky_relu)
layer_3 = tf.layers.DenseReparameterization(units=32, kernel_divergence_fn=kl_divergence_function,
activation=leaky_relu)
output_layer = tf.layers.DenseReparameterization(units=1, activation=leaky_relu)

model.add(layer_1)
model.add(layer_2)
model.add(layer_3)
model.add(output_layer)

```

[코드 37] 베이지안 신경망 모델 정의

- 구성된 베이지안 신경망인 model의 구조 확인을 위해 summary() 메서드를 사용한다.

```
model.summary()
```

Layer (type)	Output Shape	Param #
dense_reparameterization (DenseReparameterization)	(None, None, 32)	992
dense_reparameterization_1 (DenseReparameterization)	(None, None, 32)	2080
dense_reparameterization_2 (DenseReparameterization)	(None, None, 32)	2080
dense_reparameterization_3 (DenseReparameterization)	(None, None, 1)	65
Total params: 5,217		
Trainable params: 5,217		
Non-trainable params: 0		

[코드 38] 정의된 모델의 구조 확인

⑦-2. 모델 컴파일 (compile)

- 모델을 훈련시키기 전에 훈련 과정에 필요한 매개변수를 정의해야 한다. 훈련 매개변수는 'compile()' 메서드에 전달하며, 메서드를 실행할 때 필요한 매개변수는 다음과 같다.

- (1) 손실 함수(loss) : 실제 값과 예측값의 차이를 계산하는 함수이며, 값이 작을수록 오차가 적음을 의미한다. 본 실습에서는 연속형 변수인 실제 출고 수량을 예측하는 회귀 문제를 수행하므로, 이에 적합한 목적식을 활용해야 한다. 앞서 설명한 ELBO를 최적화하기 위해서는 model을 구성하는 과정에서 자동으로 추가된 쿨백-라이블러 발산 오차와 음의 로그 우도(negative

log likelihood, NLL)를 최소화한다. 사실 이론적으로 음의 로그 우도를 최소화하는 것은 회귀 문제에서 평균 제곱 오차(mean squared error, MSE)를 최소화하거나 분류 문제에서 교차 엔트로피(cross entropy)를 최소화하는 것과 동일하다.

$$NLL = -\log p(y|x, w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{1}{2\sigma^2} (y - w^T x)^2\right\}$$

[그림 26] 음의 로그 우도

```
def neg_log_likelihood(y_obs, y_pred, sigma=5):
    dist = tfp.distributions.Normal(loc=y_pred, scale=sigma)
    return tf.keras.backend.sum(-dist.log_prob(y_obs))
```

[코드 39] 손실함수 정의

- (2) 옵티마이저(optimizer) : 앞서 정의한 손실함수 값을 최소화하는 신경망의 모수를 찾을 수 있도록 하게 하는 최적화 알고리즘이다. 그 예시로 경사 하강법이 있다. 본 실습에서는 다양한 인공신경망 모델 구조에서 좋은 성능을 보이는 ‘Adaptive Moment Estimation (ADAM)’을 옵티마이저로 사용한다.

```
optimizer = tf.optimizers.Adam()
```

[코드 40] 옵티마이저 정의

- 모델 컴파일 시 앞서 정의한 손실함수와 옵티마이저를 사용하여 ‘compile()’ 메서드에 전달함으로써 학습 준비를 마친다.

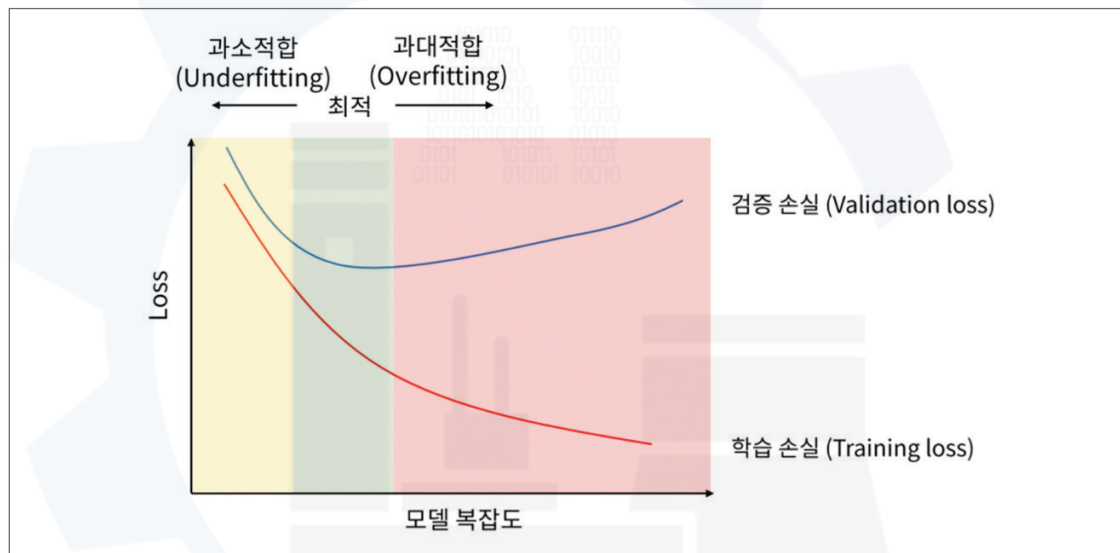
```
model.compile(optimizer=optimizer, loss=neg_log_likelihood)
```

[코드 41] 모델 컴파일(compile)

⑦-3. 모델 학습 및 조건

- 딥러닝 혹은 인공신경망 기반 모델 학습 시에는 CPU 혹은 GPU를 사용할 수 있다. 일반적으로 사진, 동영상과 같이 용량이 큰 데이터셋 사용 시 병렬처리를 통한 학습 속도 향상을 위해 GPU를 사용한다. 하지만 이런 경우에는 GPU를 추가로 구매하고 설치해야 하므로, 작은 데이터셋을 사용하여 분석할 경우에는 CPU만을 사용하여 AI 분석모델을 학습하기도 한다. 본 분석에서 사용되는 학습 데이터셋은 용량이 크지 않으므로, 본 분석에서는 학습 과정의 편의를 위해 CPU를 사용한다.
- 최종적으로 모델 훈련을 위해 ‘fit()’ 메서드를 사용하며, 필요한 매개변수는 다음과 같다.
 - (1) 입력 및 출력 데이터 : 훈련 데이터의 X와 y가 각각 입력과 출력 데이터이며, 모델을 학습시키는 데 활용된다.
 - (2) 에폭(epochs) : 모델이 전체 데이터를 순회하며 학습한 횟수를 의미하며, 에폭이 클수록 학습 정도는 증가하지만 횟수가 너무 클 경우 과적합 현상이 발생하므로 적절한 훈련 횟수를 설정하는 것이 중요하다.

- (3) 배치 사이즈(batch_size) : 한 번의 신경망 모수 업데이트시 사용하는 데이터의 개수를 의미한다. 배치의 크기는 인공신경망 학습 및 성능에 영향을 미친다. 전체 훈련 데이터를 한 번에 학습시키면 학습 시간이 오래 걸리거나 일반화 성능이 떨어지는 것이 관측되기도 하며, 너무 적은 양으로 쪼개어 학습시키면 각 실행 결과의 변동이 생겨 전체 결과값이 불안정해질 수 있으므로 적절한 배치의 크기를 찾아야 한다.
- (4) 검증용 데이터(validation_data) : 앞서 전체 데이터에서 분리해둔 검증 데이터를 사용한다. 검증 데이터를 이용하면 모델의 과적합 여부와 같은 학습 정도를 파악할 수 있다. 모델의 과적합 여부를 판단하는 방법은 [그림 27]처럼 학습 데이터에 대한 손실은 지속적으로 떨어지지만 검증 데이터에 대한 손실은 상승하는 것을 확인하는 것이다. 이는 학습 데이터에 지나치게 학습되어 오차가 감소하지만, 실제 평가 데이터와 유사할 것으로 가정한 검증 데이터에 대해서는 예측 오차가 증가하는 것을 의미한다. 과적합을 방지하기 위해서는 학습 조기 종료(early stopping) 방법을 이용하여 과적합이 일어나기 전의 최적의 구간에서 모델의 학습을 중지해야 한다.



[그림 27] 과적합 판단 기준

- (5) 콜백(callback) : 콜백은 모델의 'fit()' 메서드가 호출될 때 전달되는 객체이다. 훈련하는 동안 모델은 여러 지점에서 콜백을 호출하여 모델의 상태와 성능에 대한 정보에 접근하여, 훈련을 중지하거나 모델을 저장하고 가중치를 적재하거나 모델 상태를 변경할 수 있다.
- 본 실습에서는 TensorFlow 라이브러리의 Keras 패키지 내 Callback (콜백) 함수의 학습 조기 종료(early stopping) 기능을 사용하여 신경망의 과적합을 방지하였다. 학습 중 관찰하는(monitor) 수치에 대하여 모델에 악영향을 끼치고 있다고 판단될 시, 모델의 학습을 전체 에폭만큼 훈련되기 전에 조기 종료하도록 학습을 계획하였다. patience는 예측 결과의 개선이 없음에도 즉시 학습을 종료하지 않고 개선이 없는 에폭을 얼마나 기다릴지를 설정하는 값이다. 본 실습에서는 검증용 데이터셋의 손실 값

(val_loss)에 대하여 30회 이상 개선이 관찰되지 않으면 학습을 조기 종료하도록 설정하였다.

- 또한, 모델을 저장할 때 사용하는 콜백 함수인 모델 체크포인트(ModelCheckpoint)로 최적의 모델을 저장하였다. 관찰하는 대상인 검증용 데이터셋의 손실값(monitor=val_loss)이 가장 우수할 때(save_best_only=True), 즉, 가장 낮을 때의 에폭에서 모델을 저장함으로써 개선된 모델을 선택하여 추후에 사용할 수 있도록 하였다. 인공지능망 기반 분석모델은 매번 같은 학습 결과를 도출하지는 않기 때문에, 아래 [코드 43]의 학습 결과와 다른 결과가 도출될 수 있다.

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

save_path = './model/{epoch:02d}_{val_loss:.4f}'
callbacks = [EarlyStopping(monitor='val_loss', patience=30),
             ModelCheckpoint(filepath=save_path, monitor='val_loss', verbose=0, save_best_only=True)]
```

[코드 42] 모델 학습을 위한 콜백 설정

- 베이지안 신경망의 특성상 일반적인 인공지능망 모델과 비교할 때, 같은 구조임에도 학습시킬 모수가 2배 정도 증가하게 된다. 이는 고정된 모수 값을 찾는 것이 아니라 모수가 따르는 확률분포, 즉 사후 분포(변분 분포)의 매개변수를 찾게 되기 때문이다. 따라서, 학습을 위한 최대 에폭의 개수도 비교적 큰 수를 설정하였다.

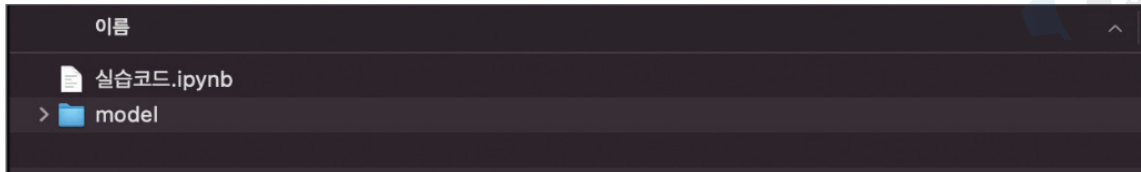
```
import os
os.makedirs('./model') # model 폴더 생성

history = model.fit(X_train_42, y_train_42, epochs=5000, batch_size=8, validation_data=(X_val_42, y_val_42),
                   callbacks=callbacks)
```

```
Epoch 1/5000
30/30 [=====] - 1s 15ms/step - loss: 11634.2861 - val_loss: 10377.9746
Epoch 2/5000
30/30 [=====] - 0s 5ms/step - loss: 11622.5273 - val_loss: 10287.3730
...
...
...
Epoch 165/5000
30/30 [=====] - 0s 3ms/step - loss: 157.8071 - val_loss: 179.8459
Epoch 166/5000
30/30 [=====] - 0s 3ms/step - loss: 175.5542 - val_loss: 157.2750
```

[코드 43] 모델 학습 및 저장

- 만약 모델이 저장되는 파일 경로인 model 폴더가 자동으로 생성되지 않는 경우 [그림 28]처럼 직접 실습 코드와 동일한 폴더 내에 model 폴더를 직접 만들 수 있다. os 모듈의 makedirs() 메서드를 활용한다. 이후 [코드 43]를 사용하여 모델 학습 및 저장을 실시한다.



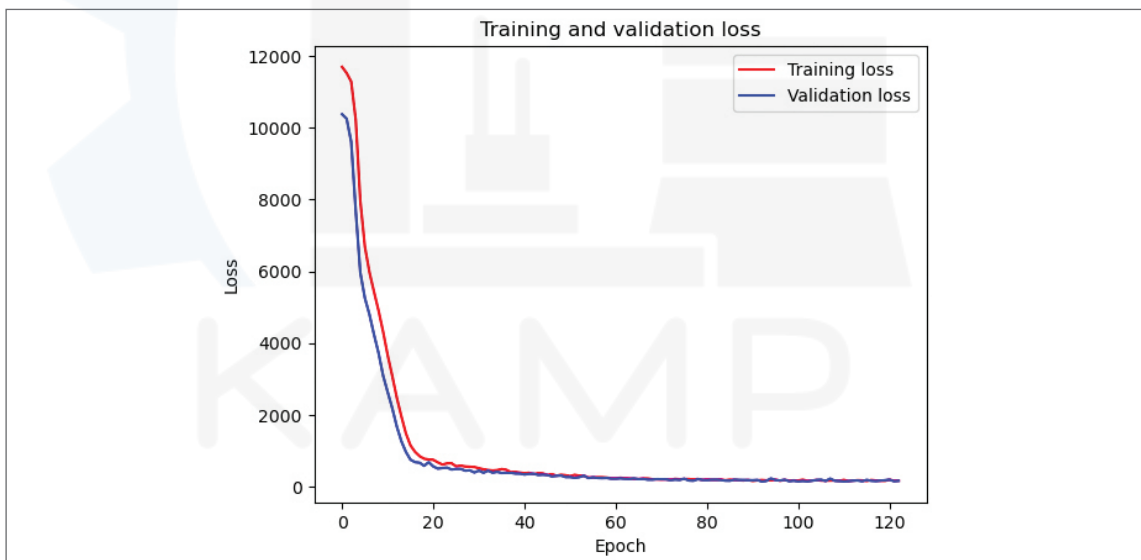
[그림 28] 실습 코드와 동일한 경로 내 model 폴더 생성

⑦-4. 학습 결과 확인

- matplotlib 패키지를 활용하여 에폭별 학습 진행 상황을 확인할 수 있다. 'plot()' 함수로 학습 손실값과 검증 손실값을 그래프로 나타내고, 'title()' 과 'legend()' 함수로 제목과 범례를 만들고 난 후, 'show()' 함수로 해당 그래프를 시각화한다.

```
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = history.epochs

plt.figure()
plt.plot(epochs, loss, color='red', label='Training loss')
plt.plot(epochs, val_loss, color='blue', label='Training loss')
plt.title('Training and validation loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



[코드 44] 학습 손실값과 검증 손실값 시각화

[단계 ⑧] 모델 성능 확인

⑧-1. 평가용 데이터셋에 대한 예측 성능 확인

- 학습이 완료된 모델을 불러오기 위해 TensorFlow 라이브러리의 load_model() 메서드를 사용하므로 이를 불러온다. 또한, 특정 폴더 내 파일을 선택하기 위해 경로상에 존재하는 파일 이름을 확인할 수 있는 os 모듈의 'listdir()'를 사용한다. 확인하고자 하



는 폴더의 경로를 'listdir()'의 인자로 사용하면 해당 폴더 내 존재하는 파일과 하위 폴더 등을 확인할 수 있다.

```
from tensorflow.keras.models import load_model
from os import listdir
```

```
listdir('./') #현재 폴더 확인
```

```
[ 'data.csv',
  '분석코드.ipynb',
  'model' ]
```

[코드 45] 현재 폴더 내 파일 확인

- model 폴더 내 저장된 학습 모델 중 가장 낮은 val_loss 값을 가진 좋은 모델을 불러온다. 콜백 함수의 모델 체크포인트를 이용하여 val_loss가 최저일 때마다 모델을 저장하였으므로, 모델 저장 경로 내의 파일들 중 가장 마지막에 저장된 모델을 선택한다.

```
best_model_path = listdir('./model/')[-1]
best_model = load_model('./model/'+best_model_path, compile=False)
```

[코드 46] 학습된 모델 불러오기

- 재모수화를 사용하여 구성한 베이지안 신경망의 모수는 결국 모수가 따르는 분포의 평균과 표준편차(정규분포일 경우)이다. 이는 best_model의 'weights'라는 attribute로 확인할 수 있다.

```
weights = best_model.weights
```

```
for weight in weights:
    print(weight.name)
```

```
dense_reparameterization/kernel_posterior_loc:0
dense_reparameterization/kernel_posterior_untransformed_scale:0
dense_reparameterization/bias_posterior_loc:0
dense_reparameterization_1/kernel_posterior_loc:0
dense_reparameterization_1/kernel_posterior_untransformed_scale:0
dense_reparameterization_1/bias_posterior_loc:0
dense_reparameterization_2/kernel_posterior_loc:0
dense_reparameterization_2/kernel_posterior_untransformed_scale:0
dense_reparameterization_2/bias_posterior_loc:0
dense_reparameterization_3/kernel_posterior_loc:0
dense_reparameterization_3/kernel_posterior_untransformed_scale:0
dense_reparameterization_3/bias_posterior_loc:0
```

[코드 47] 학습된 모델의 weights 확인

- 학습된 베이지안 신경망은 매 예측을 수행할 때마다 학습된 변분 분포에서 모수를 샘플링하여 활용한다. 따라서, 매번 같은 입력 데이터에 대해서도 다른 예측값을 출력한다. 예를 들어, 첫 번째 테스트 데이터 샘플에 대해 10회 예측을 수행하였을 때 매번 서로 다른 모수 조합을 활용하므로 상이한 예측값을 출력하는 것을 확인할 수 있다.



```
sample_data = X_test_42[0].reshape((1,X_test_42[0].shape[0]))

for _ in range(10):
    pred = best_model.predict(sample_data)
    print(pred)
```

```
1/1 [=====] - 0s 19ms/step
[[-5.0277953]]
1/1 [=====] - 0s 19ms/step
[[-3.6768775]]
1/1 [=====] - 0s 19ms/step
[[-4.6145167]]
1/1 [=====] - 0s 19ms/step
[[-2.8577619]]
1/1 [=====] - 0s 19ms/step
[[-4.1426697]]
1/1 [=====] - 0s 18ms/step
[[-4.6293955]]
1/1 [=====] - 0s 18ms/step
[[-3.0340338]]
1/1 [=====] - 0s 19ms/step
[[-3.763231]]
1/1 [=====] - 0s 19ms/step
[[-3.7648218]]
1/1 [=====] - 0s 18ms/step
[[-4.028168]]
```

[코드 48] 모수 샘플링에 따른 예측값 출력 확인

- 앞서 설명한 바와 같이, 베이지안 신경망 모델은 몬테카를로 방법론 기반으로 N회 예측을 수행하여 얻어지는 예측 분포를 사용한다. 이러한 몬테카를로 예측을 위해 사용자 정의 함수인 'MC_predict'를 만들어 사용한다. 특정 모델, 샘플링 횟수, 예측을 수행할 테스트 데이터를 사용하여 구해진 예측 분포의 평균과 표준편차를 반환하는 함수를 구성한다.

```
def MC_predict(model, num, X_test):
    preds = list()
    for _ in range(num):
        preds.append(model.predict(X_test, verbose=0))

    mean = np.mean(preds, axis=0).flatten()
    std = np.std(preds, axis=0).flatten()

    return mean, std
```

[코드 49] 예측용 사용자 정의 함수 만들기

- 테스트 데이터에 대해 몬테카를로 샘플링을 10회 수행한 예측분포의 결괏값을 출력한다.



```
y_pred_mean, y_pred_std = MC_predict(best_model, 10, X_test_42)
y_pred_mean
```

```
array([-4.65271997e+00, -2.45829463e+00,  3.39545807e+02,  4.49556398e+0
0,
      -1.01131611e+01, -2.92660046e+00,  5.57181152e+02,  1.12555895e+0
1,
      -7.96758413e+00, -2.81406736e+00,  6.11500183e+02,  2.00172638e+0
2,
      -4.05825901e+00, -2.86477232e+00,  3.80703552e+02,  8.79001465e+0
1,
      -6.85455799e+00, -2.58068132e+00,  4.19341797e+02, -2.41757417e+0
0,
      -8.79543304e+00, -4.60554659e-01,  7.00738892e+02,  2.11923096e+0
2,
      -3.42496943e+00, -3.26996803e+00,  3.51987183e+02,  3.07322979e+0
0,
      -9.84418201e+00, -2.81406736e+00,  5.83640991e+02,  1.03023727e+0
2,
      -4.75329065e+00, -2.72999716e+00,  4.45553131e+02,  4.05754089e+0
0,
      -8.24167538e+00, -2.66426897e+00,  5.98460938e+02,  1.90189163e+0
2,
      -2.75462008e+00, -2.36317372e+00,  4.01001831e+02,  1.80907425e+0
2,
      -7.06597424e+00, -2.88357782e+00,  3.50466278e+02,  2.11315775e+0
0,
      -9.54016113e+00, -2.85861397e+00,  6.00780579e+02,  1.92655182e+0
2,
      -3.35167360e+00, -2.61452365e+00,  3.21167480e+02, -3.35088658e+0
0,
      -9.98406982e+00, -8.41269791e-01,  5.86878540e+02, -8.84629726e-0
1,
      -7.33782291e+00, -8.59201074e-01,  5.76783325e+02,  1.91127092e-0
1,
      -7.11081839e+00,  5.42804360e-01,  7.14412048e+02,  2.18120392e+0
2,
      -1.77283216e+00], dtype=float32)
```

[코드 50] 평가 데이터셋에 대한 예측분포의 평균 확인

- 동일한 예측분포의 표준편차를 출력한다.

```
y_pred_std
```

```
array([ 1.2339956 ,  0.5521912 ,  9.207887 ,  5.719354 ,  0.65562755,
        0.8967532 ,  8.627282 ,  6.0920954 ,  0.6033356 ,  0.80372983,
       10.346913 , 12.361472 ,  0.98202384,  0.6070765 , 10.056386 ,
       11.077801 ,  0.9329187 ,  0.570653 , 11.332705 ,  0.87314796,
        0.79606915,  0.5365906 , 11.564653 , 15.219406 ,  0.8562707 ,
        0.87706286, 10.257204 ,  3.083022 ,  0.58573574,  0.80372983,
        9.016797 , 11.313171 ,  0.67272943,  0.30543113, 10.265393 ,
        3.9979026 ,  0.5157825 ,  0.24528232, 11.393537 , 13.40964 ,
        0.93088734,  0.63666767,  9.155274 , 12.327022 ,  0.7684885 ,
        0.43836403,  9.994751 ,  3.1626277 ,  0.6268408 ,  0.23082083,
        9.970509 , 13.43706 ,  0.80676186,  0.5091856 , 10.944358 ,
        0.5392219 ,  0.36057794,  0.21654192,  9.851467 ,  0.7623899 ,
        0.31397757,  0.24899228,  7.76606 ,  2.1868331 ,  1.206331 ,
        0.9450525 , 21.998978 , 10.611172 ,  0.98706865], dtype=float32)
2)
```

[코드 51] 평가 데이터셋에 대한 예측분포의 표준편차 확인

- sklearn (scikit-learn)의 metrics 모듈에서 평균 절대 오차(mean absolute error, MAE), 평균 제곱 오차(mean squared error, MSE), R 제곱(결정 계수)를 불러와 사용한다. 이 세 가지 기준으로 예측 결과와 실제 값을 비교한다.


```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test_42, y_pred_mean)
mse = mean_squared_error(y_test_42, y_pred_mean)
r2 = r2_score(y_test_42, y_pred_mean)

print("MAE: {}, MSE: {}, R\u00b2: {}".format(mae, mse, r2))

```

MAE: 9.671585368505422, MSE: 261.75400395180696, R²: 0.9949155005597121

[코드 52] 평가 데이터셋에 대한 성능 확인

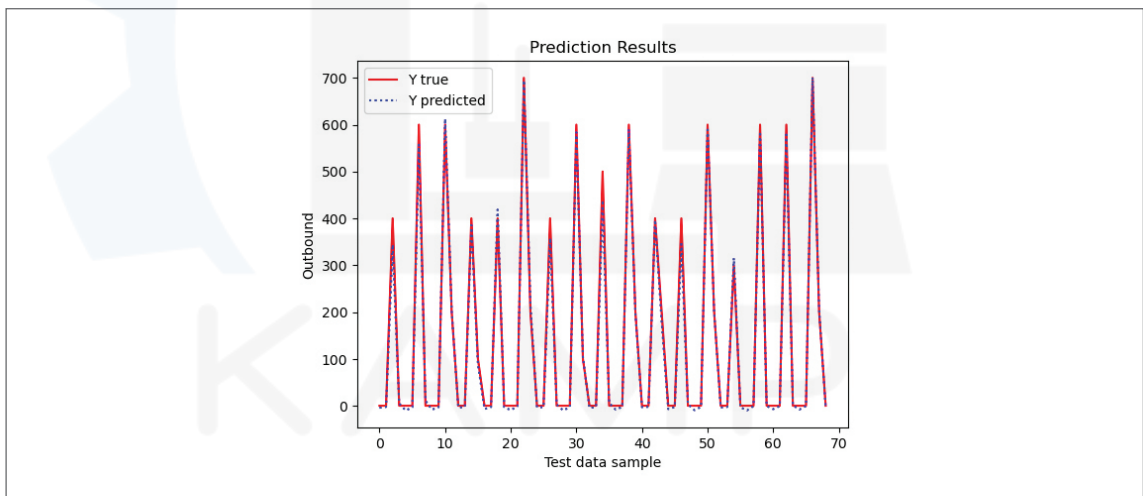
⑧-2. 예측 결과 시각화

- 이전 단계에서 몬테카를로 기반 예측을 통해 얻은 예측 평균과 표준편차 정보를 활용하여 데이터 시각화를 수행한다.

```

plt.figure()
plt.plot(range(len(X_test_42)), y_test_42, color='red', label='Y true')
plt.plot(range(len(X_test_42)), y_pred_mean, color='blue', label='Y predicted', linestyle='dotted')
plt.title('Prediction Results')
plt.xlabel('Test data sample')
plt.ylabel('Outbound')
plt.legend()
plt.show()

```



[코드 53] 예측 및 예측 결과 확인

⑧-3. 분석모델 예측 강건성 시험

- 확률론적 모델링 기반의 베이지안 신경망은 앞서 여러 차례 설명된 바와 같이, 불확실성을 고려한 예측을 수행할 수 있다. 또한, 예측에 대한 신뢰도를 얻을 수 있다는 장점이 있다. 본 분석에서는 예측 분포에서 얻은 평균뿐만 아니라 표준편차를 사용하여 예측의 신뢰도를 짐작해본다. 예를 들어 예측분포의 표준편차가 작다면, 모델이 높은 확신을 가지고 예측을 수행했다고 볼 수 있다. 반면에, 예측의 표준편차가 큰 경우, 매번

큰 차이가 나는 예측값을 출력한다고 볼 수 있으므로 모델의 확신도가 낮다고 이해할 수 있다. 이를 검증하기 위해 학습 데이터와 추이가 다른 데이터 샘플 하나를 랜덤하게 생성한다.

```
anomaly_sample = np.array([np.random.random(size=X_42.shape[1])])
```

[코드 54] 랜덤 샘플 데이터 생성

- 기존 정상적인 테스트 데이터의 예측분포의 평균적인 편차를 구하기 위해 표준오차를 계산한다. 이는 평균적으로 모델 예측이 어느 정도 크기의 편차를 가지는지를 의미하며, 평균적인 상대적 확신도로 볼 수 있다.

```
np.mean(y_pred_std)
```

```
4.7997537
```

[코드 55] 정상 평가 데이터의 표준오차

- 랜덤하게 생성한 데이터에 대한 몬테카를로 기반 예측분포를 계산하고, 이에 대한 표준편차를 계산한다. 'MC_predict' 함수가 반환하는 두 번째 값이 표준편차인데, 정상적인 테스트 데이터의 그것과 비교할 때 값이 훨씬 큰 것을 볼 수 있다. 이는 학습된 베이지안 신경망이 학습 시 보지 못한 랜덤한 데이터에 대해 매번 다른 예측을 하는 경향이 크며, 이는 곧 강건한 예측을 수행할 수 있다는 것을 의미한다.

```
MC_predict(best_model, 10, anomaly_sample)
```

```
(array([252.04922], dtype=float32), array([9.996798], dtype=float32))
```

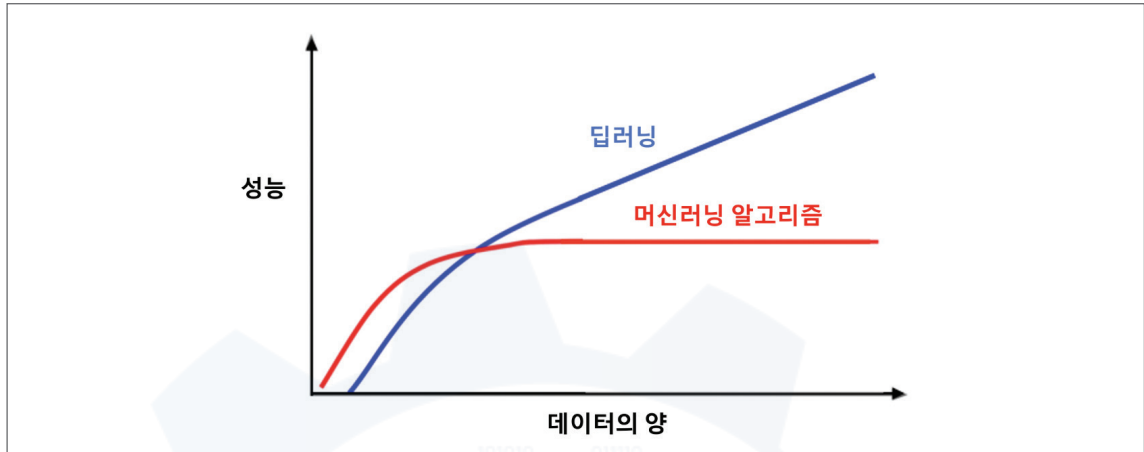
[코드 56] 랜덤 샘플 데이터에 한대 예측분포의 평균 및 표준편차 확인

[단계 ⑨] 분석결과에 대한 논의 및 해석

⑨-1. 분석 결과 해석 가이드

- 본 분석에서 구축한 베이지안 신경망은 매 예측을 수행할 때마다 서로 다른 값을 출력한다. 이는 확률적 모델링 및 베이지안 방법론의 특성이다. 따라서, 예측에 대한 성능을 평가할 때도 일회적인 예측이 아니라 몬테카를로 방식으로 여러 차례 예측한 값을 활용하여 예측 분포를 계산하는 것이 요구된다.
- 예측분포의 평균은 기존 인공지능 기반 분석모델의 예측과 동일하게 이해하고 활용하면 된다. 추가로 베이지안 신경망 모델의 예측으로부터 얻을 수 있는 표준편차, 즉 예측값의 일관성에 대한 정보는 불확실성이 고려된 예측 신뢰도 혹은 예측 확신도라고 이해할 수 있다. 이는 추후 분석모델을 현장 적용시 단순히 예측값뿐만 아니라 예측의 신뢰도를 고려하여 보다 신중한 판단을 할 수 있게 한다.
- 일반적으로 데이터 분석 및 AI 분석모델 구축 시, 수집된 데이터가 많을수록 높은 성

능의 예측 모델을 개발할 수 있는 것으로 알려져 있다. 또한, 학습 시 다양한 패턴의 데이터(정상 범주 안의)를 사용할 때 보다 바람직한 분석모델 구축이 가능하다. 이를 고려하여 분석모델을 현장 적용시, 점진적으로 데이터가 축적될수록 분석모델의 성능이 제고되며 동시에 예측에 대한 범위와 시야가 넓어질 것이라 기대된다.



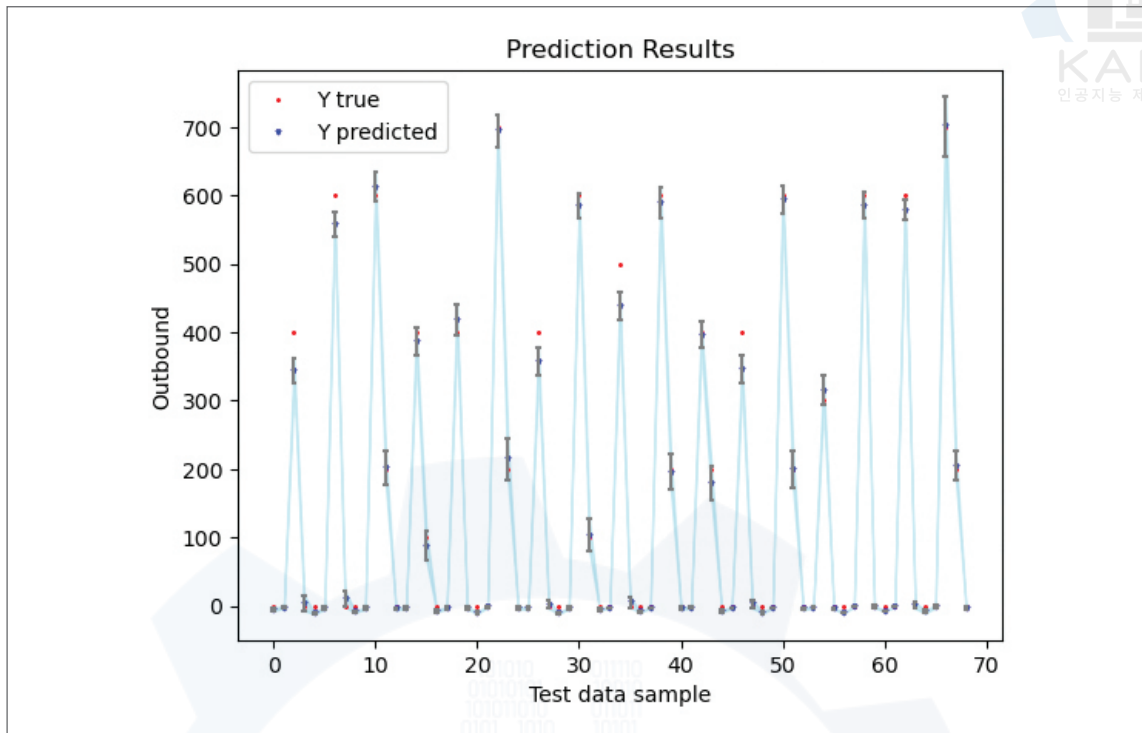
[그림 29] 데이터양에 따른 모델의 성능

⑨-2. 분석 결과 활용 가이드

- 실제 제조 현장에서 본 분석모델을 활용할 때는 예측에 대한 확신도를 적극적으로 활용할 필요가 있다. 마지막 실습으로 예측분포의 평균과 실제값, 그리고 예측분포의 표준편차를 활용하여 95% 신뢰구간을 시각화하였다. 정규분포의 특성에 따라 평균에서 부터 $2 \times \text{표준편차}(\sigma)$ 범위의 신뢰구간을 시각화하였다.

```
plt.figure()
plt.plot(range(len(X_test_42)), y_test_42, 'or', label='Y true', markersize=1)
plt.plot(range(len(X_test_42)), y_pred_mean, 'bv', label='Y predicted', markersize=2)
plt.fill_between(range(len(X_test_42)), y_pred_mean+2*y_pred_std, y_pred_mean-2*y_pred_std,
                 color='lightblue', alpha=0.5,
                 )
for i in range(len(X_test_42)):
    mean = y_pred_mean[i]
    std = y_pred_std[i]
    left = i-0.25/2; right = i+0.25/2;
    top = mean+2*std; bottom = mean-2*std
    plt.plot([i, i], [top, bottom], color='grey')
    plt.plot([left, right], [top, top], color='grey')
    plt.plot([left, right], [bottom, bottom], color='grey')

plt.title('Prediction Results')
plt.xlabel('Test data sample')
plt.ylabel('Outbound')
plt.legend()
plt.show()
```



[코드 57] 예측 결과 및 신뢰구간 시각화

- 본 분석에서 구축한 AI 분석모델 활용시 과거 시점의 수주, 생산, 재고 등의 정보를 사용하여 미래 시점의 실제 출고량을 정확하게 예측할 수 있음을 확인하였다. 또한, 예측분포 계산을 통해 예측 확신도를 얻을 수 있으므로, 실제 분석모델 활용시에도 불확실성을 고려하여 실제 출고량 예측을 수행함과 동시에 미리 이상 상황에 대한 생산 및 출고 대비를 할 수 있을 것으로 기대된다.

3. 유사 타 현장의 「열처리 공급망최적화 AI 데이터셋」 분석 적용

3.1 본 분석이 적용 가능한 제조현장 소개

- 본 실습에서는 열처리 공정 데이터를 활용하여 최적의 생산 계획 수립을 통한 공급망 최적화를 위해 AI 기반의 분석 모델 구축 및 실습을 진행하였다. 특히, 공급망의 변화에 대한 정보를 포함하는 수주량, 생산 필요량, 재고, 실제 생산량 등의 데이터를 기반으로 인공지능 경망을 사용한 실제 출고량 예측 모델을 구축하였다. 이러한 회귀 기반의 예측 방법은 공급망 관련 데이터를 활용한 다른 분석을 수행하는 경우에도 응용될 수 있다. 예를 들어, 실제 출고량 예측뿐만 아니라 수주량이나 생산 필요량 등 공급망을 구성하는 다른 요소들의 예측에도 활용 가능하다. 또한, 단일 공급망뿐만 아니라, 공정과 관련된 다양한 외재적인 변수를 고려하여 예측을 수행할 경우 보다 수준 높은 분석을 수행할 수 있다. 따라서, 열처리 공정 이외에 공급망을 활용하거나 이와 긴밀히 연결된 다양한 산업 및 제조현장에서도 본 분석이 적용될 수 있다.

- 본 분석에서 구축한 베이지안 신경망 모델은 예측 불확실성을 고려한 알고리즘으로서, 공급망 관련 데이터의 변동성을 고려할 수 있을뿐만 아니라, 예측의 신뢰도를 확인할 수 있다. 따라서, 베이지안 신경망 알고리즘을 활용한 예측 결과를 기반으로 공급망 내 특이사항이 발생할 경우에도 보다 유연하게 공급망 관리 및 계획 수립을 수행할 수 있을 것으로 기대된다.
- 기존 현장 작업자의 경험에 의존한 예측이 아닌 AI 분석모델을 이용한 예측을 통해 품목별 원활한 생산계획 수립 및 재고 관리 등이 가능하다. 특히, 다변량 데이터가 수집되는 경우 이를 종합적으로 고려하여 정량적인 분석을 수행하는 데에 AI 분석모델이 우수한 성능을 보일 수 있다. 분석모델 구축 이후, 추가적으로 공급망으로부터의 데이터 수집 및 저장을 통해 데이터 베이스를 구축함으로써 실제 공급망의 특성을 반영하는 정보 자원을 축적할 수 있다. 이를 활용하여 AI 분석모델을 개선한다면 공급망 관리의 자동화 및 시장 변화에 대한 빠른 대응이 가능할 것으로 기대된다.

3.2 본 분석을 원용하여 타 제조현장 적용시, 주요 고려사항

- 공급망이 존재하거나 공급망을 활용하는 제조현장 어디에서나 본실습에서 구축한 AI 분석모델을 응용할 수 있다. 특히, 공급망에 기록되는 정보가 다양한 현장일수록 AI 분석이 용이할 수 있다. 실제 분석을 위해서는 높은 품질의 데이터 확보가 필수적이므로, 공급망에 기록되는 데이터의 품질 유지 및 지속적인 데이터 저장이 필요하다.
- 타 제조현장에서 본 분석에서 활용한 AI 분석모델을 이용할 때, 다른 종류의 독립변수와 종속변수에 대해 동일한 예측 문제로 환원이 가능한지에 대한 판단이 선행되어야 한다. 또한, 회귀 분석의 경우 독립변수와 종속변수의 실제 상관성이 존재하는 문제에서 AI 분석모델을 사용 시, 변수 간의 내재된 특성을 학습함으로써 정확한 예측을 수행할 수 있다. 특히, 도메인 지식을 활용하여 열처리 공급망 데이터를 활용한 예측과 비슷하게 풀 수 있는 문제라면, 본 분석모델을 응용할 수 있을 것으로 사료된다.
- 본 실습에서는 열처리 공정을 통해 생산되는 일부 제품에 대한 데이터만 수집되고 있는 경우에 공급망을 최적화하기 위한 AI 모델을 구축하였으나, 한 공정을 통해 생산되는 모든 제품에 대한 데이터가 수집 및 이용 가능하다면 품목 간의 상관관계까지 고려하는 AI 모델 구축이 가능할 것이다.
- 이와 같이 잘 작동하는 AI 분석모델을 원활히 구축하기 위해서는 올바른 형태의 데이터 수집이 필수적이므로 데이터가 수집되는 현장 관리자와 데이터 분석 및 모델 구축 담당자 사이의 충분한 소통이 중요하다.

3 부록_분석환경 구축을 위한 설치 가이드

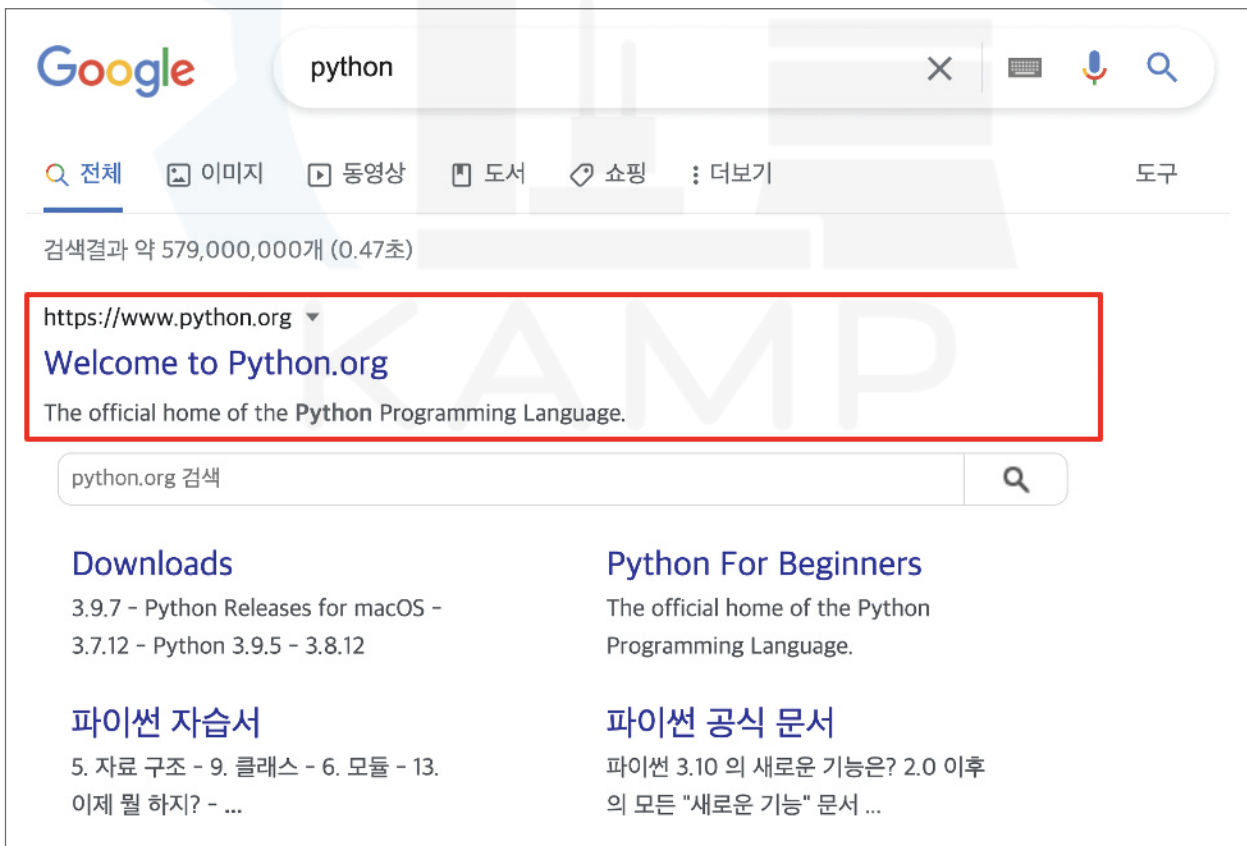
1. 파이썬(Python) 설치

파이썬은 컴퓨터 언어로서 최근 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다. 다운로드 및 설치가 간편하고 활용도가 높은 파이썬을 로컬 컴퓨터에 설치하고 적용하는 방법을 안내한다.

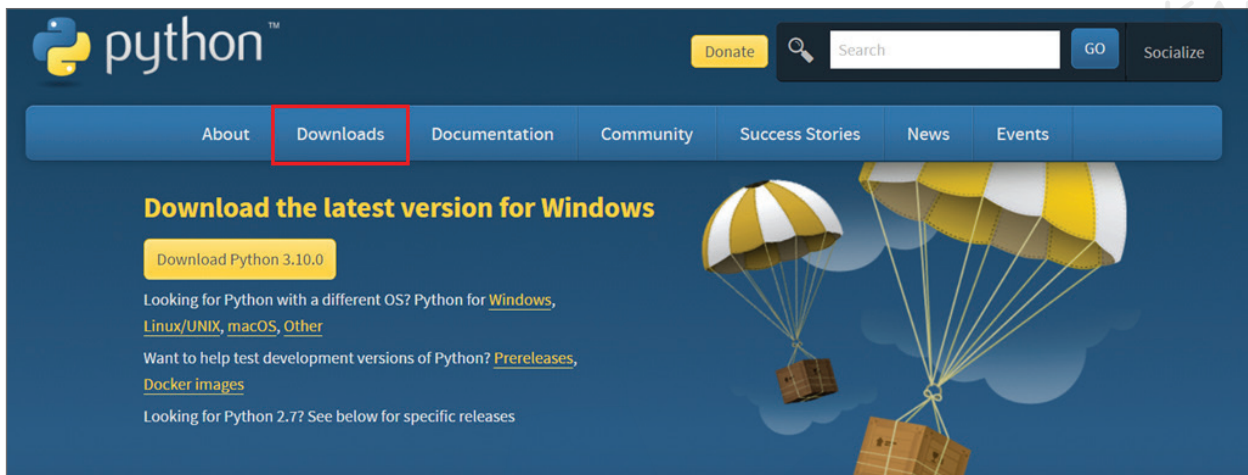
① google.com 등의 검색 엔진에 'python'을 검색한다.



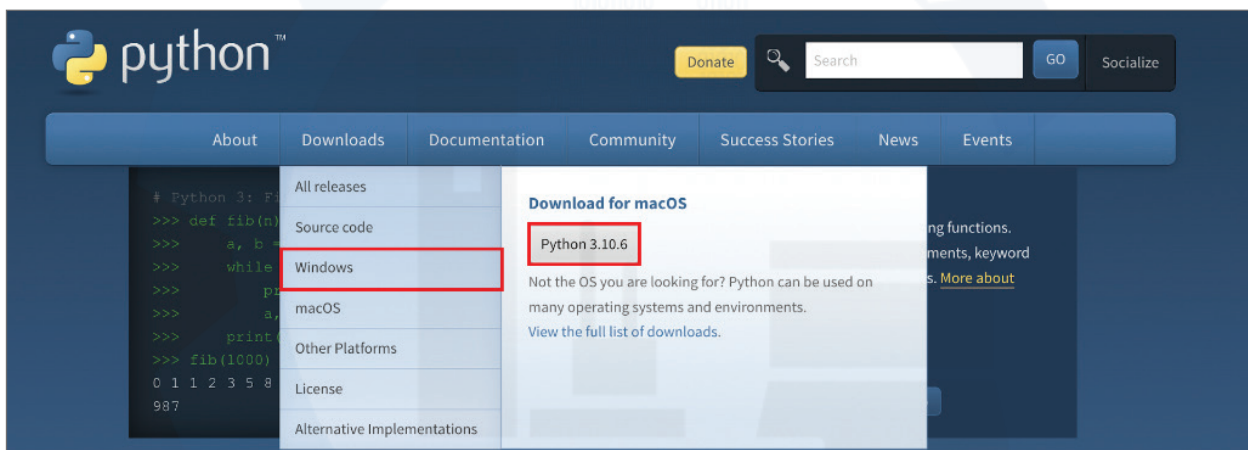
② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



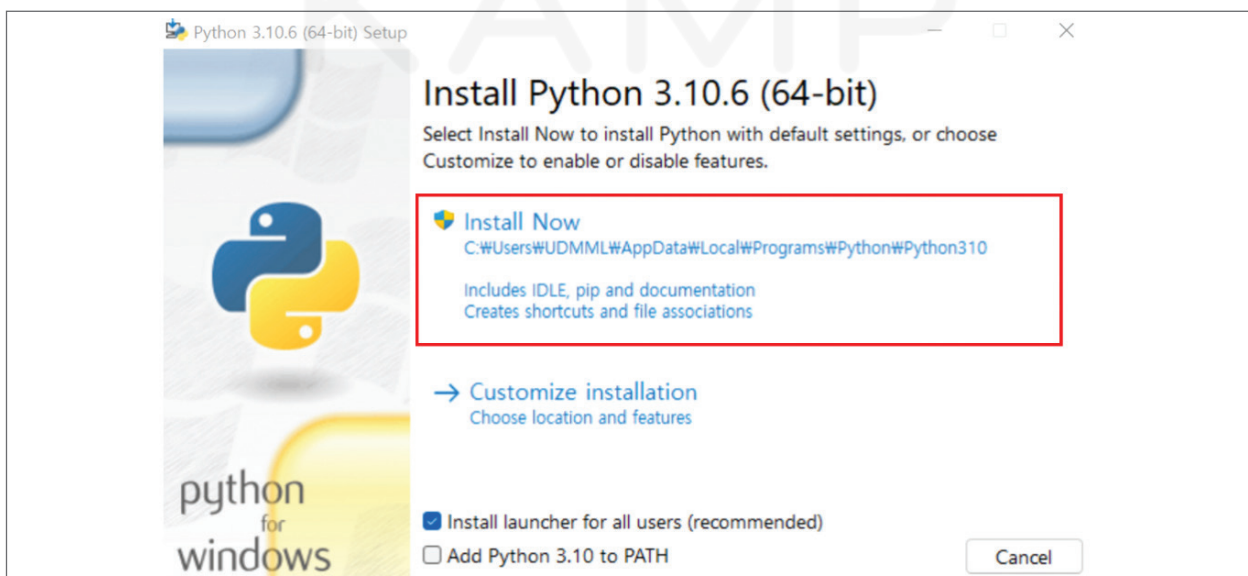
③ 클릭후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



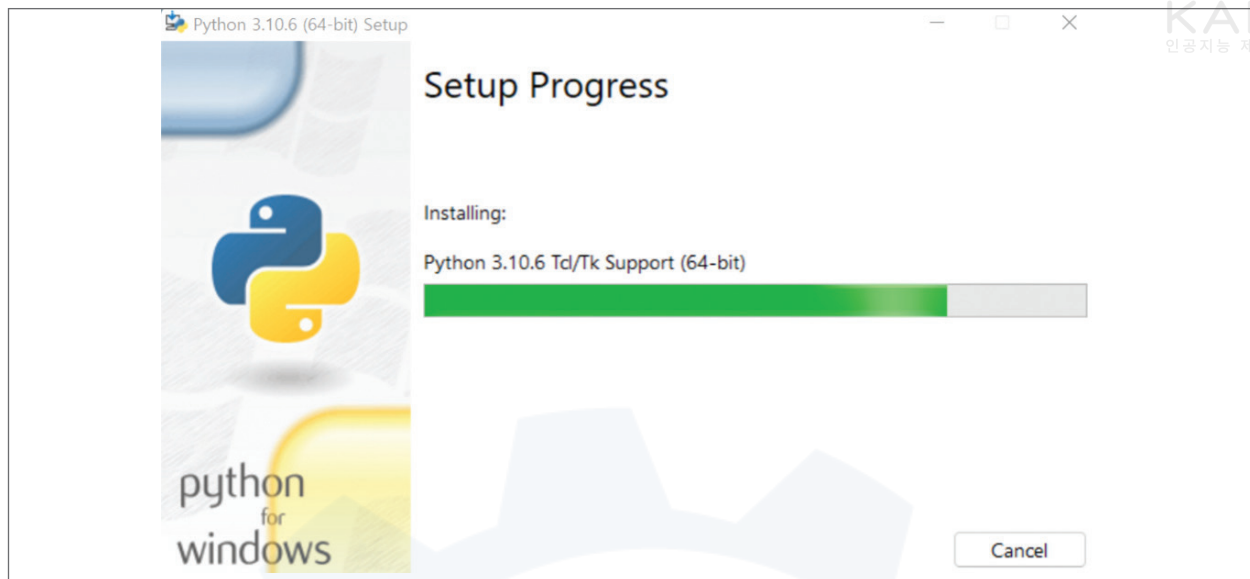
④ 컴퓨터의 운영체제에 따라 상단에서 세번째(macOS의 경우 네번째) 탭을 선택한 후, Python 3.10.6을 다운로드한다. (Python 3.10.6 버전은 업데이트 될 수 있다)



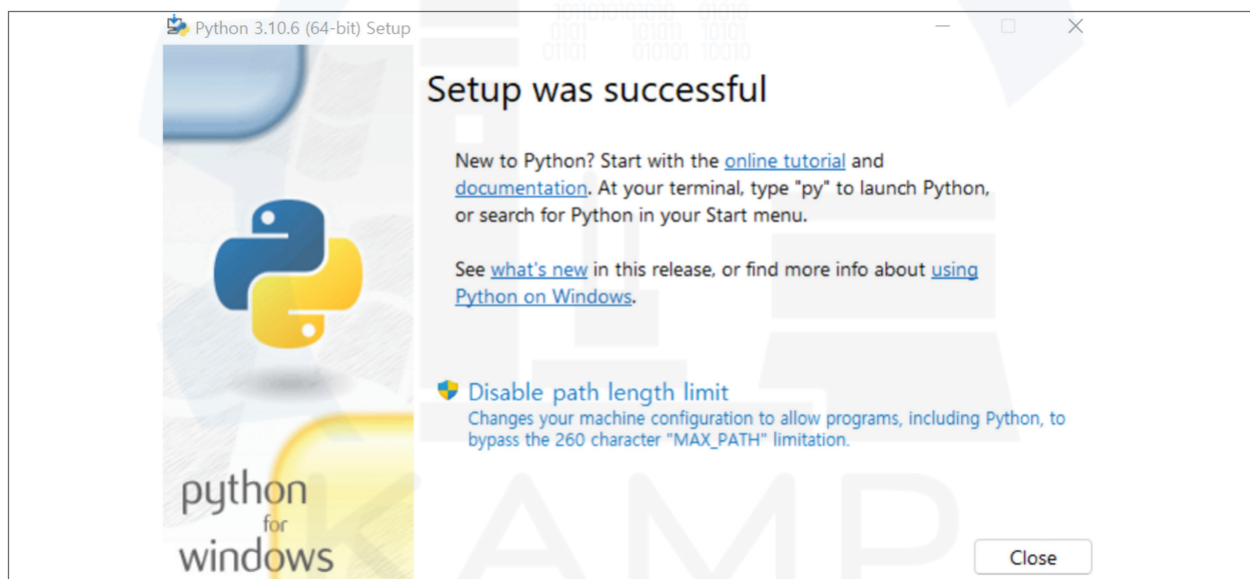
⑤ 아래와 같은 설치창이 뜨면, 'Install Now'를 클릭한다.



⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



⑦ 완료가 되면 아래와 같은 창이 뜨는 것을 확인 후 종료한다. [설치완료]

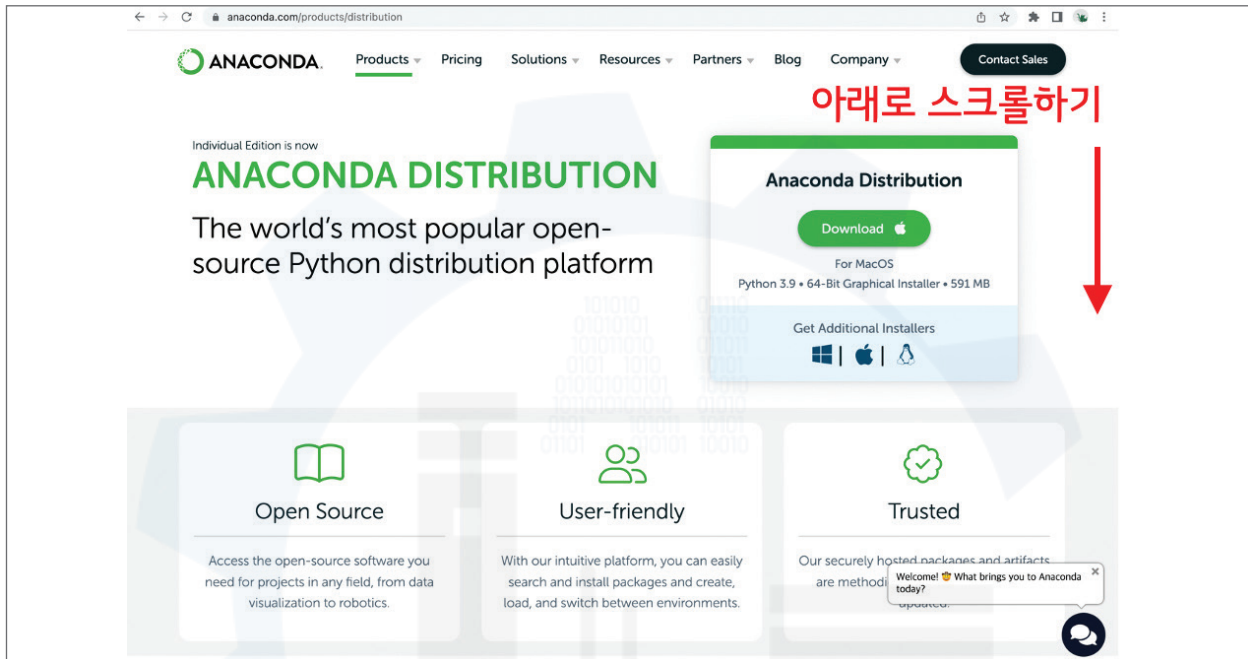




2. 아나콘다(Anaconda) 설치

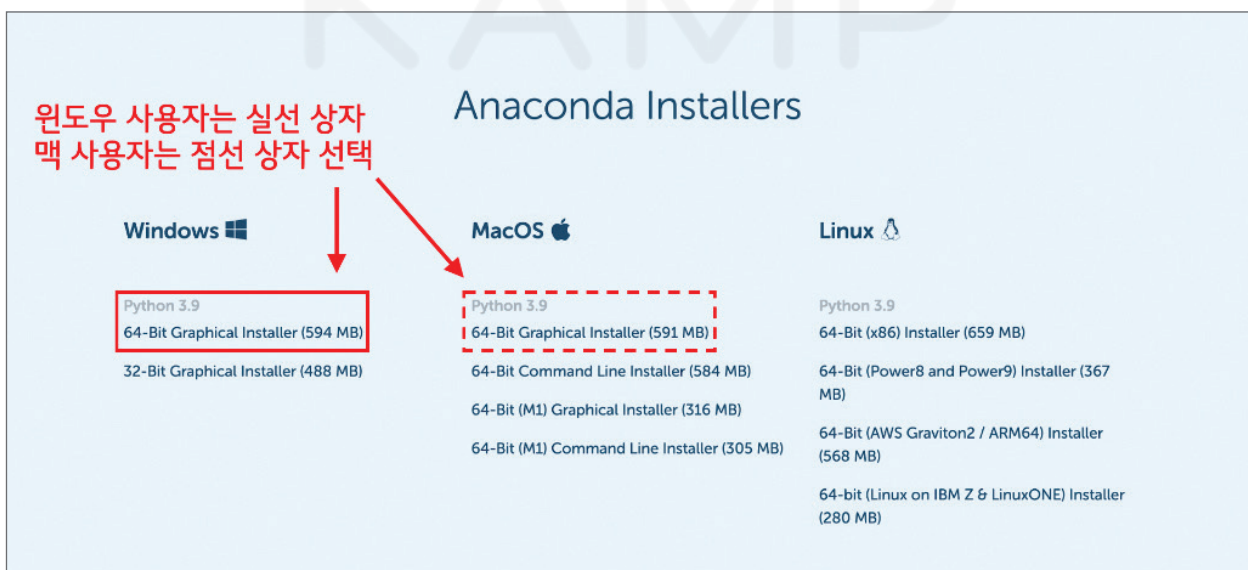
아나콘다란 파이썬과 같은 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다. 아나콘다를 설치함으로써 다양한 기능들을 바로 사용할 수 있고, 결과물을 쉽게 확인할 수 있다. 아나콘다를 설치하고 분석을 실시할 수 있는 환경을 구축하는 방법에 대해 안내한다.

① <https://www.anaconda.com/distribution/> 로 접속한다.



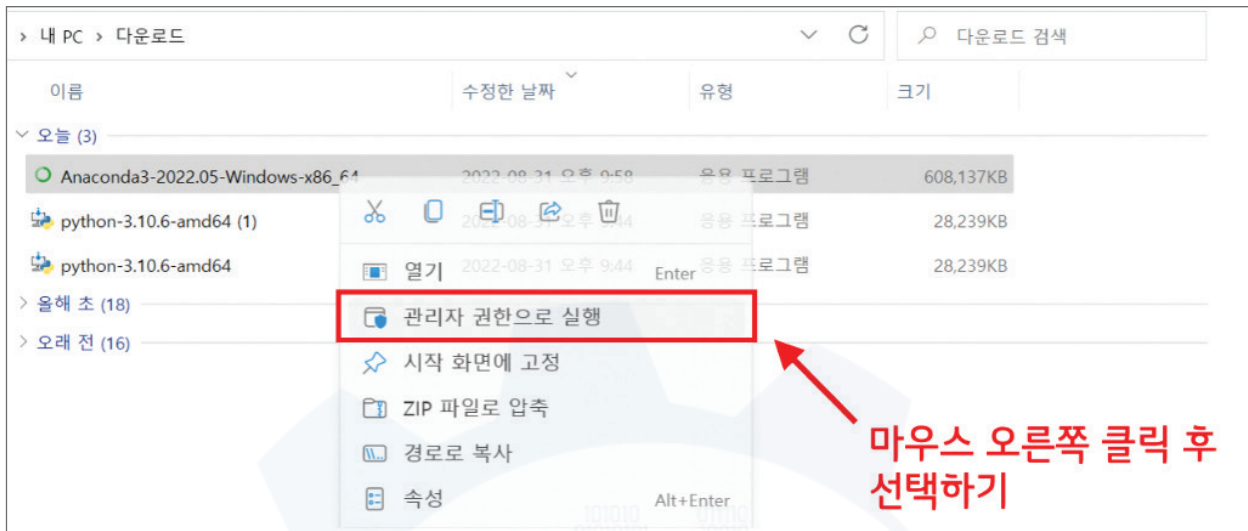
② 다음과 같은 화면이 나올 때 까지 스크롤하여 아래로 내린 후, 로컬 컴퓨터 사용 환경에 맞는 파일을 다운받는다.

- ▶ Windows: 64-Bit Graphical Installer
- ▶ MacOS: 64-Bit Graphical Installer

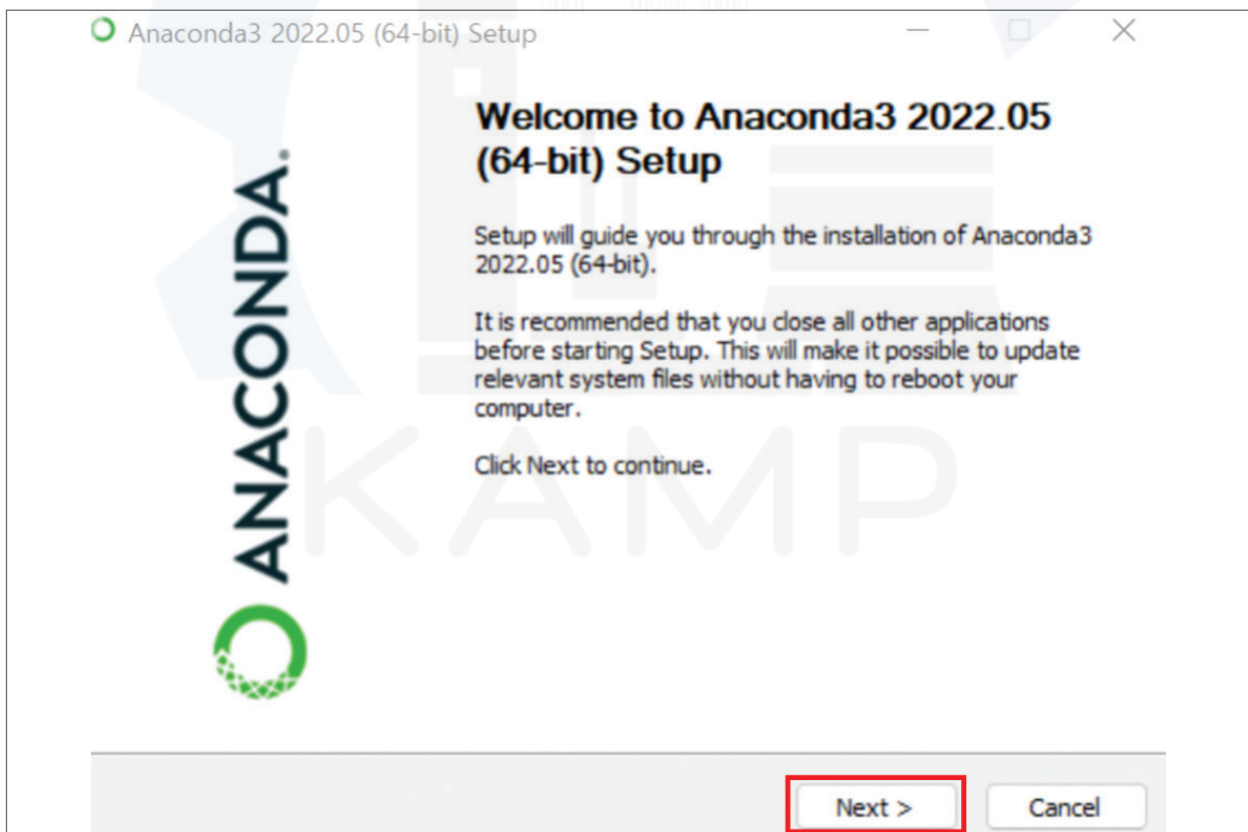


③ 다운로드 받은 파일로 이동하여, 설치된 아나콘다 파일 위에서 마우스 오른쪽을 클릭한 후, 방패모양의 ‘관리자 권한으로 실행’을 클릭한다.

▶ (예) ‘다운로드’ 폴더로 아나콘다를 다운 받은 경우

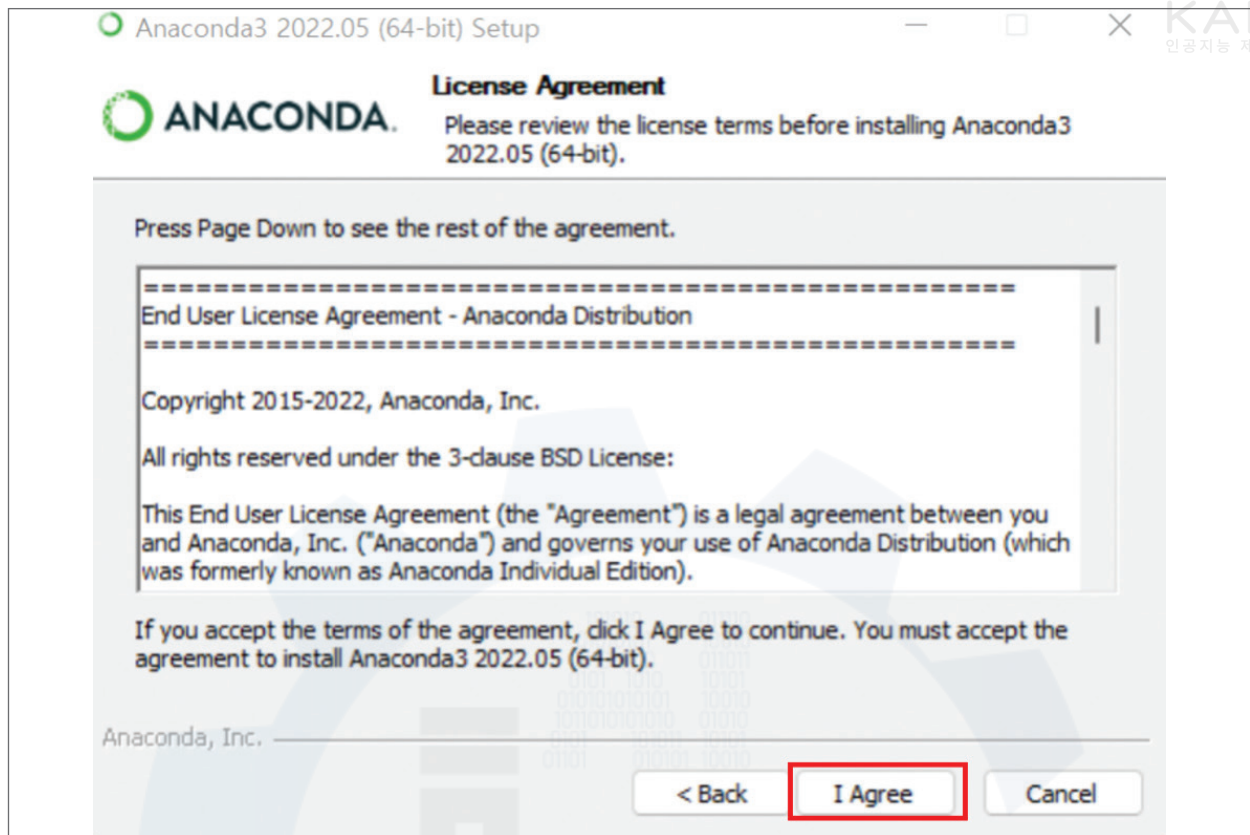


④ 파일을 실행 한 후, ‘Next’ 버튼을 클릭한다.

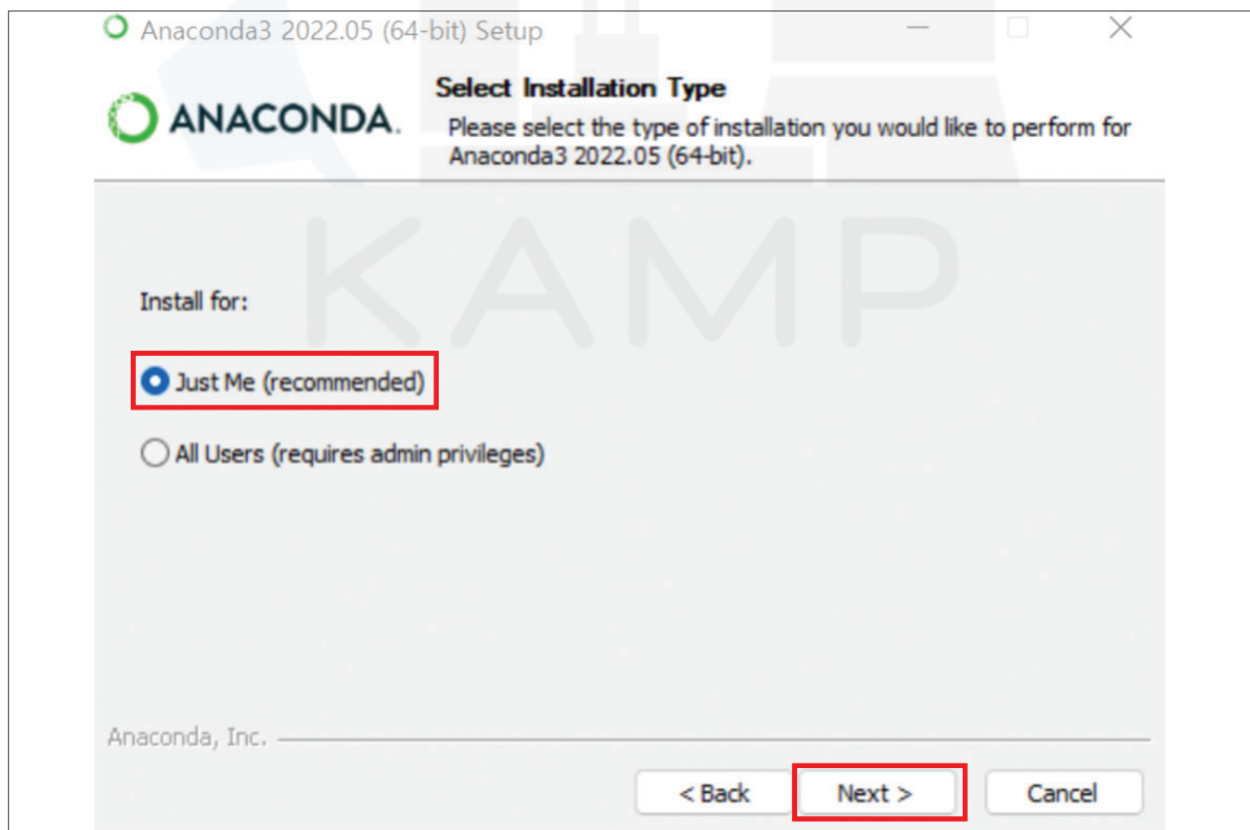




⑤ 다음 창이 나타나면 'I Agree'를 선택한다.

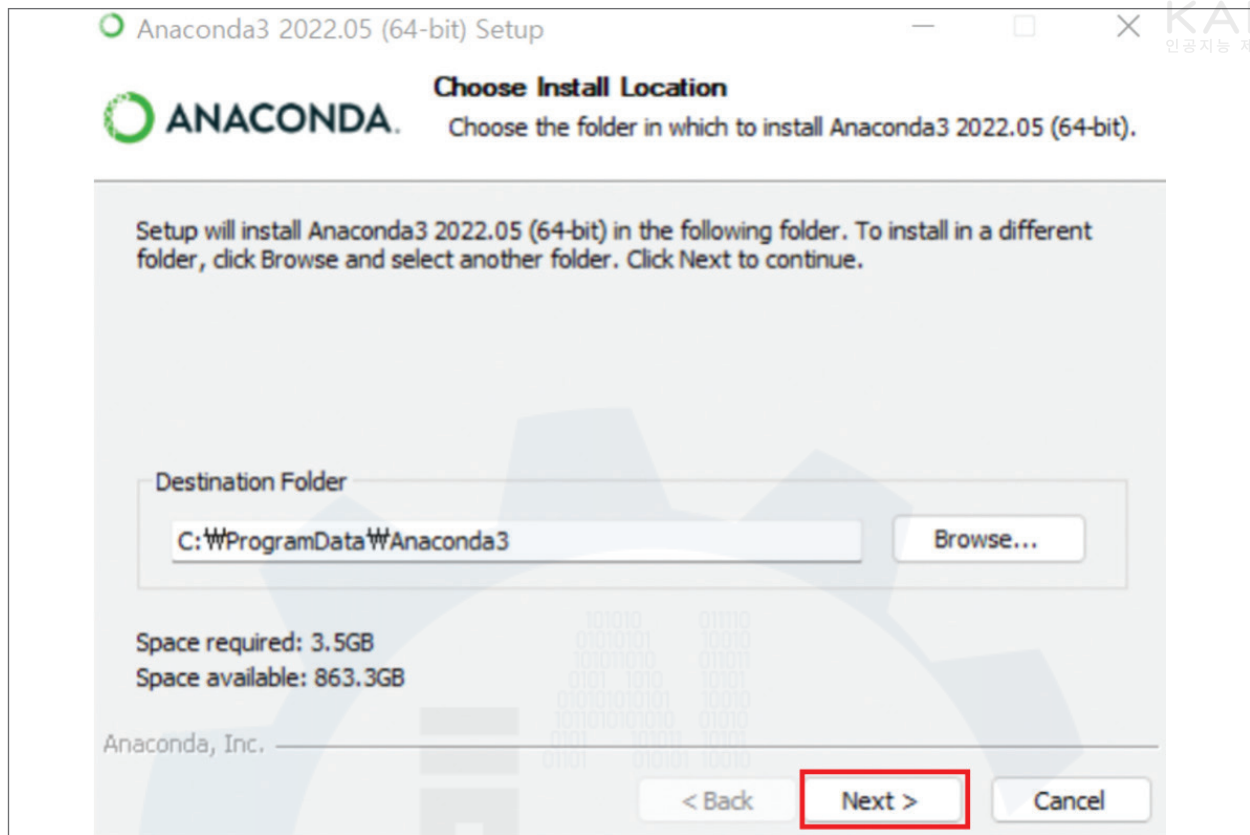


⑥ 셋팅 창이 뜨면 화면의 'Just Me'를 선택 후 아래의 'Next'를 클릭한다.

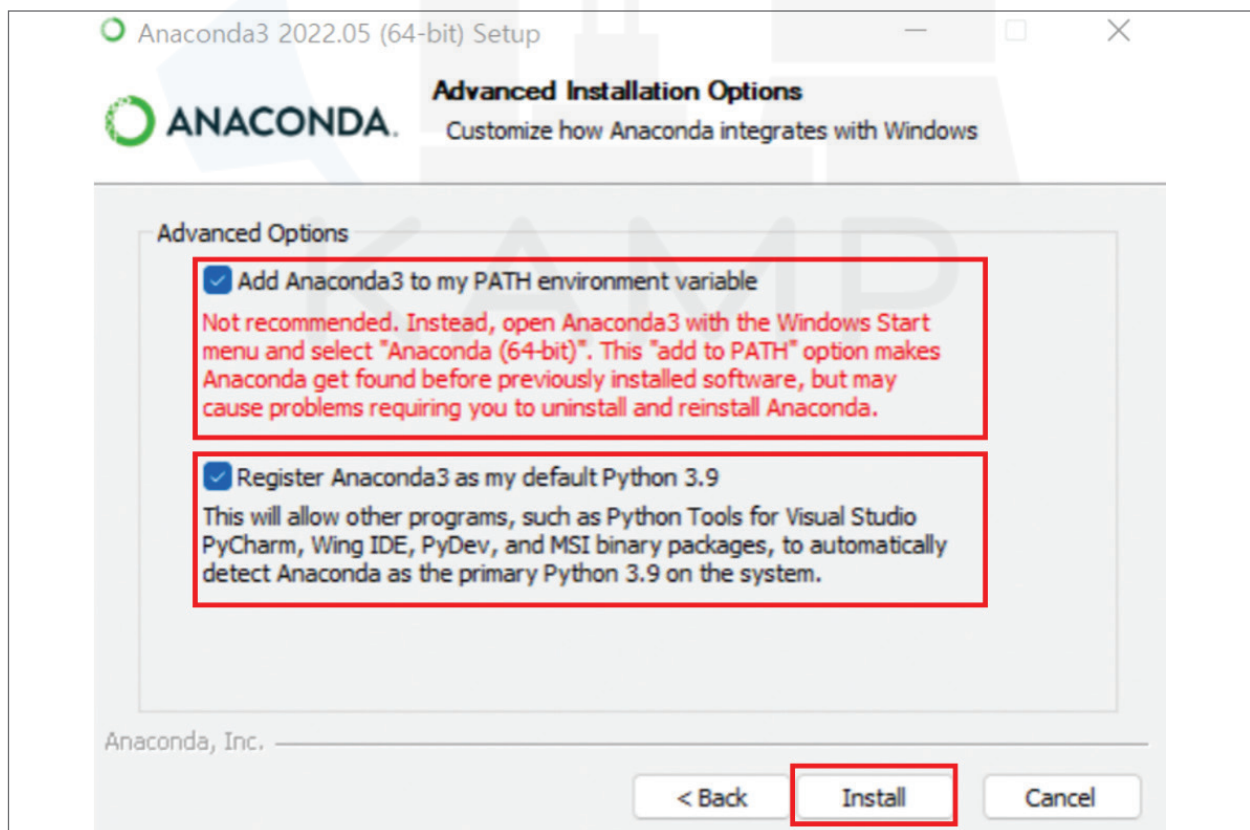




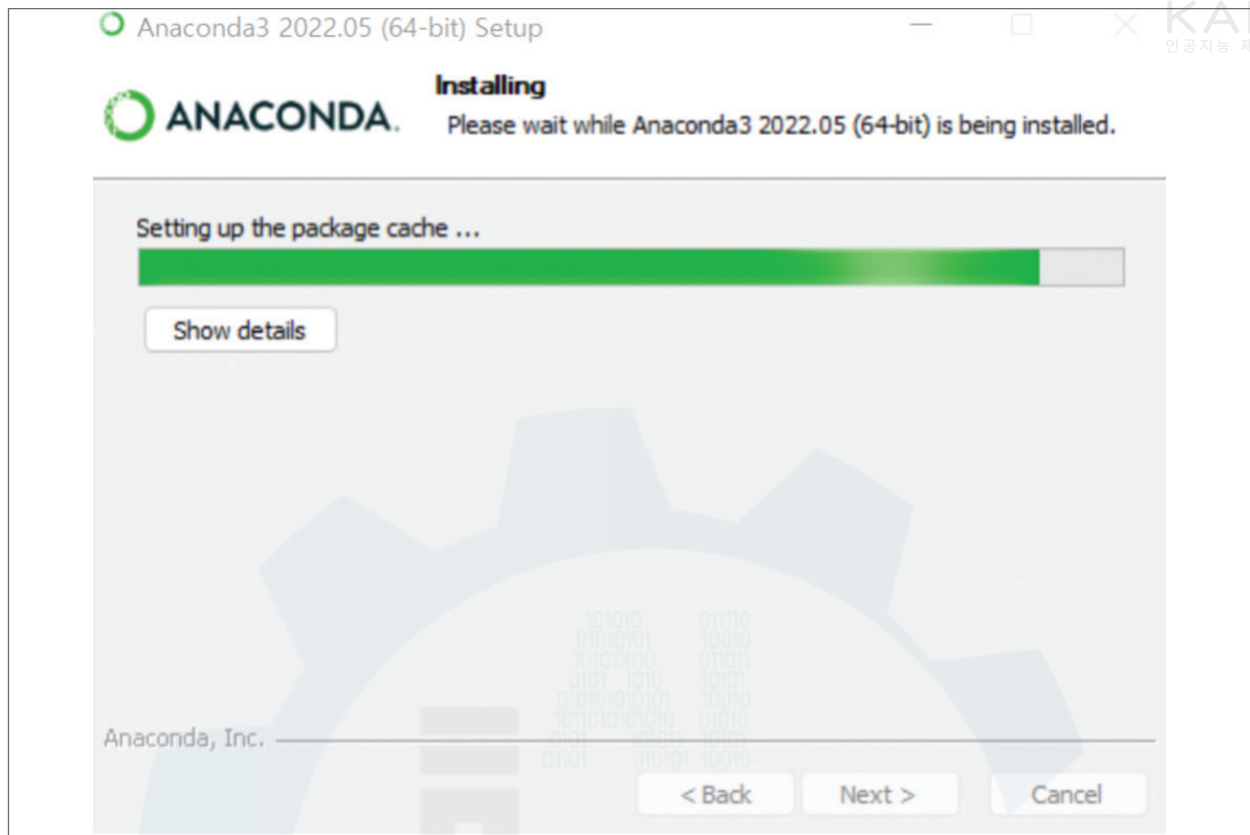
⑦ 다운로드 받을 경로를 물어보는 창에 설치할 경로를 지정하고, 아래의 'Next'를 클릭한다.



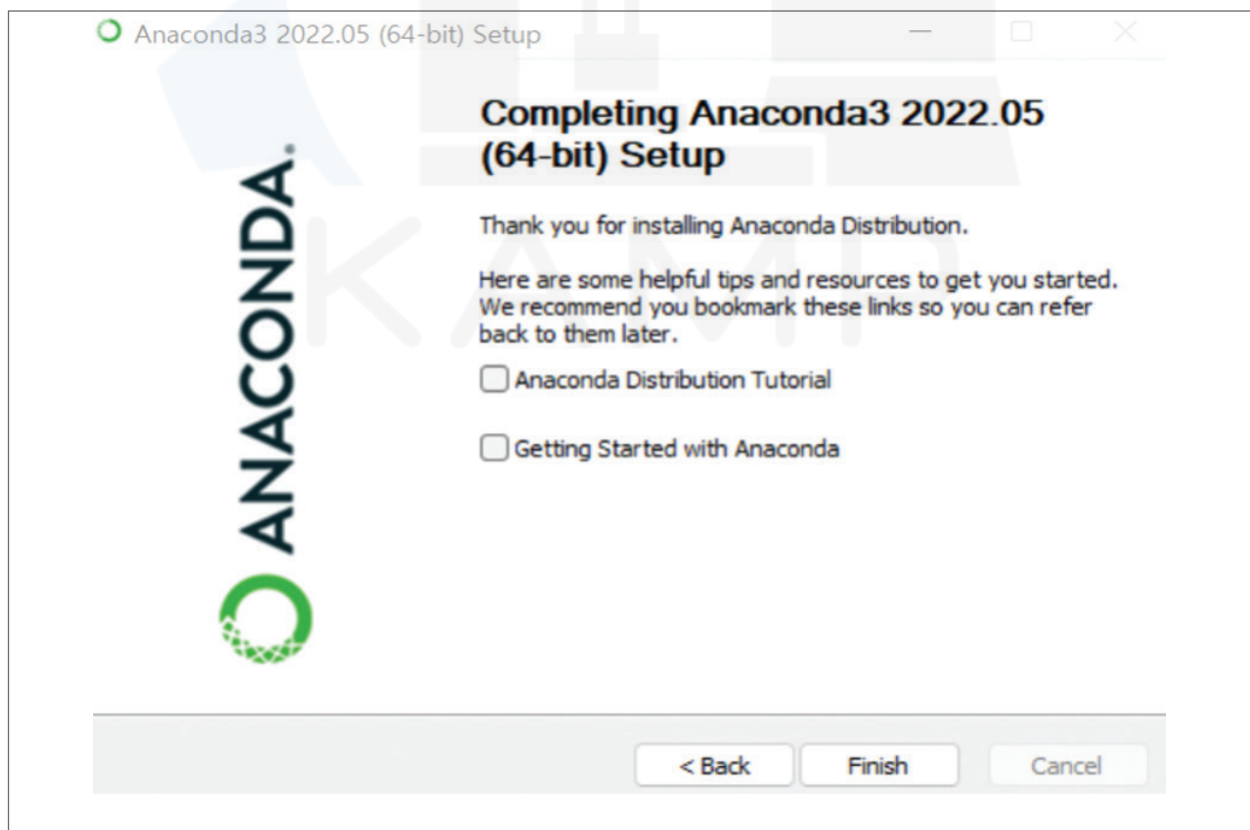
⑧ 고급 옵션 선택창이 뜨면, 아래와 같이 모두 선택 후, 'Install'을 클릭한다.



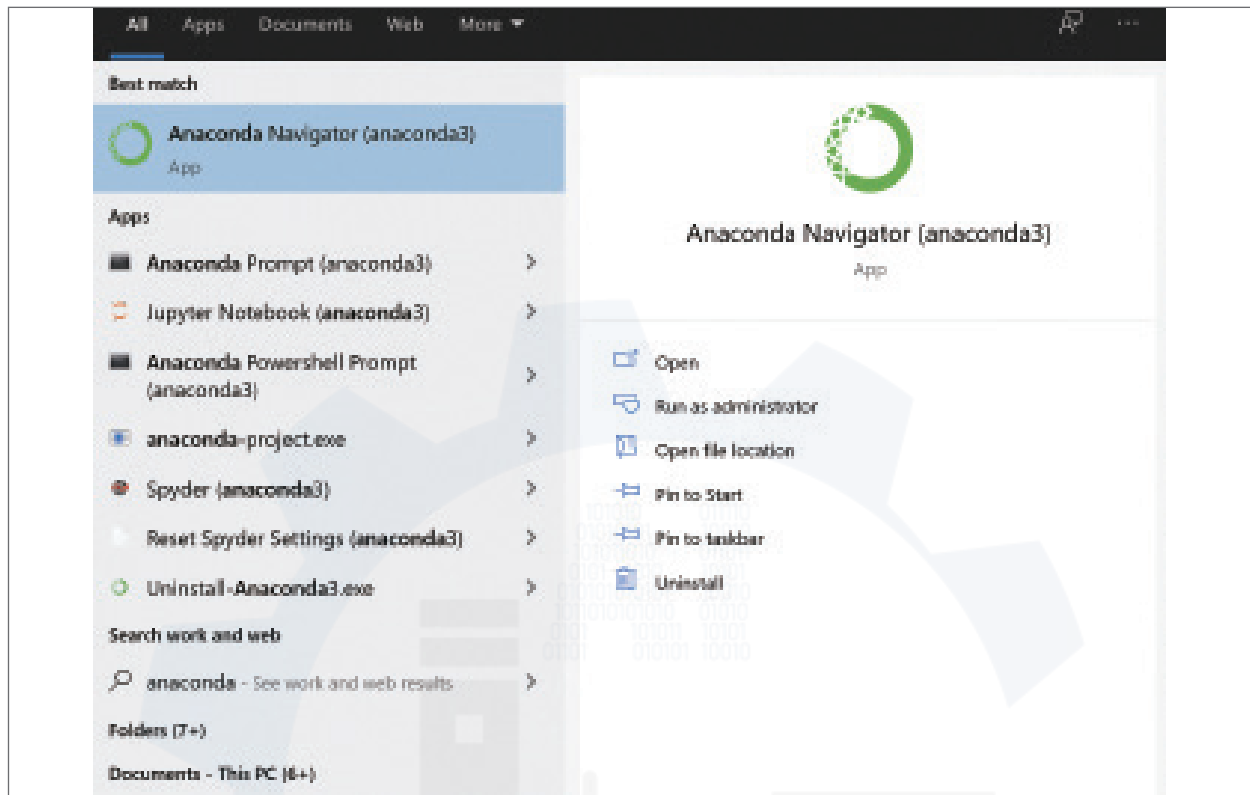
⑨ 다음과 같은 설치창이 뜨면 완료가 될 때까지 대기 (5분이상 소요)



⑩ 마지막 화면에서, 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



- ⑪ 화면상의 '홈(🏠)'키를 눌러서 화면과 같이 anaconda prompt가 잘 설치 되었는지를 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램들도 함께 확인이 된다면 설치가 완료된 것이다.

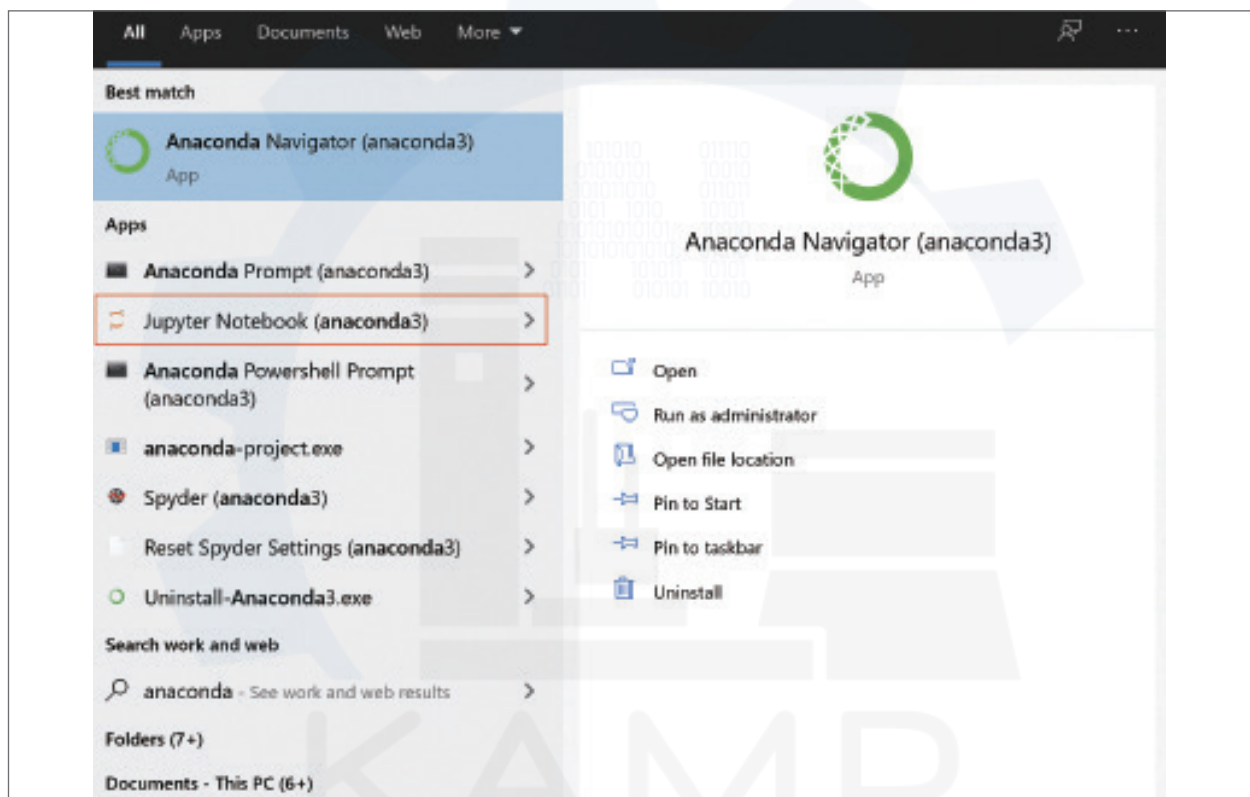




3. 주피터 노트북 (Jupyter Notebook) 실행

주피터 노트북은 실제로 사용자가 코딩(분석 문장 작성)을 할 수 있는 도구이다. 쉽게 비교하자면, 문서 도구로 마이크로소프트 사의 ‘Word’ 프로그램이나, 한컴소프트 사의 ‘한글’ 프로그램 등과 같은 도구라고 생각할 수 있다. 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드북에서는 주피터 노트북을 활용하는 방법을 안내하기로 한다. Anaconda를 설치한 이후 주피터 노트북(Jupyter Notebook) 설치 방법을 확인하면 된다.

- ① 화면상의 ‘홈(Windows)’키를 눌러서 화면과 같이 ‘anaconda’ 검색 및 폴더 리스트 중 ‘Jupyter Notebook (anaconda3)’을 실행한다.





② Jupyter Notebook을 클릭시 아래처럼 2개의 윈도우가 실행된다.

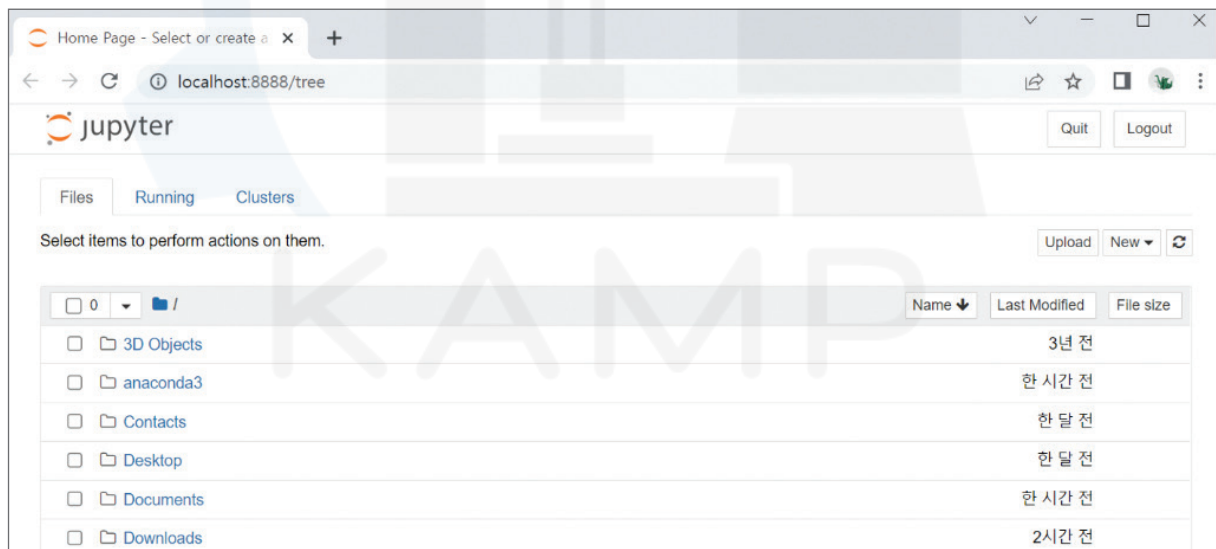
(1) 검은색 배경의 화면은 주피터 노트북이 실행되는 환경에 대한 상태를 나타내는 상태 표시 창이다. 주피터 노트북을 사용하는 동안 종료하면 안된다.

```

Jupyter Notebook (anaconda3)
[W 2022-08-31 23:28:02.700 LabApp] 'notebook_dir' has moved from NotebookApp to ServerApp. This config will be passed to
ServerApp. Be sure to update your config before our next release.
[W 2022-08-31 23:28:02.700 LabApp] 'notebook_dir' has moved from NotebookApp to ServerApp. This config will be passed to
ServerApp. Be sure to update your config before our next release.
[I 2022-08-31 23:28:02.710 LabApp] JupyterLab extension loaded from C:\Users\UDMML\anaconda3\lib\site-packages\jupyterlab
[I 2022-08-31 23:28:02.710 LabApp] JupyterLab application directory is C:\Users\UDMML\anaconda3\share\jupyterlab
[I 2022-08-31 23:28:02.710 NotebookApp] Serving notebooks from local directory: C:\Users\UDMML
[I 2022-08-31 23:28:02.710 NotebookApp] Jupyter Notebook 6.4.8 is running at:
[I 2022-08-31 23:28:02.710 NotebookApp] http://localhost:8888/?token=48c81fb7aa6c69796126efbd6ed99252501720b32361cae0
[I 2022-08-31 23:28:02.710 NotebookApp] or http://127.0.0.1:8888/?token=48c81fb7aa6c69796126efbd6ed99252501720b32361cae0
[I 2022-08-31 23:28:02.710 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 23:28:02.740 NotebookApp]

To access the notebook, open this file in a browser:
file:///C:/Users/UDMML/AppData/Roaming/jupyter/runtime/nbserver-22196-open.html
Or copy and paste one of these URLs:
http://localhost:8888/?token=48c81fb7aa6c69796126efbd6ed99252501720b32361cae0
or http://127.0.0.1:8888/?token=48c81fb7aa6c69796126efbd6ed99252501720b32361cae0
  
```

(2) 사용하는 인터넷 프로그램(크롬, 인터넷 익스플로러)에 주피터 노트북이 열린다. (다른 확장 프로그램 사용하고 싶다면 기본 브라우저를 변경해 주어야 한다.)

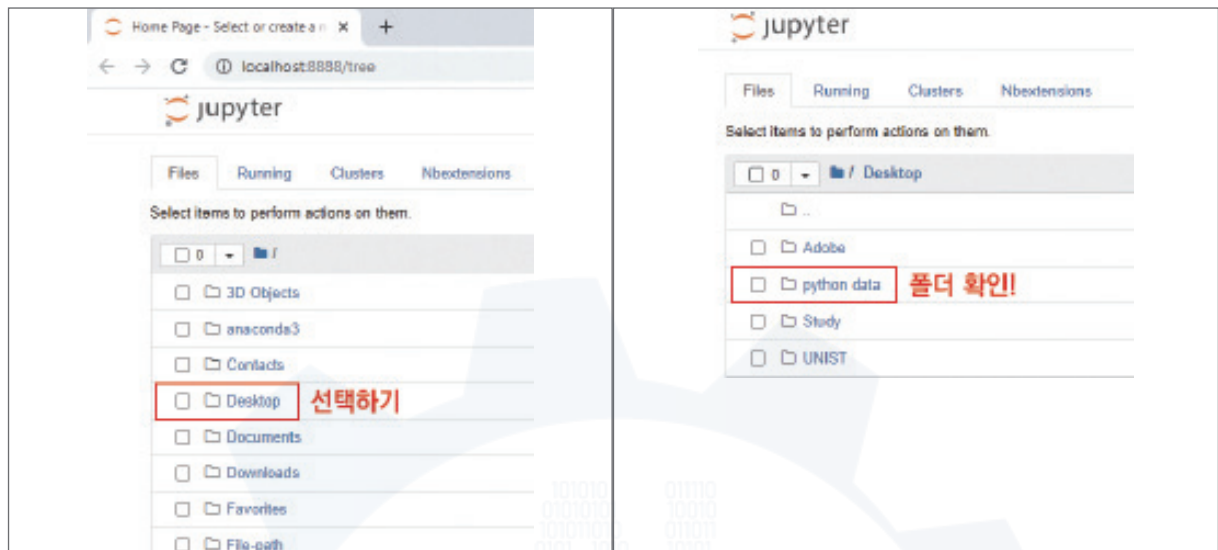




③ 데이터를 저장하고, 불러오고, 분석할 경로의 폴더를 하나 생성한다.

▶ (예) '바탕화면'에 'python data' 폴더를 생성 후 (미리 생성), 파이썬 코드 실행하고 저장한다.

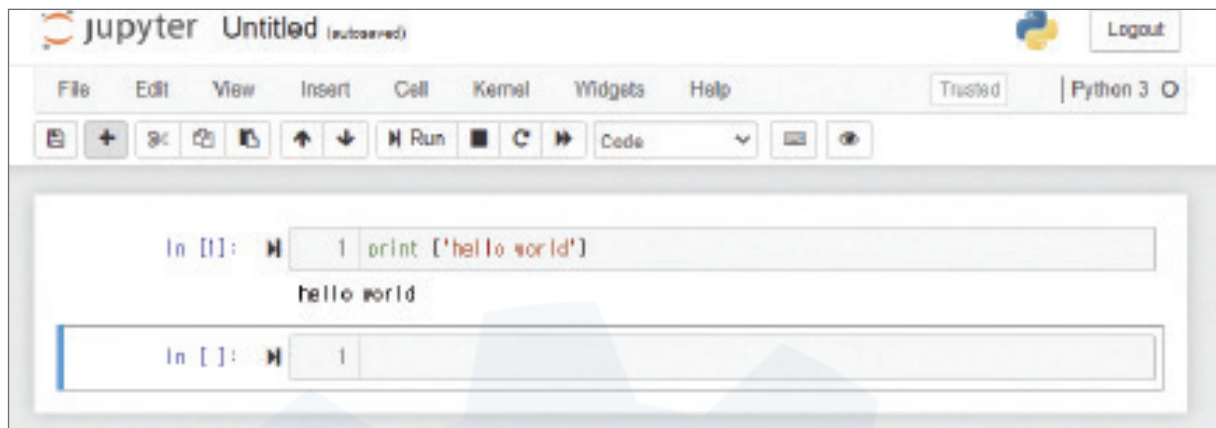
(1) 주피터 노트북에서 'python data' 폴더를 확인한다.



(2) python data에서 파이썬 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



- (3) 'hello world' 출력을 확인해본다. 보이는 In [] 우측 회색 창에 `print('hello world')` 입력 후, 상단의 **Run** 버튼 혹은 `shift + Enter` 키를 눌러서 실행한다. 코드 파일의 확장자는 '.ipynb'로 저장된다.



4 부록_데이터 품질 전처리 [실습 코드]

앞선 '2.분석 실습'에서 데이터 품질 전처리에 필요한, Gartner에서 제시한 6종의 데이터 품질 평가 지표에 대하여 논의하였다.

(완전성, 유일성, 유효성, 일관성, 정확성, 무결성)

▶ 데이터 품질 확인을 하기 위해 분석에 활용할 'data.csv' 를 df라는 이름으로 불러온다.

```
import pandas as pd
df=pd.read_csv('./data.csv')
```

[코드 1] 데이터 불러오기

▶ 완전성 품질 지수 = $(1 - (\text{결측치} / \text{전체 데이터 수})) * 100$

- ① Null 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 컬럼별 Null 값의 비율을 확인하여 삭제한다. 본 데이터는 30%를 넘는 컬럼이 존재하지 않으므로 컬럼을 제거하지 않는다.

```
perc=30
(df.isna().sum()/len(df))*100
```

시간	0.0
품목	0.0
생산필요량	0.0
재고	0.0
생산량	0.0
수주량	0.0
dtype:	float64

```
perc=30
(df.isna().sum()/len(df))*100>perc
```

시간	False
품목	False
생산필요량	False
재고	False
생산량	False
수주량	False
dtype:	bool

[코드 2] 분석 도구를 활용한 완전성 품질 지수 확인

- ② 데이터의 결측치를 확인하기 위해 'isna()' 메서드를 사용한 뒤 'sum()' 메서드를 이용하여 총 결측치 개수를 구한다.

```
df.isna().sum()
```

시간	0
품목	0
생산필요량	0
재고	0
생산량	0
수주량	0
dtype:	float64

[코드 3] column별 결측치 개수 확인



```
cmpt_len=df.isna().sum().sum()
print(cmpt_len)
```

0

[코드 4] 결측치 개수 확인

③ 구한 결측치의 개수를 이용하여 완전성 품질 지수를 구한다.

```
print("결측치 = %d개 \n완전성 지수 : %.2f%%"%(cmpt_len,(1-cmpt_len/len(df))*100))
#결과는 아래에서 확인 가능하다.
```

```
결측치 = 0개
완전성 지수 : 100.00%
```

[코드 5] 분석 도구를 활용한 완전성 품질 지수 구하기

▶ 유일성 품질 지수 = ((유일한 데이터 수)/전체 데이터 수) * 100

: 본 데이터는 '품목'별 '시간' 열이 유일한 값을 가지는 기본키이다.

- ① 'groupby()' 메서드를 이용하여 '품목'별 '시간' 열이 가지는 유일한 값들을 확인하고, 유일한 데이터 개수를 구한다.

```
df.groupby(['품목','시간']).size()
```

```
품목  시간
GEAR_21 2022.3.1 11:00    1
        2022.3.1 14:00    1
        2022.3.1 17:00    1
        2022.3.1 8:00     1
        2022.3.10 11:00   1
        ..
GEAR_42 2022.6.8 8:00     1
        2022.6.9 12:00    1
        2022.6.9 16:00    1
        2022.6.9 20:00    1
        2022.6.9 8:00     1
Length: 688, dtype: int64
```

[코드 6] 분석 도구를 활용한 유일한 값들 확인하기

```
len(df.groupby(['품목','시간']).size())
```

688

[코드 7] 분석 도구를 활용한 유일한 값들의 개수 확인하기

② 구한 유일한 데이터 개수를 이용하여 유일성 품질지수를 구한다.

```
uniq_len=len(df.groupby(['품목','시간']).size())
print("유일성 지수 : %.2f%%"%((uniq_len/len(df))*100)) #결과는 아래에서 확인 가능하다.
```

유일성 지수 : 100.00%

[코드 8] 분석 도구를 활용한 유일성 품질지수 구하기



▶ 유효성 품질 지수 = (유효성 만족 데이터 수/전체 데이터 수)*100

① 데이터가 유효 범위 내에 있는가?

: 데이터셋에 정의된 수집 범위를 이용하여 조건(유효 범위)을 모두 충족하는 데이터들을 valid_df에 저장한다. 본 데이터는 전체 부품 2개의 발주 주문 로그가 저장되어 있는 데이터다. 'nunique()' 메서드를 사용하여 고유한 '품목'의 개수가 2개인지 확인을 한다.

```
print(df['품목'].nunique())
```

```
2
```

[코드 9] 분석 도구를 활용하여 유효 범위 내에 있는지 확인하기

: 모든 데이터가 유효 범위 내에 있는 것을 알 수 있다.

```
valid_df=df
```

[코드 10] 분석 도구를 활용하여 유효 범위내에 있는 데이터를 valid_df에 저장하기

② 데이터가 형식에 맞는가?

: '시간' 열의 타입을 datetime으로 변경할 때 errors='coerce' 옵션을 추가한다. errors='coerce' 옵션은 올바르지 않은 날짜 형식을 Null값으로 채워주는 옵션이다.

```
valid_df['시간']=pd.to_datetime(valid_df['시간'],format='%Y.%m.%d %H:%M',errors='coerce')
```

[코드 11] 분석 도구를 활용하여 데이터 형식 맞춰주기

: '시간' 이 Null인 경우를 'isnull()' 메서드를 통해 모든 데이터의 형식이 올바른다는 것을 알 수 있다.

```
valid_df['시간'].isnull().sum()
```

```
0
```

[코드 12] 분석 도구를 활용한 데이터 형식 확인하기

: '품목' 열의 데이터 형태가 'GEAR_XX' 형태여야 한다. 해당 열의 데이터에 유효하지 않은 형식이 들어있는지 '품목' 열 데이터의 앞에서 5자리까지 문자열을 추출하고 'unique()' 메서드를 이용하여 확인한다. [코드 13]의 결과를 보면 유효성을 위배하는 데이터가 없는 것을 알 수 있다.

```
valid_df['품목'].str[:5].unique()
```

```
array(['GEAR_', dtype=object)
```

[코드 13] 분석 도구를 활용한 데이터 유효성 검사



③ 수집된 날짜 안에 들어가 있는가?

: 2022년 03월 1일부터 2022년 06월 30일에 수집된 데이터이기 때문에, 데이터가 해당 기간을 벗어나지 않았는지 확인한다.

```
import datetime
d0=pd.Timestamp(datetime.date(2022,3,1)) #데이터 수집 시작 날짜
d1=pd.Timestamp(datetime.date(2022,7,1)) #데이터 수집 종료 날짜+1
con1=valid_df['시간']>=d0
con2=valid_df['시간']<=d1
valid_df=valid_df[con1&con2]
```

[코드 14] 분석 도구를 활용하여 데이터 수집 기간 확인하기

④ 따라서 3가지 경우를 모두 만족하는 데이터는 valid_df에 저장하고, 최종적으로 완성된 데이터를 이용하여 유효성 품질 지수를 구한다.

```
vald_len=len(valid_df)
print("유효성 지수 : %.2f%%"%(vald_len/len(df)*100)) #결과는 아래에서 확인 가능하다.
```

유효성 지수 : 100%

[코드 15] 분석 도구를 활용하여 유효성 품질 지수 최종 확인하기

▶ 일관성 품질 지수 = (일관성 만족 데이터 수/ 전체 데이터 수) * 100

: 본 데이터가 구조, 값, 형태를 일관성 있게 지키고 있는지 확인한다. '시간' 열의 데이터가 일관된 날짜 형태의 데이터를 가졌는지, '품목' 열의 데이터는 'GEAR_XX'의 형태를 가졌는지는 유효성을 확인하면서 확인하였다. 'dtypes()' 메서드를 이용하여 이외의 데이터 값들의 형태가 int64인지 확인한다.

df.dtypes

```
시간    datetime64[ns]
품목      object
생산필요량    int64
재고        int64
생산량       int64
수주량       int64
dtype: object
```

[코드 16] 분석 도구를 활용하여 데이터 일관성 확인하기

▷ 본 데이터는 일관성이 있다고 할 수 있다.

▶ 정확성 품질 지수 = (1-(정확성 위배 데이터 수/전체 데이터 수)) * 100

: 'GEAR_21'와 'GEAR_42' 품목은 각각 다른 3시간, 4시간 주기로 수집된다. 'GEAR_21'은 3시간 주기로, 8시, 11시, 14시, 17시에 수집되고 'GEAR_42'은 각 4시간 주기로 8시, 12시, 16시, 20시에 수집된다. 각 품목의 수집 시간이 맞는지 확인하여 규칙 정확성을 확인한다.

: 품목이 'GEAR_21' 인 데이터를 'df_21' 에 저장하고, pandas 패키지의 'to_datetime()' 메서드를 사용하여 시간정보를 변환한다. 변환한 데이터 내 '시간' 변수를 기반으로 시간 정보를 '시' 열에 새로 추가한다. 'unique()' 메서드를 사용하여 'GEAR_21' 의 수집 시간이 8시, 11시, 14시, 17시에 해당하는지 확인한다.

```
# GEAR_21
df_21 = df[df['품목']=='GEAR_21']
df_21['시간'] = pd.to_datetime(df_21['시간'])
df_21['시'] = df_21['시간'].dt.hour
df_21['시'].unique()
```

```
array([ 8, 11, 14, 17])
```

[코드 17] 분석 도구를 활용한 정확성 품질 지수 확인하기

: 품목이 'GEAR_42' 인 경우도 동일하게 실행한다. 단, 수집 시간은 각 4시간 주기로 8시, 12시, 16시, 20시에 해당하는지 확인한다.

```
# GEAR_42
df_42 = df[df['품목']=='GEAR_42']
df_42['시간'] = pd.to_datetime(df_42['시간'])
df_42['시'] = df_42['시간'].dt.hour
df_42['시'].unique()
```

```
array([ 8, 12, 16, 20])
```

[코드 18] 분석 도구를 활용한 정확성 품질 지수 확인하기

▷ [코드 17]의 결과값을 통해 'GEAR_21' 의 수집 시간이 정확한 것을 확인 할 수 있다. 또한 [코드 18]의 결과값을 통해 'GEAR_42' 의 수집 시간이 정확한 것을 확인할 수 있으므로 정확성을 위배하는 데이터는 없다고 할 수 있다.

▶ 무결성 품질 지수 = $(1 - (\text{유일성, 유효성, 일관성 지수 중 100\%가 아닌 지수 개수} / 3)) * 100$

: 본 데이터에서 유일성, 유효성과 일관성은 100%로, 무결성 품질 지수는 100%이다.

▶ 가중치지수 = 품질 지수 * 가중치

: 데이터의 특성에 따라 품질 지수의 중요도는 달라진다. 활용되는 데이터의 중요한 품질 지수가 무엇인지를 평가하고, 데이터의 특성에 따라 적절한 가중치를 부여한다.

▶ 데이터 품질 지수 = $\sum$ 가중치지수

: 최종적으로 구한 6가지 데이터 품질 지수에 적절한 가중치를 부여하여 품질 지수를 구한다. 품질 지수 가중치는 활용되는 데이터의 특성에 따라 부여할 수 있다. 중요한 품질지수가 무



엇인지를 평가하고, 데이터의 특성에 따라 가중치를 부여한다.

: 본 데이터는 열처리 공정 관련 수주량, 생산량, 재고량, 출고량 등에 대한 로그 데이터라는 특성상 하루에 평균적으로 3~4회 데이터가 기록된다. 즉, 유일성에 위반하는 데이터가 존재할 수 있는 특성이 있기에 유일성의 가중치와 품질 지수 산출 과정에 유일성을 고려하는 무결성의 가중치를 10%로 부여하여 상대적으로 적게 가중치를 둔다. 이 외에 완전성, 유효성, 일관성, 정확성은 동일하게 중요하다는 가정하에 가중치를 동일하게 각각 20%로 부여하였다.

: 제조 데이터 품질 지수 결과는 아래와 같다. 가중치를 부여하지 않고 6가지 품질 지수(완전성, 유일성, 유효성, 일관성, 정확성, 무결성)를 평균을 낸 품질 지수는 100%다. 가중치 지수는 각각의 품질 지수에 가중치를 적용한 값을 나타낸다. 본 데이터는 품질 지수가 100%이기에 가중치를 적용한 품질 지수 또한 100%이다.

구분	품질 지수	가중치	가중치지수	오류율
완전성	100%	20%	20%	0%
유일성	100%	10%	10%	0%
유효성	100%	20%	20%	0%
일관성	100%	20%	20%	0%
정확성	100%	20%	20%	0%
무결성	100%	10%	10%	0%
품질 지수	100%	100%	100%	0%(0%)



「열처리 공급망최적화 AI 데이터셋」 분석실습 가이드북